

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống hỏi đáp tài liệu kỹ thuật tiếng
Việt với tinh chỉnh embedding và chưng cất mô
hình xếp hạng lại**

TRẦN QUANG HUY

huy.tq225201@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lê Thanh Hương – Khoa Khoa học máy tính
TS. Đỗ Tiến Dũng – Khoa Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 07/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống hỏi đáp tài liệu kỹ thuật tiếng
Việt với tinh chỉnh embedding và chưng cất mô
hình xếp hạng lại**

TRẦN QUANG HUY
huy.tq225201@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lê Thanh Hương – Khoa Khoa học máy tính
TS. Đỗ Tiến Dũng – Khoa Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

Chữ ký GVHD

Chữ ký GVHD

HÀ NỘI, 07/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn sâu sắc đến PGS. TS. Lê Thanh Hương, giảng viên hướng dẫn chính của em, người đã trực tiếp định hướng, kiên nhẫn góp ý và đồng hành cùng em qua từng giai đoạn của đề án. Những chỉ dẫn chuyên môn, sự tận tình và những góp ý quý báu của cô đã giúp em nhìn nhận vấn đề rõ ràng hơn, từng bước hoàn thiện đề tài và có thêm nhiều kinh nghiệm trong quá trình nghiên cứu, xây dựng hệ thống.

Em cũng xin chân thành cảm ơn TS. Đỗ Tiến Dũng, giảng viên đồng hướng dẫn, đã dành thời gian hỗ trợ trong quá trình em thực hiện đề án.

Em xin cảm ơn các thầy cô Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội, đã truyền đạt cho em không chỉ kiến thức chuyên môn mà còn cả tư duy học tập, nghiên cứu và tinh thần làm nghề trong suốt những năm đại học. Những kiến thức và trải nghiệm được tích lũy tại trường là nền tảng quan trọng giúp em có thể thực hiện và hoàn thành đề án này.

Con xin cảm ơn bố mẹ và gia đình, những người luôn là chỗ dựa vững vàng và lặng lẽ phía sau mọi cố gắng của con. Em cũng xin cảm ơn những người anh em, bạn bè đã cùng em học tập, chia sẻ, động viên và vượt qua nhiều giai đoạn khó khăn trong suốt hành trình đại học.

Cuối cùng, em muốn cảm ơn chính mình vì đã không ngừng nỗ lực, kiên trì và kiên định để đi đến cuối hành trình. Tất cả sẽ là một trong những kỷ niệm đẹp nhất của em trong những năm tháng tuổi đôi mươi.

Em xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hệ thống hỏi–đáp dựa trên truy hồi (Retrieval-Augmented Generation, RAG) cho tài liệu kỹ thuật tiếng Việt phải đối mặt với hai thách thức chính: chất lượng truy hồi trên văn bản chuyên ngành nhiều thuật ngữ và bảng biểu, cùng chi phí và độ trễ của tầng xếp hạng lại (reranking). Các giải pháp hiện tại thường dùng mô hình cross-encoder lớn như BGE-M3 (568M tham số) để rerank và một mô hình ngôn ngữ lớn qua API để định tuyến câu hỏi; cách này cho độ chính xác cao nhưng nặng, chậm khi chạy trên CPU và phát sinh chi phí, trong khi embedding nền tảng lại chưa được tối ưu cho miền dữ liệu.

Đồ án lựa chọn hướng tối ưu toàn bộ pipeline thành các thành phần nhẹ, có thể triển khai trên CPU với chi phí thấp mà vẫn giữ chất lượng gần với mô hình lớn, thay vì phụ thuộc vào mô hình nặng hoặc dịch vụ API. Theo hướng này, embedding được fine-tune theo miền, reranker được chưng cất tri thức (knowledge distillation) xuống các mô hình nhỏ kèm cắt tỉa từ vựng (vocabulary pruning), và bộ định tuyến LLM được thay bằng một router cục bộ dựa trên Sentence-BERT/SetFit.

Cụ thể, embedding Vietnamese_Embedding_v2 (nền BGE-M3) được huấn luyện theo curriculum hai giai đoạn: giai đoạn 1 dùng hàm mất mát GIST — một biến thể của MNRL có guide model BGE-M3 lọc các false negative; giai đoạn 2 chuyển sang hàm mất mát MNRL trên các hard negative đã khai phá. Cả hai giai đoạn đều kết hợp với Matryoshka Representation Learning cho ba kích thước 256/512/1024, trong đó cấu hình 512 chiều được chọn làm chính. Reranker được chưng cất từ teacher BGE-reranker-v2-m3 sang student mmarco-mMiniLMv2 117M và một bản pruned 35.6M, sử dụng hàm mất mát ADR-MSE dựa trên thứ hạng và huấn luyện hai giai đoạn trên bộ dữ liệu dùng chung với embedding.

Kết quả, embedding 512 chiều đạt $\text{Acc}@1$ 76.43% (so với 73.89% của baseline) và $\text{MRR}@10 = 0.8042$ — cao nhất trong các mô hình so sánh trên tập tài liệu không xuất hiện trong quá trình huấn luyện. Reranker pruned 35.6M đạt $\text{MRR}@10 = 0.8718$, vượt ViRanker 568M (0.8434) dù nhỏ hơn khoảng 16 lần và nhanh hơn khoảng 5.8 lần (23.8ms so với 138.9ms), còn student 117M giữ được 99.4% MRR của teacher. Toàn bộ pipeline chạy được trên CPU, chứng minh tính khả thi của một hệ thống RAG tiếng Việt hiệu quả với chi phí thấp.

Sinh viên thực hiện

Huy

Trần Quang Huy

ABSTRACT

Retrieval-Augmented Generation (RAG) systems for Vietnamese technical documents face two main challenges: retrieval quality over specialized text dense with terminology and tables, and the cost and latency of the reranking stage. Existing solutions typically rely on a large cross-encoder such as BGE-M3 (568M parameters) for reranking and a large language model accessed through an API for query routing; this achieves high accuracy but is heavy, slow on CPU, and costly, while the underlying embedding model is not optimized for the target domain.

This thesis adopts the approach of optimizing the entire pipeline into lightweight components that can be deployed on CPU at low cost while preserving quality close to that of large models, rather than depending on heavy models or external APIs. Following this direction, the embedding model is fine-tuned for the domain, the reranker is compressed through knowledge distillation into smaller models together with vocabulary pruning, and the LLM router is replaced by a local Sentence-BERT/SetFit-based intent router.

Specifically, the embedding model `Vietnamese_Embedding_v2` (BGE-M3 backbone) is trained with a two-stage curriculum: Stage 1 uses the GIST loss — a variant of MNRL in which a BGE-M3 guide model filters false negatives — while Stage 2 switches to the MNRL loss on mined hard negatives. Both stages are combined with Matryoshka Representation Learning for three sizes (256/512/1024), with the 512-dimensional configuration chosen as the primary one. The reranker is distilled from the BGE-reranker-v2-m3 teacher into an `mmarco-mMiniLMv2` 117M student and a pruned 35.6M variant, using the rank-based ADR-MSE loss and two-stage training on data shared with the embedding pipeline.

As a result, the fine-tuned 512-dimensional embedding achieves an Acc@1 of 76.43% (compared with 73.89% for the baseline) and an MRR@10 of 0.8042 — the highest among all compared models on documents not seen during training. The pruned 35.6M reranker attains an MRR@10 of 0.8718, surpassing the 568M Vi-Ranker (0.8434) despite being about 16 times smaller and 5.8 times faster (23.8ms versus 138.9ms), while the 117M student retains 99.4% of the teacher’s MRR. The entire pipeline runs on CPU, demonstrating the feasibility of an efficient, low-cost Vietnamese RAG system.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	ix
DANH MỤC THUẬT NGỮ.....	xi
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện có và hạn chế.....	1
1.3 Mục tiêu và định hướng giải pháp	2
1.4 Đóng góp đề án	2
1.5 Bố cục đề án	3
CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN.....	4
2.1 Tổng quan về hệ thống RAG	4
2.1.1 Hạn chế của mô hình ngôn ngữ lớn thuần túy	4
2.1.2 Retrieval-Augmented Generation	4
2.1.3 Đặc thù của RAG cho tài liệu kỹ thuật tiếng Việt	5
2.2 Truy hồi thông tin.....	5
2.2.1 BM25 và truy hồi thưa (sparse retrieval)	5
2.2.2 Truy hồi dày (dense retrieval) và bi-encoder	6
2.2.3 Truy hồi lai và Reciprocal Rank Fusion.....	7
2.3 Xếp hạng lại bằng cross-encoder.....	8
2.3.1 Bi-encoder và cross-encoder	8
2.3.2 BGE-reranker-v2-m3.....	9
2.3.3 Chung cất tri thức cho reranker	9
2.3.4 Các hàm mất mát cho chung cất reranker	10
2.3.5 Nén reranker và cắt tỉa từ vựng.....	10

2.4 Mô hình nhúng văn bản và tinh chỉnh embedding	11
2.4.1 Từ Transformer đến Sentence-BERT	11
2.4.2 Mô hình embedding cho truy hồi	11
2.4.3 MultipleNegativesRankingLoss.....	12
2.4.4 False negatives và GISTEmbedLoss	12
2.4.5 Matryoshka Representation Learning.....	13
2.4.6 Học theo lộ trình cho embedding truy hồi.....	14
2.5 Mô hình ngôn ngữ lớn và thiết kế prompt trong RAG	14
2.5.1 Vai trò của LLM trong sinh câu trả lời.....	14
2.5.2 Kỹ thuật prompt Chain-of-Thought.....	15
2.6 Định tuyến truy vấn và phân loại ý định.....	15
2.6.1 Định tuyến truy vấn trong pipeline RAG	15
2.6.2 PhoBERT, SBERT và SetFit.....	15
2.6.3 Mất cân bằng lớp trong phân loại ý định	16
2.7 Các độ đo đánh giá	16
2.7.1 Độ đo cho truy hồi và xếp hạng lại	16
2.7.2 Độ đo cho phân loại ý định và pipeline	17
2.8 Khoảng trống nghiên cứu	18
CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT.....	19
3.1 Tổng quan kiến trúc hệ thống đề xuất	19
3.2 Vai trò của hệ thống v1 trong đồ án	21
3.3 Quy trình tiền xử lý tài liệu.....	21
3.3.1 Trích xuất văn bản từ tài liệu PDF	21
3.3.2 Chiến lược chia đoạn nhận biết bảng.....	22
3.3.3 Xây dựng chỉ mục	23

3.4 Kiến trúc hai pipeline RAG	25
3.4.1 Pipeline Baseline Router.....	25
3.4.2 Pipeline Router-Summary.....	26
3.4.3 So sánh kiến trúc hai pipeline.....	27
3.5 Xác định các nút thắt	27
3.5.1 Nút thắt độ trễ: BGE-reranker-v2-m3	27
3.5.2 Nút thắt chi phí: bộ định tuyến ý định GPT	28
3.5.3 Nút thắt chất lượng: embedding chưa tối ưu theo miền.....	28
3.6 Tổng quan định hướng v2.....	29
3.7 Tinh chỉnh embedding bằng MRL và học theo lộ trình	29
3.7.1 Pipeline xây dựng dữ liệu embedding	29
3.7.2 Thiết kế hàm mất mát cho tầng embedding.....	31
3.7.3 Huấn luyện theo lộ trình hai giai đoạn.....	33
3.7.4 Chiến lược triển khai với MRL.....	34
3.8 Reranker nhẹ bằng chứng cất tri thức.....	35
3.8.1 Động lực và thiết kế tổng quan.....	35
3.8.2 Điều chỉnh dữ liệu cho reranker.....	36
3.8.3 Giai đoạn A: Học tương phản.....	38
3.8.4 Giai đoạn B: Chứng cất tri thức.....	39
3.8.5 Cắt tỉa từ vựng cho mmarco	41
3.9 Tinh chỉnh bộ phân loại ý định Sentence-BERT.....	42
3.9.1 Kiến trúc và dữ liệu.....	42
3.9.2 Xử lý mất cân bằng lớp.....	44
3.9.3 Logic định tuyến trong pipeline v2	45

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM	46
4.1 Thiết lập thực nghiệm	46
4.1.1 Môi trường và phần cứng.....	46
4.1.2 Tập dữ liệu đánh giá.....	46
4.1.3 Độ đo đánh giá	47
4.2 Đánh giá reranker	47
4.2.1 Kết quả chính trên tập kiểm thử reranker.....	47
4.2.2 Nghiên cứu loại bỏ thành phần	49
4.2.3 Cắt tia từ vựng cho mmarco	51
4.2.4 Phân tích sự phụ thuộc của reranker vào độ mạnh của bộ truy hồi .	52
4.2.5 Tương tác giữa reranker và bộ truy hồi.....	52
4.2.6 So sánh các phương pháp truy hồi	53
4.2.7 Các hướng đã thử nhưng không cải thiện.....	53
4.2.8 Đánh giá đầu cuối trên câu hỏi trắc nghiệm	54
4.2.9 Phân rã nguồn lỗi đầu cuối.....	56
4.3 Đánh giá embedding	57
4.3.1 Thiết lập đánh giá	57
4.3.2 Kết quả trên hai tập kiểm thử	57
4.3.3 Phân tích số chiều biểu diễn MRL	58
4.3.4 Nghiên cứu loại bỏ curriculum và GIST	58
4.4 Đánh giá bộ phân loại ý định	59
4.5 Khuyến nghị cấu hình triển khai.....	61
4.6 Phân tích lỗi tổng thể	62
CHƯƠNG 5. KẾT LUẬN.....	64
5.1 Tổng kết	64
5.2 Đóng góp chính.....	64

5.3 Hạn chế	65
5.4 Hướng phát triển	65
TÀI LIỆU THAM KHẢO	69
PHỤ LỤC A. MÔI TRƯỜNG VÀ CẤU HÌNH HUẤN LUYỆN	70
A.1 Môi trường phần cứng và phần mềm	70
A.2 Siêu tham số huấn luyện embedding	70
A.3 Siêu tham số huấn luyện reranker	71
A.4 Siêu tham số huấn luyện bộ phân loại ý định	71
PHỤ LỤC B. VÍ DỤ DỮ LIỆU HUẤN LUYỆN	72
B.1 Ví dụ bộ ba cho embedding.....	72
B.2 Các loại negative tạo sinh	72
B.3 Ví dụ hai lớp ý định	73
PHỤ LỤC C. CÁC PROMPT SỬ DỤNG VỚI MÔ HÌNH NGÔN NGỮ .	74
C.1 Prompt định tuyến ý định.....	74
C.2 Prompt sinh câu trả lời theo ý định	74
C.3 Prompt giám khảo xác định positive	75
C.4 Prompt sinh hard negative.....	76
C.5 Prompt kiểm tra hard negative.....	77

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc tổng quan của hệ thống RAG	5
Hình 3.1	Kiến trúc tổng thể của hệ thống đề xuất	20
Hình 3.2	Luồng xử lý xây dựng chỉ mục	24
Hình 3.3	Luồng xử lý của pipeline Baseline	25
Hình 3.4	Luồng xử lý của pipeline Router-Summary	26
Hình 3.5	Quy trình xây dựng dữ liệu huấn luyện embedding	29
Hình 3.6	Hai giai đoạn huấn luyện cho tầng embedding	33
Hình 3.7	Kiến trúc huấn luyện reranker hai giai đoạn	35
Hình 3.8	Kiến trúc huấn luyện bộ phân loại ý định theo SetFit	43

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh BM25, truy hồi dày (dense) và truy hồi lai (hybrid)	7
Bảng 2.2	So sánh bi-encoder và cross-encoder	8
Bảng 2.3	Tóm tắt các nhóm hàm mất mát cho chung cất reranker . . .	10
Bảng 3.1	Vai trò của các tập dữ liệu trong quy trình phát triển	21
Bảng 3.2	So sánh kiến trúc Baseline Router và Router-Summary . . .	27
Bảng 3.3	Ánh xạ nút thất của v1 sang giải pháp v2	29
Bảng 3.4	Các nguồn negative trong bộ dữ liệu embedding (pool thô trước khi tạo bộ ba)	30
Bảng 3.5	Cấu hình huấn luyện theo lộ trình cho embedding	34
Bảng 3.6	Các chế độ triển khai từ một checkpoint MRL	34
Bảng 3.7	Thống kê negative tạo sinh sau lọc	37
Bảng 3.8	Thống kê negative khai thác theo nguồn	37
Bảng 3.9	Tổng quan tập dữ liệu huấn luyện reranker	38
Bảng 3.10	Siêu tham số của giai đoạn A	38
Bảng 3.11	Ví dụ minh họa cách tính ADR-MSE với năm ứng viên . . .	41
Bảng 3.12	Siêu tham số của giai đoạn B	41
Bảng 3.13	Siêu tham số tinh chỉnh Sentence-BERT (SetFit) cho bộ phân loại ý định	44
Bảng 4.1	Các tập dữ liệu dùng trong đánh giá	46
Bảng 4.2	Kết quả reranker trên 343 câu hỏi	48
Bảng 4.3	So sánh cấu hình huấn luyện trên MiniLM, mmarco và pruned	49
Bảng 4.4	So sánh các hàm mất mát chung cất trên MiniLM 33M (giai đoạn A no-mMARCO, giai đoạn B $\alpha = 0.7$)	49
Bảng 4.5	Ảnh hưởng của tập MMARCO-VI ở giai đoạn A trên MiniLM	50
Bảng 4.6	Ảnh hưởng của cách tổ chức curriculum và learning rate . .	51
Bảng 4.7	Nén Mmarco bằng cắt tia từ vụng	51
Bảng 4.8	Chất lượng pruned reranker sau tinh chỉnh	51
Bảng 4.9	So sánh các reranker trên bộ truy hồi mạnh	52
Bảng 4.10	Reranker trên bộ truy hồi yếu so với bộ truy hồi mạnh (MRR@10)	52
Bảng 4.11	So sánh các phương pháp truy hồi (trước rerank) và đóng góp của reranker	53
Bảng 4.12	Đánh giá đầu cuối trên 390 câu trắc nghiệm: pipeline v1 so với v2	55
Bảng 4.13	Độ trễ trung bình theo thành phần (ms/câu)	55

Bảng 4.14	Dung lượng lưu trữ của embedding cache và index trong pipeline đầu cuối	55
Bảng 4.15	Ma trận định tuyến $\times$ đáp án cuối trên 390 câu trắc nghiệm .	56
Bảng 4.16	Các tập dữ liệu đánh giá embedding	57
Bảng 4.17	Kết quả trên tập D1	57
Bảng 4.18	Kết quả trên tập D2	57
Bảng 4.19	Phân tích số chiều biểu diễn MRL trên D1 và D2	58
Bảng 4.20	Cấu hình huấn luyện embedding cuối cùng	59
Bảng 4.21	Ablation curriculum loss: GIST vs MNRL theo stage (512d)	59
Bảng 4.22	So sánh cấu hình ý định classifier theo F1-macro	60
Bảng 4.23	Đánh giá nội tại bộ phân loại ý định v2 (SetFit)	60
Bảng 4.24	So sánh bộ định tuyến v1 (GPT) và v2 (Sentence-BERT/SetFit)	61
Bảng 4.25	Các cấu hình pipeline khuyến nghị	62
Bảng A.1	Các thư viện chính sử dụng trong đồ án	70
Bảng A.2	Siêu tham số huấn luyện embedding	70
Bảng A.3	Siêu tham số huấn luyện reranker	71
Bảng A.4	Siêu tham số huấn luyện bộ phân loại ý định	71
Bảng B.1	Ví dụ một bộ ba huấn luyện embedding	72
Bảng B.2	Ví dụ bốn loại negative tạo sinh	73
Bảng B.3	Ví dụ câu hỏi hai lớp ý định	73

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
Acc	Accuracy	Độ chính xác
ADR-MSE	Approx Discounted Rank MSE	Hàm mất mát chung cất dựa trên thứ hạng (Schlatt và cộng sự, 2025)
API	Application Programming Interface	Giao diện lập trình ứng dụng
BCE	Binary Cross-Entropy	Entropy chéo nhị phân
BERT	Bidirectional Encoder Representations from Transformers	Mô hình biểu diễn ngôn ngữ hai chiều dựa trên Transformer
BGE	BAAI General Embedding	Họ mô hình embedding/reranker của BAAI
BM25	Best Matching 25	Thuật toán xếp hạng văn bản theo từ khóa
CoT	Chain-of-Thought	Chuỗi suy luận từng bước
CPU	Central Processing Unit	Bộ xử lý trung tâm
F1	F1-score	Trung bình điều hòa giữa precision và recall
GIST	Guided In-sample Selection of Training Negatives	Chọn lọc negative trong batch có hướng dẫn
GPT	Generative Pre-trained Transformer	Mô hình Transformer sinh ngôn ngữ
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
Hit	Hit rate	Tỉ lệ truy hồi có chứa đáp án đúng trong top- k
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu dạng khóa–giá trị
KD	Knowledge Distillation	Chung cất tri thức
KL	Kullback–Leibler Divergence	Độ lệch Kullback–Leibler giữa hai phân phối
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MCQ	Multiple Choice Question	Câu hỏi trắc nghiệm
mMARCO	multilingual MS MARCO	Bộ dữ liệu MS MARCO đa ngôn ngữ
MNRL	Multiple Negatives Ranking Loss	Hàm mất mát xếp hạng đa negative
MRL	Matryoshka Representation Learning	Học biểu diễn lồng nhau (Matryoshka)

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
MRR	Mean Reciprocal Rank	Nghịch đảo thứ hạng trung bình
NDCG	Normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
PDF	Portable Document Format	Định dạng tài liệu di động
Recall@k	Recall at k	Tỉ lệ đáp án đúng xuất hiện trong top- k
RAG	Retrieval-Augmented Generation	Sinh có tăng cường truy hồi
RRF	Reciprocal Rank Fusion	Hợp nhất theo thứ hạng nghịch đảo
SBERT	Sentence-BERT	Mô hình BERT tạo embedding ở mức câu
UNK	Unknown token	Token không có trong từ vựng tokenizer
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên của GPU
XLNet	XLNet	Mô hình RoBERTa đa ngôn ngữ

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích
Ablation	Thực nghiệm loại bỏ hoặc thay đổi từng thành phần để đo đóng góp của thành phần đó
Baseline	Mô hình hoặc cấu hình cơ sở dùng làm mốc so sánh
BGE-M3	Mô hình embedding/truy hồi đa ngôn ngữ thuộc họ BGE, dùng làm backbone hoặc guide model trong đề án
BGE-reranker-v2-m3	Mô hình reranker dạng cross-encoder thuộc họ BGE, dùng làm teacher cho chung cất reranker
Bi-encoder	Kiến trúc mã hóa query và tài liệu độc lập thành hai vector riêng
Candidate pool	Tập ứng viên (chunk) được truy hồi để đưa vào bước xếp hạng lại
Checkpoint	Bản lưu trọng số mô hình tại một thời điểm huấn luyện
Chunk / Chunking	Đoạn văn bản chia nhỏ từ tài liệu / quá trình chia tài liệu thành các đoạn
Corpus	Tập văn bản hoặc tập đoạn được dùng để truy hồi, huấn luyện hoặc đánh giá
Cross-encoder	Kiến trúc đưa đồng thời query và tài liệu qua mô hình để chấm điểm liên quan
Curriculum	Cách tổ chức dữ liệu hoặc hàm mất mát theo lộ trình từ dễ đến khó qua nhiều giai đoạn
Dense retrieval	Truy hồi dựa trên vector ngữ nghĩa
Embedding	Vector biểu diễn ngữ nghĩa của văn bản
Fine-tune	Tinh chỉnh mô hình tiền huấn luyện trên dữ liệu của miền cụ thể
Gold chunk	Đoạn văn bản chứa thông tin trả lời đúng cho query
Guide model	Mô hình dẫn hướng dùng để lọc false negative trong GIST
Hard negative	Negative khó: gần positive về ngữ nghĩa nhưng không trả lời đúng query
In-batch negative	Negative lấy từ các mẫu khác trong cùng một batch huấn luyện
Intent classifier	Bộ phân loại ý định câu hỏi, dùng để chọn nhánh xử lý tra cứu hoặc tính toán
Latency	Độ trễ xử lý mỗi truy vấn
Logit	Điểm số thô đầu ra của mô hình trước khi chuẩn hóa thành xác suất hoặc phân phối
Markdown	Định dạng văn bản đánh dấu nhẹ dùng để lưu cấu trúc tài liệu sau OCR/chuyển đổi

Thuật ngữ	Giải thích
PhoBERT	Mô hình BERT được tiền huấn luyện cho tiếng Việt, dùng trong backbone phân loại ý định
Pipeline	Chuỗi các bước xử lý nối tiếp của hệ thống
Prompt	Chỉ dẫn đầu vào gửi cho mô hình ngôn ngữ hoặc mô hình giám khảo
Pruned model	Mô hình đã được cắt tỉa một phần tham số hoặc từ vựng để giảm dung lượng
Query	Câu truy vấn của người dùng
Reranker	Mô hình xếp hạng lại các ứng viên đã được truy hỏi
Router	Thành phần định tuyến câu hỏi sang nhánh xử lý phù hợp trong pipeline
SetFit	Phương pháp fine-tune sentence transformer bằng học tương phản rồi huấn luyện classifier nhẹ
Sparse retrieval	Truy hỏi dựa trên trùng khớp từ khóa (ví dụ BM25)
Teacher / Student	Mô hình thầy (lớn) và trò (nhỏ) trong chung cất tri thức
Tokenizer	Bộ tách văn bản thành token trước khi đưa vào mô hình ngôn ngữ
Triplet	Mẫu huấn luyện gồm query, positive và negative dùng cho học tương phản hoặc reranking
Vocabulary pruning	Cắt tỉa từ vựng của tokenizer để giảm kích thước mô hình
Zero-shot	Đánh giá trên dữ liệu hoặc tài liệu chưa từng xuất hiện khi huấn luyện

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLM) đã mở ra hướng tiếp cận mới cho bài toán hỏi đáp tự động. Tuy nhiên, LLM thuần túy bộc lộ hạn chế rõ rệt khi cần trả lời câu hỏi về kiến thức chuyên biệt, cập nhật theo thời gian hoặc đòi hỏi tham chiếu tài liệu cụ thể — những tình huống thường xuyên gặp trong môi trường doanh nghiệp. Kiến trúc Retrieval-Augmented Generation (RAG) [1] ra đời nhằm giải quyết vấn đề này bằng cách kết hợp truy xuất thông tin từ kho tài liệu nội bộ với khả năng sinh ngôn ngữ của LLM, giúp câu trả lời bám sát nguồn và có thể kiểm chứng.

Đề án này được thực hiện trên tập dữ liệu của cuộc thi Viettel AI Race 2025, gồm 991 câu hỏi trắc nghiệm tiếng Việt trải rộng trên 200 tài liệu kỹ thuật đa lĩnh vực. Tập tài liệu có cấu trúc phức tạp — bảng biểu, ký hiệu kỹ thuật, thuật ngữ chuyên ngành — và đặt ra thách thức không nhỏ cho các bước chia đoạn (chunking), truy hồi và xếp hạng. Đây là bối cảnh thực tiễn phản ánh đúng những khó khăn khi triển khai RAG trong môi trường sản xuất thực tế, không phải trên các tập đánh giá học thuật lý tưởng hóa.

1.2 Các giải pháp hiện có và hạn chế

Các hệ thống RAG hiện nay thường kết hợp truy hồi lai (BM25 + truy hồi dày đặc + RRF), reranker dạng cross-encoder, và sinh câu trả lời bằng LLM. Đây là kiến trúc được chứng minh hiệu quả về mặt học thuật. Tuy nhiên, trong quá trình xây dựng và đánh giá hệ thống trên tập dữ liệu cuộc thi, chúng tôi quan sát thấy ba vấn đề thực tế mà các công trình hiện có chưa giải quyết triệt để:

- Reranker quá nặng cho triển khai thực tế. BGE reranker [2] (568M tham số) cho chất lượng xếp hạng cao nhưng chi phí suy luận lớn. Trên môi trường không có GPU mạnh — chẳng hạn khi triển khai trên CPU hoặc hạ tầng nội bộ hạn chế — độ trễ của mô hình tăng cao và trở thành điểm nghẽn của toàn bộ luồng xử lý. Các công trình hiện tại thường chỉ báo cáo độ chính xác mà bỏ qua chi phí suy luận, trong khi đây là yếu tố quyết định khả năng triển khai thực tế.
- Định tuyến bằng LLM lớn tốn kém không cần thiết. Luồng Router-Summary sử dụng GPT-4o-mini [3] để phân loại ý định — một tác vụ thực chất chỉ cần phân biệt hai lớp (tra_cuu / tinh_toan). Mỗi lần gọi API thêm khoảng 800ms độ trễ và phát sinh chi phí, trong khi độ chính xác phân loại đạt 99.4% —

ngưỡng này hoàn toàn có thể đạt được bằng mô hình nhỏ hơn nhiều bậc.

- Mô hình embedding đã huấn luyện sẵn chưa tối ưu cho miền. Mô hình Vietnamese Embedding v2 [4], dù là embedding mạnh nhất hiện có cho tiếng Việt, vẫn chỉ đạt $\text{Acc}@1 = 73.89\%$ trên tập kiểm thử nội bộ khi đánh giá riêng bước truy hồi. Đây là giới hạn trên của toàn luồng xử lý — nếu truy hồi không lấy đúng đoạn văn bản, các bước sau không thể cứu vãn.

1.3 Mục tiêu và định hướng giải pháp

Đề án hướng tới mục tiêu xây dựng và tối ưu hệ thống RAG tiếng Việt theo hai giai đoạn phát triển có căn cứ thực nghiệm.

Giai đoạn 1 xây dựng và đánh giá hai luồng xử lý RAG — Baseline Router và Router-Summary — trên tập 991 câu hỏi, nhằm có bức tranh định lượng rõ ràng về độ chính xác, độ trễ và các dạng lỗi thực tế trước khi quyết định tối ưu.

Giai đoạn 2 dựa trên kết quả giai đoạn 1 để đề xuất ba hướng tối ưu có chủ đích, mỗi hướng giải quyết đúng một điểm nghẽn đã xác định:

- Chung cất BGE reranker (568M tham số) sang ba reranker nhỏ hơn: MiniLM 33M, mmarco 117M và pruned mmarco 35.6M. Mục tiêu là giảm độ trễ và dung lượng trong khi duy trì chất lượng ranking.
- Huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit để thay thế GPT-4o-mini, nhằm loại bỏ chi phí API và giảm độ trễ định tuyến.
- Tinh chỉnh mô hình embedding với Matryoshka Representation Learning, Curriculum Training và GIST, nhằm cải thiện chất lượng truy hồi trên miền tài liệu kỹ thuật tiếng Việt.

1.4 Đóng góp đề án

Các đóng góp chính của đề án bao gồm:

- **[C1] Quy trình xây dựng dữ liệu huấn luyện dùng chung cho embedding và reranker.** Quy trình gồm năm giai đoạn: dùng mô hình ngôn ngữ làm giám khảo để xác định positive (gold chunk), tổng hợp hard negative theo hai nguồn (loại tạo sinh có kiểm soát và loại khai thác từ kết quả truy hồi), lọc khả năng trả lời độc lập, rồi gộp và chia tập huấn luyện. Cùng một nguồn dữ liệu được tái sử dụng cho cả hai tầng: tầng embedding dùng curriculum hai giai đoạn, còn tầng reranker áp dụng tiêu chí lọc và ghép cặp chặt hơn để tạo triplet phục vụ học xếp hạng.
- **[C2] Tinh chỉnh embedding theo curriculum kết hợp Matryoshka Representation Learning.** Áp dụng curriculum hai giai đoạn $\text{GIST} \rightarrow \text{MNRL}$ — học

ranh giới ngữ nghĩa từ negative tạo sinh trước, rồi tinh chỉnh khả năng phân biệt bằng hard negative khai thác — kết hợp MRL để tạo biểu diễn đa kích thước. Nhờ đó embedding 512 chiều đạt chất lượng gần tương đương 1024 chiều, giảm một nửa dung lượng vector index mà vẫn cải thiện độ chính xác truy hồi so với base model trên cả tập in-domain lẫn tập held-out.

- **[C3] Nén reranker bằng chưng cất tri thức kết hợp cắt tỉa từ vựng.** Chưng cất teacher BGE-reranker-v2-m3 (568M tham số) sang các student nhẹ qua quy trình hai giai đoạn (thích nghi miền bằng contrastive learning, rồi chưng cất tín hiệu xếp hạng của teacher), sau đó cắt ma trận embedding của bộ tách từ từ 250.002 xuống 36.324 token. Kết hợp hai kỹ thuật, reranker pruned 35.6M tham số (giảm khoảng 70% so với mmarco 117M) đạt $\text{MRR}@10 = 0.8718$ — giữ 99.6% chất lượng của mmarco và vượt ViRanker 568M (0.8434) dù nhỏ hơn khoảng 16 lần và nhanh hơn khoảng 5.8 lần (23.8ms so với 138.9ms).
- **[C4] Hệ thống RAG tối ưu đầu cuối.** Tích hợp embedding tinh chỉnh 512 chiều, reranker chưng cất và router cục bộ dựa trên Sentence-BERT/SetFit (thay GPT router) để giảm độ trễ, dung lượng và chi phí suy luận so với hệ thống v1, trong khi vẫn duy trì chất lượng truy hồi và xếp hạng.

1.5 Bố cục đồ án

Nội dung còn lại của đồ án được tổ chức như sau:

- Chương 2 trình bày nền tảng lý thuyết của các kỹ thuật được sử dụng, từ BM25, dense retrieval, cross-encoder reranker, đến chưng cất tri thức (knowledge distillation), MRL, Sentence-BERT/SetFit và PhoBERT.
- Chương 3 trình bày mô hình đề xuất: hệ thống RAG ban đầu (v1), quy trình tiền xử lý tài liệu, kiến trúc luồng xử lý, các điểm nghẽn quan sát được và thiết kế các thành phần tối ưu cho v2.
- Chương 4 báo cáo kết quả đánh giá thực nghiệm, bao gồm nghiên cứu loại bỏ (ablation), so sánh với các mô hình nền (baseline), đánh giá từng thành phần và đánh giá đầu cuối v1 so với v2.
- Chương 5 đưa ra kết luận, thảo luận hạn chế và định hướng phát triển.

CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN

2.1 Tổng quan về hệ thống RAG

2.1.1 Hạn chế của mô hình ngôn ngữ lớn thuần túy

Các mô hình ngôn ngữ lớn (Large Language Models – LLM) như GPT, Llama hay Qwen đã đạt kết quả tốt trong nhiều tác vụ xử lý ngôn ngữ tự nhiên. Tuy nhiên, khi áp dụng trực tiếp vào bài toán hỏi đáp trên kho tài liệu kỹ thuật nội bộ, LLM thuần túy vẫn gặp một số hạn chế căn bản.

Thứ nhất là hiện tượng *hallucination*, tức mô hình tạo ra câu trả lời nghe hợp lý nhưng không được hỗ trợ bởi nguồn tài liệu chính xác. Nguyên nhân là LLM sinh văn bản dựa trên xác suất của token tiếp theo, trong khi không có cơ chế mặc định để kiểm chứng câu trả lời với tài liệu bên ngoài. Với tài liệu kỹ thuật, nơi các câu trả lời thường liên quan tới thông số, điều kiện vận hành, đơn vị đo hoặc quy trình cụ thể, *hallucination* có thể làm giảm đáng kể độ tin cậy của hệ thống.

Thứ hai là giới hạn về tri thức. Tri thức của LLM bị cố định tại thời điểm huấn luyện và không tự động cập nhật theo tài liệu mới. Nếu tổ chức thay đổi quy trình, bổ sung phiên bản tài liệu mới hoặc cập nhật thông số thiết bị, LLM thuần túy không thể biết các thay đổi này nếu không được tinh chỉnh lại hoặc không có cơ chế truy cập tài liệu ngoài.

Thứ ba là khoảng cách miền dữ liệu (*domain gap*). Dữ liệu huấn luyện của LLM thường có tính tổng quát, trong khi tài liệu kỹ thuật nội bộ có nhiều thuật ngữ chuyên ngành, bảng biểu, ký hiệu, mã tài liệu và cách diễn đạt đặc thù. Vì vậy, LLM có thể hiểu tốt ngôn ngữ tự nhiên nói chung nhưng vẫn trả lời thiếu chính xác khi câu hỏi yêu cầu thông tin rất cụ thể trong một corpus hẹp.

2.1.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation – RAG [1] là hướng tiếp cận kết hợp truy hồi thông tin với khả năng sinh ngôn ngữ của LLM. Thay vì yêu cầu LLM trả lời hoàn toàn dựa trên tri thức trong tham số mô hình, RAG truy hồi các đoạn tài liệu liên quan trước, sau đó đưa các đoạn này vào prompt để LLM sinh câu trả lời dựa trên ngữ cảnh được cung cấp.

Một hệ thống RAG thường gồm hai giai đoạn chính:

- **Giai đoạn offline:** tài liệu được tiền xử lý, chia thành các đoạn nhỏ gọi là đoạn, mã hóa thành vector embedding và lưu trong chỉ mục truy hồi.
- **Giai đoạn online:** hệ thống nhận truy vấn từ người dùng, truy hồi các đoạn

liên quan, có thể xếp hạng lại các đoạn này, sau đó đưa top đoạn vào prompt cho LLM sinh câu trả lời.

Kiến trúc tổng quát của RAG có thể được mô tả như sau:

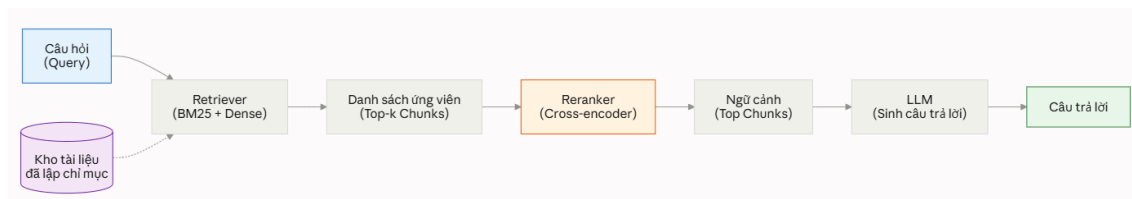


Figure 2.1: Kiến trúc tổng quát của hệ thống RAG

Trong pipeline này, chất lượng câu trả lời cuối phụ thuộc mạnh vào chất lượng của truy hồi và xếp hạng lại. Nếu đoạn chứa thông tin cần thiết không xuất hiện trong tập ứng viên, LLM gần như không có cơ sở để trả lời đúng. Ngược lại, nếu tập ứng viên đã chứa gold đoạn nhưng thứ hạng chưa tốt, reranker có thể cải thiện thứ tự trước khi đưa ngữ cảnh vào LLM. Vì vậy, trong các hệ thống RAG thực tế, truy hồi và xếp hạng lại thường là hai thành phần quan trọng cần tối ưu trước khi tối ưu prompt hoặc LLM.

2.1.3 Đặc thù của RAG cho tài liệu kỹ thuật tiếng Việt

Bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt có một số đặc thù so với RAG trên văn bản tổng quát. Thứ nhất, nhiều câu hỏi yêu cầu truy hồi chính xác một giá trị, một điều kiện hoặc một thông số nằm trong bảng biểu. Thứ hai, tài liệu có thể chứa nhiều ký hiệu, mã thiết bị, tên module hoặc định danh tài liệu, khiến khớp từ khóa chính xác vẫn giữ vai trò quan trọng. Thứ ba, cùng một chủ đề kỹ thuật có thể xuất hiện ở nhiều đoạn khác nhau, làm tăng nguy cơ false negatives khi huấn luyện truy hồi model. Thứ tư, hệ thống cần cân bằng giữa chất lượng trả lời, độ trễ và chi phí triển khai, đặc biệt khi chạy trên CPU hoặc môi trường tài nguyên hạn chế.

Các đặc thù này dẫn đến lựa chọn kiến trúc trong đề án: sử dụng hybrid truy hồi để kết hợp sparse và dense truy hồi, dùng cross-encoder reranker để cải thiện thứ hạng, sau đó tối ưu các thành phần nặng bằng tinh chỉnh, chưng cất và nén mô hình.

2.2 Truy hồi thông tin

2.2.1 BM25 và truy hồi thưa (sparse retrieval)

BM25 (Best Matching 25) là một mô hình truy hồi dựa trên thống kê từ khóa, được sử dụng rộng rãi như một baseline mạnh trong information truy hồi [5]. BM25 thuộc nhóm sparse truy hồi vì tài liệu và truy vấn được biểu diễn thông qua các term rời rạc thay vì vector dày đặc.

Với truy vấn q gồm các term q_i và tài liệu d , điểm BM25 được tính như sau:

$$\text{BM25}(d, q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (2.1)$$

Trong đó, $f(q_i, d)$ là tần suất xuất hiện của term q_i trong tài liệu d , $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình của các tài liệu trong corpus, k_1 điều khiển mức bão hòa của term frequency và b điều khiển mức chuẩn hóa theo độ dài tài liệu.

Thành phần IDF được dùng để tăng trọng số cho các term hiếm và giảm trọng số cho các term phổ biến:

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}, \quad (2.2)$$

trong đó N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa term q_i .

Ưu điểm của BM25 là đơn giản, nhanh, không cần GPU và hoạt động tốt với các truy vấn có chứa từ khóa chính xác, ví dụ mã tài liệu, tên thiết bị, ký hiệu kỹ thuật hoặc giá trị cụ thể. Tuy nhiên, BM25 không nắm bắt đầy đủ quan hệ ngữ nghĩa. Nếu truy vấn và tài liệu diễn đạt cùng một ý bằng các từ khác nhau, BM25 có thể xếp hạng thấp tài liệu liên quan.

2.2.2 Truy hồi dày (dense retrieval) và bi-encoder

Dense truy hồi sử dụng các mô hình neural embedding để mã hóa truy vấn và document thành các vector dày đặc trong cùng một không gian ngữ nghĩa. Kiến trúc phổ biến nhất là bi-encoder, trong đó truy vấn và document được mã hóa độc lập:

$$\mathbf{e}_q = E_q(q), \quad \mathbf{e}_d = E_d(d). \quad (2.3)$$

Độ liên quan giữa truy vấn và document thường được tính bằng cosine similarity:

$$s(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (2.4)$$

Trong nhiều hệ thống, E_q và E_d dùng chung trọng số. Khi các vector được chuẩn hóa L2, cosine similarity tương đương với tích vô hướng giữa hai vector. Ưu điểm lớn của bi-encoder là document embeddings có thể được tính trước ở giai đoạn offline. Khi suy luận, hệ thống chỉ cần mã hóa truy vấn một lần rồi tìm các vector

gần nhất trong vector database hoặc approximate nearest neighbor index.

So với BM25, dense truy hồi có khả năng bắt quan hệ ngữ nghĩa tốt hơn. Ví dụ, truy vấn “thiết bị chịu được nhiệt độ bao nhiêu” có thể gần với đoạn “nhiệt độ vận hành tối đa là 85°C” dù hai câu không trùng khớp hoàn toàn về mặt từ vựng. Hạn chế của dense truy hồi là chất lượng phụ thuộc mạnh vào mô hình embedding. Nếu model chưa được tối ưu cho miền tài liệu kỹ thuật, các đoạn liên quan có thể không được xếp đủ cao trong top-k.

2.2.3 Truy hồi lai và Reciprocal Rank Fusion

Sparse truy hồi và dense truy hồi có tính bổ sung. BM25 mạnh với khớp chính xác, trong khi dense truy hồi mạnh với khớp ngữ nghĩa. Hybrid truy hồi kết hợp cả hai hướng để tăng khả năng đưa gold đoạn vào tập ứng viên.

Một khó khăn khi kết hợp là điểm số của hai retriever nằm trên các thang đo khác nhau. Điểm BM25 không bị chặn trên, còn cosine similarity thường nằm trong khoảng $[-1, 1]$. Việc cộng trực tiếp hai loại điểm có thể khiến kết quả bị chi phối bởi retriever có scale lớn hơn.

Reciprocal Rank Fusion – RRF [6] giải quyết vấn đề này bằng cách kết hợp dựa trên thứ hạng thay vì điểm số tuyệt đối:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}. \quad (2.5)$$

Trong đó, R là tập các retriever, $\text{rank}_r(d)$ là thứ hạng của tài liệu d theo retriever r , và k là hằng số làm mượt, thường được đặt bằng 60. Một tài liệu xuất hiện ở thứ hạng cao trong nhiều retriever sẽ nhận điểm RRF cao hơn. Cách này giúp hybrid truy hồi ổn định hơn khi các retriever có scale điểm khác nhau.

Table 2.1: So sánh BM25, truy hồi dày (dense) và truy hồi lai (hybrid)

Đặc điểm	BM25	Dense truy hồi	Hybrid truy hồi
Biểu diễn	Từ khóa sparse	Vector embedding	Kết hợp nhiều retriever
Điểm mạnh	Khớp chính xác, mã tài liệu, thuật ngữ kỹ thuật	Ngữ nghĩa, diễn đạt tương đương	Tăng recall tổng thể
Điểm yếu	Kém khi khác cách diễn đạt	Phụ thuộc mô hình embedding	Cần cơ chế fusion
Vai trò trong RAG	First-stage truy hồi	First-stage truy hồi	Candidate generation

2.3 Xếp hạng lại bằng cross-encoder

2.3.1 Bi-encoder và cross-encoder

Trong truy hồi pipeline, bi-encoder phù hợp cho truy hồi nhanh trên corpus lớn, nhưng do truy vấn và document được mã hóa độc lập, mô hình có thể bỏ sót các tương tác chi tiết giữa token của truy vấn và token của document. Cross-encoder khắc phục hạn chế này bằng cách nhận đồng thời truy vấn và document làm đầu vào.

Với một cặp (q, d) , input của cross-encoder thường có dạng:

$$[\text{CLS}] \ q \ [\text{SEP}] \ d \ [\text{SEP}]. \quad (2.6)$$

Transformer encoder xử lý chuỗi ghép này và cho phép cross-attention đầy đủ giữa truy vấn và document. Điểm liên quan được sinh từ hidden state của token $[\text{CLS}]$ hoặc một pooling layer:

$$s(q, d) = \text{Linear}(\mathbf{h}_{[\text{CLS}]}) . \quad (2.7)$$

Nhờ cross-attention, cross-encoder thường cho chất lượng ranking cao hơn bi-encoder. Ví dụ, token “tối đa” trong truy vấn có thể attend trực tiếp tới cụm “85°C” trong document, giúp mô hình đánh giá liên quan chính xác hơn. Hạn chế là cross-encoder không thể tính trước document embeddings. Với mỗi truy vấn mới, mô hình phải chạy lại trên từng cặp truy vấn–ứng viên. Do đó, độ trễ của cross-encoder tăng gần tuyến tính theo số ứng viên cần xếp hạng lại.

Trong pipeline RAG thực tế, kiến trúc thường dùng là hai tầng: bi-encoder hoặc hybrid truy hồi tạo tập ứng viên nhanh, sau đó cross-encoder xếp hạng lại một số lượng nhỏ ứng viên như top-20 hoặc top-50.

Table 2.2: So sánh bi-encoder và cross-encoder

Đặc điểm	Bi-encoder	Cross-encoder
Cách mã hóa	Query và document độc lập	Query và document cùng một input
Tính trước document	Có	Không
Tốc độ trên corpus lớn	Nhanh	Chậm nếu tập ứng viên lớn
Khả năng tương tác token	Hạn chế	Đầy đủ qua cross-attention
Vai trò phổ biến	First-stage truy hồi	Second-stage xếp hạng lại

2.3.2 BGE-reranker-v2-m3

Trong đồ án, teacher reranker là BGE-reranker-v2-m3 [2]. Đây là mô hình cross-encoder thuộc họ BGE reranker, nhận cặp truy vấn–ứng viên và trả về một liên quan score dạng logit. Logit này là điểm số thô, không phải xác suất liên quan đã chuẩn hóa trong khoảng $[0, 1]$.

Với một truy vấn q và danh sách n ứng viên đoạn $\{c_1, \dots, c_n\}$, teacher tạo vector điểm:

$$\mathbf{s}^T = [s_1^T, s_2^T, \dots, s_n^T], \quad s_i^T = f_T(q, c_i). \quad (2.8)$$

Các ứng viên được sắp xếp theo s_i^T giảm dần để tạo thứ hạng teacher. Trong quá trình chưng cất, student cần học lại tín hiệu xếp hạng này bằng một mô hình nhỏ hơn. Việc phân biệt rõ BGE-reranker-v2-m3 với BGE-M3 embedding là cần thiết: BGE-M3 thường được dùng để chỉ mô hình embedding/truy hỏi đa ngôn ngữ, còn BGE-reranker-v2-m3 là cross-encoder reranker.

2.3.3 Chưng cất tri thức cho reranker

Knowledge Distillation – KD [7] là kỹ thuật chuyển tri thức từ teacher model lớn sang student model nhỏ hơn. Trong bài toán xếp hạng lại, teacher thường là một cross-encoder mạnh, còn student là cross-encoder nhẹ hơn. Thay vì chỉ học hard label relevant/irrelevant, student học từ điểm số hoặc thứ hạng do teacher tạo ra.

Cơ chế KD tổng quát có thể được viết dưới dạng:

$$\mathcal{L}_{\text{KD}} = D(g(\mathbf{s}^T), g(\mathbf{s}^S)), \quad (2.9)$$

trong đó $\mathbf{s}^T$ và $\mathbf{s}^S$ lần lượt là vector logits của teacher và student trên cùng ứng viên list, $g(\cdot)$ là phép chuẩn hóa hoặc biến đổi tín hiệu, còn $D(\cdot, \cdot)$ là hàm đo khoảng cách giữa hai tín hiệu.

Tùy theo cách sử dụng teacher signal, các loss cho reranker chưng cất có thể chia thành ba nhóm chính:

- **Listwise:** học phân phối điểm trên toàn bộ ứng viên list.
- **Pairwise:** học quan hệ ưu tiên giữa từng cặp ứng viên.
- **Rank-based:** học trực tiếp từ thứ hạng, giảm phụ thuộc vào scale logit của teacher.

2.3.4 Các hàm mất mát cho chung cất reranker

Khi chung cất reranker, lựa chọn hàm mất mát quyết định cách student tiếp nhận tín hiệu của teacher. Các phương pháp phổ biến khác nhau ở chỗ chúng khai thác *loại tín hiệu* nào từ teacher: điểm thô (logit), chênh lệch điểm giữa các cặp, hay thứ hạng của các ứng viên.

- **Listwise KL Divergence** chuyển logit của teacher và student thành hai phân phối mềm trên toàn bộ danh sách ứng viên (bằng softmax với nhiệt độ τ) rồi buộc phân phối của student tiến gần phân phối của teacher. Cách này học ưu tiên tương đối trên cả danh sách nhưng nhạy với nhiễu và nhiệt độ.
- **Margin-MSE** [8] cho student khớp *chênh lệch điểm* giữa các cặp ứng viên thay vì điểm tuyệt đối, phù hợp khi thứ tự tương đối quan trọng hơn giá trị điểm; tuy nhiên dễ bị chi phối bởi các cặp dễ có margin lớn.
- **RankNet** [9] mô hình hóa xác suất một ứng viên được ưu tiên hơn ứng viên khác qua hàm sigmoid của hiệu điểm, tạo tín hiệu theo từng cặp phong phú nhưng vẫn phụ thuộc vào thang điểm của teacher.
- **ADR-MSE**, kế thừa từ Rank-DistILLM [10], chỉ dùng *thứ hạng* của ứng viên (ánh xạ về một khoảng cố định) làm mục tiêu, nên không buộc student khớp thang điểm tuyệt đối của teacher — giảm được ảnh hưởng của chênh lệch thang điểm giữa hai mô hình.

Công thức chi tiết của từng hàm mất mát và lựa chọn sử dụng trong đồ án được trình bày ở mục 3.8.4. Bảng 2.3 tóm tắt khác biệt cốt lõi giữa bốn phương pháp.

Table 2.3: Tóm tắt các nhóm hàm mất mát cho chung cất reranker

Hàm mất mát	Tín hiệu teacher	Đặc điểm chính
Listwise KL	Phân phối softmax trên danh sách ứng viên	Học toàn bộ danh sách, nhạy với câu hỏi nhiễu và nhiệt độ
Margin-MSE	Chênh lệch logit giữa hai ứng viên	Giữ thông tin khoảng cách tương đối, có thể chịu ảnh hưởng bởi margin lớn
RankNet	Xác suất ưu tiên theo từng cặp	Tín hiệu theo cặp phong phú, số cặp tăng theo $O(n^2)$
ADR-MSE	Vị trí xếp hạng của teacher	Ít phụ thuộc vào thang điểm tuyệt đối của teacher

2.3.5 Nén reranker và cắt tỉa từ vựng

Reranker cross-encoder thường có chất lượng cao nhưng chi phí suy luận lớn. Để triển khai thực tế, đặc biệt trên CPU, cần giảm kích thước và độ trễ của reranker. Hai hướng phổ biến là knowledge chung cất và cắt tỉa.

Knowledge chung cất giảm chi phí bằng cách huấn luyện student nhỏ hơn bất chước teacher lớn. Student có thể có ít layer hơn, hidden size nhỏ hơn hoặc kiến trúc nhẹ hơn. Nếu dữ liệu chung cất đúng miền, student có thể học được hành vi ranking quan trọng của teacher mà không cần giữ toàn bộ capacity.

Vocabulary cắt tĩa (cắt tĩa từ vựng) là một dạng nén khác, tập trung vào embedding matrix của mô hình ngôn ngữ. Với các model đa ngôn ngữ, tokenizer thường chứa số lượng token rất lớn để hỗ trợ nhiều ngôn ngữ. Khi triển khai trên một miền hẹp, chẳng hạn tài liệu kỹ thuật tiếng Việt, chỉ một phần nhỏ từ vựng được sử dụng thường xuyên. Cắt bỏ các token không cần thiết giúp giảm số tham số ở embedding matrix và giảm dung lượng lưu trữ. Tuy nhiên, cắt tĩa từ vựng thường không làm giảm đáng kể độ trễ của Transformer encoder nếu số layer và hidden size được giữ nguyên.

Trong đồ án, từ vựng cắt tĩa được xem như một hướng nén bổ sung cho chung cất: student đã học ranking từ teacher, sau đó giảm từ vựng để giảm dung lượng mô hình mà vẫn giữ năng lực xử lý tiếng Việt trong miền đích.

2.4 Mô hình nhúng văn bản và tinh chỉnh embedding

2.4.1 Từ Transformer đến Sentence-BERT

Phần lớn các mô hình embedding và reranker hiện đại đều dựa trên kiến trúc Transformer [11], với cơ chế self-attention cho phép mỗi token tham chiếu đến toàn bộ các token khác trong chuỗi để nắm bắt quan hệ ngữ cảnh. BERT [12] kế thừa Transformer encoder hai chiều và được tiền huấn luyện trên khối dữ liệu lớn, tạo ra biểu diễn ngữ cảnh mạnh cho nhiều tác vụ xử lý ngôn ngữ. PhoBERT [13] là biến thể của BERT cho tiếng Việt, huấn luyện trên ngữ liệu tiếng Việt và yêu cầu đầu vào đã tách từ — là điểm cần lưu ý khi xử lý dữ liệu trong đồ án.

Tuy nhiên, BERT gốc không tạo ra biểu diễn câu phù hợp để so khớp trực tiếp bằng độ tương đồng. Sentence-BERT [14] khắc phục hạn chế này bằng cách tinh chỉnh BERT để sinh ra vector biểu diễn cho cả câu, cho phép đo mức tương đồng giữa hai câu bằng cosine similarity một cách hiệu quả. Đây chính là nền tảng cho hai thành phần của đồ án: mô hình embedding dùng để truy hồi (mục 2.4.2) và bộ phân loại ý định dựa trên embedding câu (mục 2.6.2).

2.4.2 Mô hình embedding cho truy hồi

Trong RAG, mô hình embedding có nhiệm vụ ánh xạ truy vấn và đoạn vào cùng một không gian vector sao cho các cặp liên quan nằm gần nhau. Với corpus tiếng Việt, mô hình embedding cần xử lý tốt cả ngôn ngữ tự nhiên, thuật ngữ kỹ thuật, ký hiệu, bảng biểu và cách diễn đạt đặc thù.

Các mô hình như BGE-M3 [15] và các biến thể đã tinh chỉnh cho tiếng Việt hỗ trợ dense truy hồi đa ngôn ngữ. Trong đồ án, AITeamVN/Vietnamese_Embedding_v2 được sử dụng làm base mô hình embedding. Mô hình này dựa trên BGE-M3 và tạo vector 1024 chiều. Các chương sau trình bày cách tinh chỉnh mô hình này để phù hợp hơn với miền tài liệu kỹ thuật, đồng thời hỗ trợ triển khai embedding 512 chiều thông qua Matryoshka Representation Learning.

2.4.3 MultipleNegativesRankingLoss

MultipleNegativesRankingLoss – MNRL [16] là một loss phổ biến cho contrastive learning trong sentence embedding. Với batch gồm N cặp truy vấn–positive $\{(q_i, p_i)\}_{i=1}^N$, positive của các truy vấn khác được xem như in-batch negatives đối với truy vấn q_i .

Loss được tính như sau:

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}. \quad (2.10)$$

Trong đó, τ là temperature. Nếu dùng scale γ , hai cách viết tương đương qua quan hệ $\gamma = 1/\tau$. MNRL có ưu điểm là tận dụng được nhiều negatives trong một batch mà không cần gán nhãn negative tường minh. Tuy nhiên, giả định “mọi positive khác trong batch đều là negative” có thể không đúng trong corpus kỹ thuật, nơi nhiều đoạn khác nhau cùng liên quan tới một chủ đề.

2.4.4 False negatives và GISTEmbedLoss

False negative xảy ra khi một mẫu bị xem là negative trong quá trình huấn luyện nhưng thực tế vẫn liên quan tới truy vấn. Trong MNRL, false negatives có thể xuất hiện vì positive của truy vấn khác trong cùng batch được dùng làm negative. Với tài liệu kỹ thuật, hiện tượng này khá phổ biến do nhiều đoạn cùng mô tả một thiết bị, một điều kiện vận hành hoặc các phần khác nhau của cùng một bảng thông số.

GISTEmbedLoss – Guided In-sample Selection of Training Negatives [17] giảm ảnh hưởng của false negatives bằng cách sử dụng một guide model mạnh để lọc các in-batch negatives đáng ngờ. Với truy vấn q_i , positive p_i và một ứng viên p_j trong batch, guide model tính độ tương đồng $\text{sim}_g(q_i, p_j)$. Candidate p_j được giữ lại trong mẫu số của loss nếu nó không bị guide đánh giá là tương đồng với truy vấn ngang bằng hoặc hơn positive tương ứng.

Một cách biểu diễn mask là:

$$m_{ij} = \begin{cases} 1, & \text{nếu } j = i, \\ 1, & \text{nếu } \text{sim}_g(q_i, p_j) < \text{sim}_g(q_i, p_i) - \delta, \\ 0, & \text{ngược lại.} \end{cases} \quad (2.11)$$

Trong đó, $m_{ij} = 1$ nghĩa là ứng viên được giữ lại trong loss, còn $m_{ij} = 0$ nghĩa là ứng viên bị loại khỏi tập negative. Khi $\delta = 0$, các ứng viên có độ tương đồng với truy vấn lớn hơn hoặc bằng positive sẽ bị loại khỏi negative set.

Loss có dạng:

$$\mathcal{L}_{\text{GIST}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j:m_{ij}=1} \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}. \quad (2.12)$$

So với MNRL, điểm khác biệt chính là mẫu số chỉ chứa positive và các negatives không bị guide loại bỏ. Nhờ đó, mô hình giảm nguy cơ bị phạt khi xếp hạng cao một đoạn thực chất vẫn liên quan tới truy vấn.

2.4.5 Matryoshka Representation Learning

Matryoshka Representation Learning – MRL [18] huấn luyện mô hình embedding sao cho các prefix của vector embedding cũng là các biểu diễn hữu ích. Ý tưởng này lấy cảm hứng từ búp bê Matryoshka: vector đầy đủ chứa bên trong các vector nhỏ hơn nhưng vẫn có khả năng sử dụng độc lập.

Giả sử embedding đầy đủ có chiều D , MRL chọn một tập kích thước:

$$\mathcal{M} = \{m_1, m_2, \dots, m_K\}, \quad m_K = D. \quad (2.13)$$

Với embedding $\mathbf{e}$, prefix m chiều được ký hiệu:

$$f_m(\mathbf{e}) = \mathbf{e}[:m]. \quad (2.14)$$

Loss MRL được tính bằng tổng có trọng số của loss truy hồi trên nhiều kích thước:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} w_m \mathcal{L}(f_m(\mathbf{e}_q), f_m(\mathbf{e}_p)). \quad (2.15)$$

Nếu $\mathcal{L}$ là MNRL hoặc GISTEmbedLoss, mô hình sẽ đồng thời tối ưu khả năng truy hồi ở nhiều kích thước embedding. Lợi ích thực tiễn là cùng một checkpoint

có thể dùng cho nhiều cấu hình triển khai. Khi cần giảm dung lượng vector index hoặc tăng tốc tìm kiếm, hệ thống có thể dùng embedding prefix ngắn hơn mà không cần huấn luyện lại model từ đầu.

2.4.6 Học theo lộ trình cho embedding truy hồi

Curriculum learning [19] là chiến lược tổ chức dữ liệu huấn luyện theo độ khó tăng dần. Thay vì đưa toàn bộ dữ liệu khó vào ngay từ đầu, mô hình được học từ các mẫu rõ ràng hơn trước, sau đó chuyển dần sang các mẫu khó hơn.

Trong truy hồi embedding, độ khó của negative có thể đến từ mức độ gần ngữ nghĩa với positive. Các negative quá dễ giúp model học ranh giới cơ bản nhưng ít đóng góp cho khả năng phân biệt tinh vi. Ngược lại, hard negatives gần positive hơn và sát lỗi thực tế của bộ truy hồi hơn, nhưng nếu dùng quá sớm hoặc chứa nhiều, chúng có thể làm quá trình huấn luyện kém ổn định.

Với tài liệu kỹ thuật, curriculum learning phù hợp vì có thể tách dữ liệu thành nhiều nguồn:

- **Synthesized negatives (Mẫu âm tổng hợp):** negative được tạo có kiểm soát bằng cách thay đổi thực thể, số liệu, điều kiện hoặc logic trong positive đoạn.
- **Mined hard negatives (Mẫu âm khó khai thác):** các đoạn thật nằm gần gold trong kết quả truy hồi/tái xếp hạng nhưng không trả lời đúng truy vấn.

Một curriculum hợp lý có thể dùng tạo sinh negatives ở stage đầu để học ranh giới cơ bản, sau đó dùng khai thác hard negatives ở stage sau để cải thiện khả năng phân biệt trong miền đích. Khi kết hợp với MRL, mỗi stage có thể tối ưu đồng thời nhiều kích thước embedding.

2.5 Mô hình ngôn ngữ lớn và thiết kế prompt trong RAG

2.5.1 Vai trò của LLM trong sinh câu trả lời

Trong hệ thống RAG, LLM thường đảm nhận vai trò sinh câu trả lời cuối cùng từ ngữ cảnh đã được truy hồi. Prompt đầu vào thường gồm câu hỏi, các đoạn liên quan và yêu cầu mô hình chỉ trả lời dựa trên thông tin được cung cấp. Cấu trúc prompt có ảnh hưởng lớn tới độ chính xác, đặc biệt trong bài toán trắc nghiệm hoặc bài toán yêu cầu trả lời ngắn.

Nếu bộ truy hồi cung cấp ngữ cảnh đúng, LLM có thể tổng hợp thông tin và sinh câu trả lời tự nhiên. Tuy nhiên, nếu ngữ cảnh thiếu hoặc chứa nhiều đoạn nhiễu, LLM có thể chọn sai thông tin. Vì vậy, trong RAG, việc tối ưu giai đoạn sinh câu trả lời cần đi kèm với tối ưu truy hồi và tái xếp hạng.

2.5.2 Kỹ thuật prompt Chain-of-Thought

Chain-of-Thought – CoT prompting [20] là kỹ thuật yêu cầu LLM thực hiện các bước suy luận trung gian trước khi đưa ra đáp án cuối. CoT đặc biệt hữu ích với các câu hỏi cần tính toán, so sánh nhiều dữ kiện hoặc suy luận qua nhiều bước.

Trong bài toán hỏi đáp tài liệu kỹ thuật, có thể phân biệt hai nhóm câu hỏi:

- `tra_cuu`: câu hỏi yêu cầu lấy thông tin có sẵn trong tài liệu.
- `tinh_toan`: câu hỏi yêu cầu tính toán hoặc suy luận để tạo ra kết quả mới.

Với câu hỏi `tra_cuu`, prompt trực tiếp thường đủ và có độ trễ thấp hơn. Với câu hỏi `tinh_toan`, CoT giúp LLM trích xuất dữ kiện, lập công thức và tính toán theo từng bước. Điều này tạo động lực cho thành phần định tuyến truy vấn: hệ thống cần xác định ý định của câu hỏi trước khi chọn prompt phù hợp.

2.6 Định tuyến truy vấn và phân loại ý định

2.6.1 Định tuyến truy vấn trong pipeline RAG

Định tuyến truy vấn là bước phân tích câu hỏi đầu vào để chọn chiến lược xử lý phù hợp. Trong một pipeline RAG đơn giản, mọi câu hỏi có thể đi qua cùng một luồng xử lý. Tuy nhiên, trong thực tế, các câu hỏi khác nhau có thể cần prompt, số lượng ngữ cảnh, công cụ hoặc chiến lược suy luận khác nhau.

Với bài toán của đồ án, định tuyến tập trung vào hai ý định chính: `tra_cuu`, `tinh_toan`. Câu hỏi `tra_cuu` cần tìm đúng thông tin có sẵn trong tài liệu. Câu hỏi `tinh_toan` cần lấy dữ kiện từ tài liệu rồi thực hiện phép tính hoặc suy luận. Nếu route sai, hệ thống có thể dùng prompt không phù hợp: câu hỏi tính toán bị trả lời trực tiếp, hoặc câu hỏi tra cứu đơn giản lại bị xử lý qua pipeline reasoning dài không cần thiết.

2.6.2 PhoBERT, SBERT và SetFit

PhoBERT là mô hình BERT cho tiếng Việt, được huấn luyện trên corpus tiếng Việt và thường được sử dụng cho các tác vụ phân loại văn bản. Với phân loại truyền thống, một đầu phân loại được đặt trên hidden state của token [CLS]:

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}). \quad (2.16)$$

Cách này hiệu quả khi có đủ dữ liệu huấn luyện cân bằng. Tuy nhiên, với bài toán có ít mẫu ở lớp thiểu số, tinh chỉnh trực tiếp bằng cross-entropy có thể kém ổn định.

SetFit [21] là một phương pháp tinh chỉnh sentence transformer trong điều kiện

ít dữ liệu. Thay vì chỉ huấn luyện đầu phân loại, SetFit trước hết tinh chỉnh encoder bằng contrastive learning trên các cặp câu. Sau đó, một classifier nhẹ như Logistic Regression được huấn luyện trên embedding câu.

Với một cặp câu (x_a, x_b) và nhãn tương đồng $y_{ab} \in \{0, 1\}$, CosineSimilarityLoss có thể được viết dưới dạng:

$$\mathcal{L}_{\cos} = (\cos(\mathbf{e}_a, \mathbf{e}_b) - y_{ab})^2. \quad (2.17)$$

Các cặp cùng ý định được gán target 1, còn các cặp khác ý định được gán target 0. Sau tinh chỉnh, embedding của các câu cùng ý định có xu hướng gần nhau hơn, giúp classifier tuyến tính học ranh giới tốt hơn.

2.6.3 Mất cân bằng lớp trong phân loại ý định

Trong phân loại ý định, mất cân bằng lớp là vấn đề quan trọng nếu một lớp xuất hiện ít hơn nhiều so với lớp còn lại. Nếu chỉ tối ưu độ chính xác, mô hình có thể ưu tiên lớp đa số và bỏ qua lớp thiểu số. Với bài toán RAG, lỗi ở lớp thiểu số vẫn có thể gây ảnh hưởng lớn vì câu hỏi tinh_toan thường cần pipeline xử lý khác.

Một số chiến lược thường dùng để giảm ảnh hưởng của mất cân bằng lớp gồm:

- dùng chia phân tầng (stratified split) hoặc stratified cross-validation để giữ tỉ lệ lớp trong các fold;
- dùng balanced sampling hoặc oversampling khi tạo cặp huấn luyện;
- sử dụng macro-F1 thay vì chỉ độ chính xác để đánh giá công bằng hơn giữa các lớp;
- phân tích ma trận nhầm lẫn để xem mô hình nhầm theo hướng nào.

Trong bối cảnh SetFit, việc lấy mẫu cặp cân bằng giúp lớp thiểu số xuất hiện đủ trong cả cặp dương và cặp âm, từ đó cải thiện tín hiệu contrastive cho ý định hiếm.

2.7 Các độ đo đánh giá

2.7.1 Độ đo cho truy hồi và xếp hạng lại

Các hệ thống truy hồi và xếp hạng lại thường được đánh giá bằng các metric dựa trên thứ hạng. Gọi r_q là vị trí của gold đoạn đầu tiên trong danh sách kết quả của truy vấn q .

Hit@k.

Hit@k kiểm tra liệu có ít nhất một gold đoạn nằm trong top- k hay không:

$$\text{Hit@k}(q) = \begin{cases} 1, & r_q \leq k, \\ 0, & \text{ngược lại.} \end{cases} \quad (2.18)$$

Metric này đặc biệt quan trọng với RAG vì nếu gold đoạn không xuất hiện trong top- k ngữ cảnh, LLM khó có thể trả lời đúng.

MRR@k.

Mean Reciprocal Rank đo vị trí của gold đoạn đầu tiên:

$$\text{MRR@k} = \frac{1}{|Q|} \sum_{q \in Q} \begin{cases} \frac{1}{r_q}, & r_q \leq k, \\ 0, & r_q > k. \end{cases} \quad (2.19)$$

MRR nhạy với vị trí top đầu: đưa gold từ rank 5 lên rank 1 làm tăng điểm nhiều hơn đưa gold từ rank 20 lên rank 16. Vì vậy, MRR phù hợp để đánh giá reranker.

NDCG@k.

Normalized Discounted Cumulative Gain [22] đo chất lượng toàn bộ thứ tự xếp hạng:

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}, \quad (2.20)$$

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}, \quad (2.21)$$

trong đó rel_i là mức liên quan tại vị trí i , còn IDCG là DCG lý tưởng. NDCG phù hợp khi có nhiều mức liên quan như gold, partial và irrelevant.

2.7.2 Độ đo cho phân loại ý định và pipeline

Với phân loại ý định, độ chính xác đo tỉ lệ dự đoán đúng trên toàn bộ mẫu. Tuy nhiên, khi dữ liệu mất cân bằng lớp, độ chính xác dễ bị lớp đa số chi phối nên không phản ánh đúng chất lượng. Trong trường hợp này, macro-F1 — trung bình điểm F1 (trung bình điều hòa của precision và recall) trên các lớp — là độ đo phù hợp hơn, vì mỗi lớp đóng góp ngang nhau bất kể số lượng mẫu. Đây là lý do đề án dùng macro-F1 làm độ đo chính cho bộ phân loại ý định, nơi hai lớp tra_cuu và tinh_toan chênh lệch nhiều về số lượng.

Với toàn bộ pipeline RAG, ngoài độ chính xác của câu trả lời cuối, độ trễ và dung lượng triển khai cũng là yếu tố quan trọng: một cấu hình có độ chính xác cao

nhưng độ trễ quá lớn có thể không phù hợp với sử dụng thực tế. Vì vậy, đề án đánh giá hệ thống theo hướng đa mục tiêu, cân nhắc đồng thời chất lượng truy hồi, chất lượng xếp hạng lại, độ chính xác cuối, độ trễ và dung lượng triển khai.

2.8 Khoảng trống nghiên cứu

Từ cơ sở lý thuyết và đặc thù của bài toán, đề án xác định một số khoảng trống làm cơ sở cho các chương tiếp theo.

Gap 1 – Reranker nhẹ cho tài liệu kỹ thuật tiếng Việt

Cross-encoder reranker mạnh có thể cải thiện chất lượng ranking nhưng thường có kích thước lớn và độ trễ cao. Trong bối cảnh tiếng Việt, đặc biệt với tài liệu kỹ thuật, vẫn cần khảo sát cách nén reranker bằng chưng cất và từ vựng cắt tia để giữ chất lượng gần teacher trong khi giảm chi phí triển khai.

Gap 2 – Tinh chỉnh embedding có kiểm soát false negatives

Các mô hình embedding đa ngôn ngữ hoặc tiếng Việt tổng quát chưa chắc tối ưu cho corpus kỹ thuật hẹp. Khi tinh chỉnh bằng contrastive learning, false negatives dễ xuất hiện vì nhiều đoạn cùng liên quan tới một truy vấn hoặc cùng chủ đề. Do đó, cần thiết kế curriculum và loss phù hợp để vừa học được hard negatives, vừa hạn chế nhiễu từ false negatives.

Gap 3 – Tối ưu đồng thời chất lượng, độ trễ và dung lượng

Nhiều hệ thống RAG chỉ báo cáo độ chính xác hoặc truy hồi metrics mà ít phân tích đồng thời độ trễ, chi phí API, dung lượng index và kích thước model. Với hệ thống triển khai thực tế, các yếu tố này cần được đánh giá cùng nhau. Đề án tiếp cận bài toán theo hướng tối ưu nhiều mục tiêu thay vì chỉ tối ưu một metric riêng lẻ.

Gap 4 – Thay thế LLM router bằng classifier cục bộ

LLM-based định tuyến có thể đạt chất lượng tốt nhưng tạo thêm độ trễ và chi phí API cho mỗi truy vấn. Khi bài toán định tuyến chỉ gồm một số lớp ý định rõ ràng, một classifier cục bộ dựa trên SBERT/SetFit có thể là lựa chọn hiệu quả hơn. Vấn đề cần giải quyết là làm sao giữ độ chính xác cao trong điều kiện dữ liệu ít và mất cân bằng lớp.

Các khoảng trống trên dẫn tới các hướng tối ưu được trình bày ở các chương sau: xây dựng dữ liệu huấn luyện từ pipeline v1, tinh chỉnh embedding bằng curriculum GIST→MNRL kết hợp MRL, chưng cất và nén reranker, đồng thời thay thế GPT router bằng ý định classifier chạy cục bộ.

CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT

Chương này trình bày mô hình đề xuất của đề án cho bài toán hỏi đáp trên kho tài liệu kỹ thuật tiếng Việt. Bài toán nhận đầu vào là một câu hỏi và kho tài liệu đã được lập chỉ mục, yêu cầu truy hồi đúng đoạn văn bản chứa thông tin liên quan rồi sinh câu trả lời bám sát nguồn, trong điều kiện ràng buộc về độ trễ, chi phí và khả năng triển khai trên hạ tầng nội bộ. Như đã nêu ở Chương 1, ba hạn chế chính của một pipeline RAG thông thường trên bài toán này là reranker quá nặng, định tuyến bằng LLM lớn tốn kém, và embedding pretrained chưa tối ưu cho miền.

Thay vì đề xuất giải pháp dựa trên giả định, đề án theo cách tiếp cận hai giai đoạn có căn cứ thực nghiệm. Trước hết, một hệ thống nền tảng (gọi là v1) được xây dựng và đo lường để xác định chính xác các nút thắt. Sau đó, mỗi thành phần cải tiến của hệ thống v2 được thiết kế để giải quyết một nút thắt đã được định lượng.

Để người đọc nắm được bức tranh tổng thể trước khi đi vào chi tiết, Mục 3.1 giới thiệu kiến trúc tổng quan của hệ thống đề xuất. Các mục tiếp theo trình bày chi tiết: Mục 3.2–3.5 phân tích hệ thống v1, quy trình tiền xử lý, kiến trúc pipeline và các nút thắt; Mục 3.6–3.9 trình bày các thành phần của hệ thống v2, gồm tinh chỉnh embedding, nén reranker và bộ phân loại ý định.

3.1 Tổng quan kiến trúc hệ thống đề xuất

Hệ thống đề xuất hoạt động theo hai giai đoạn tách biệt: giai đoạn chuẩn bị và huấn luyện (offline) thực hiện một lần, và giai đoạn phục vụ truy vấn (online) chạy cho mỗi câu hỏi của người dùng. Hình 3.1 minh họa kiến trúc tổng thể cùng vị trí của ba thành phần cải tiến do đề án đề xuất.

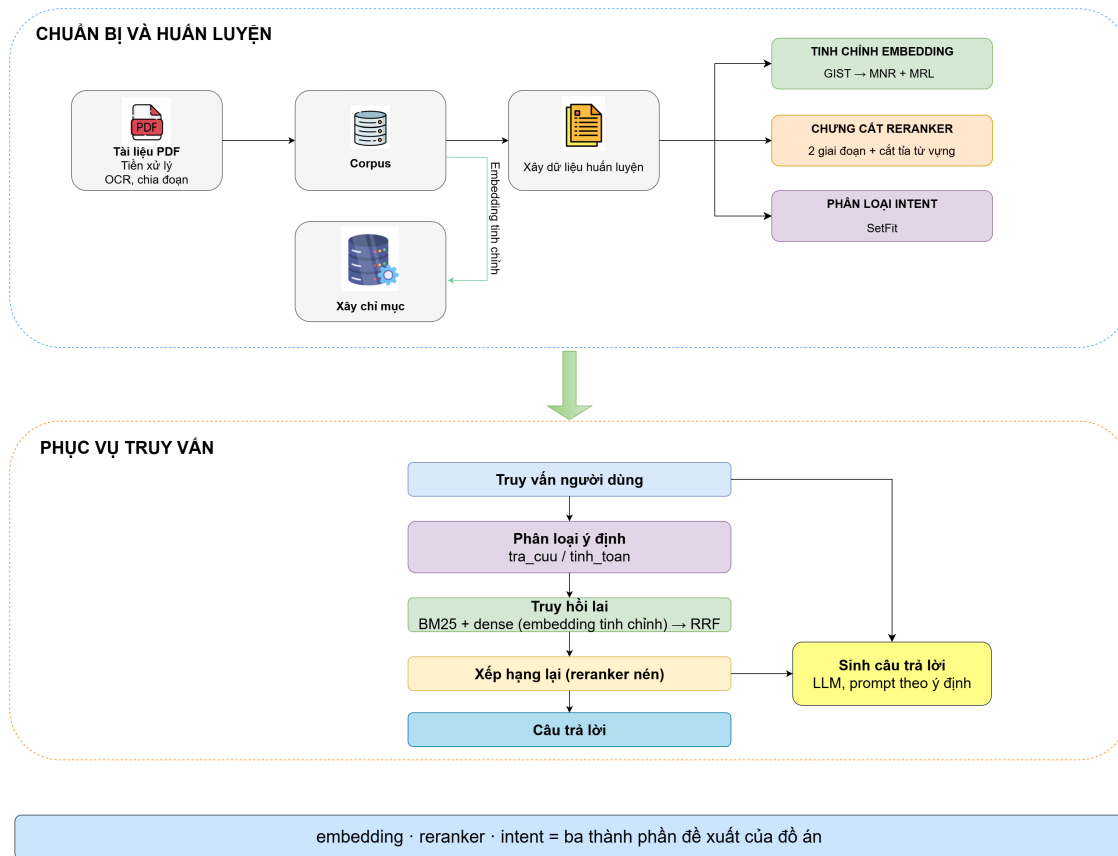


Figure 3.1: Kiến trúc tổng thể của hệ thống đề xuất

Ở giai đoạn offline, tài liệu PDF được tiền xử lý (trích xuất văn bản, chia đoạn) thành kho đoạn văn bản. Từ kho này, một quy trình chung xây dựng dữ liệu huấn luyện được dùng cho cả ba thành phần. Trên nền dữ liệu đó, đồ án (1) tinh chỉnh mô hình embedding theo lộ trình GIST→MNRL kết hợp MRL, (2) chung cất reranker từ teacher BGE-reranker-v2-m3 qua hai giai đoạn kèm cắt tỉa từ vựng, và (3) huấn luyện bộ phân loại ý định bằng SetFit. Kho đoạn cũng được lập chỉ mục thành hai dạng: chỉ mục BM25 và chỉ mục vector dùng chính embedding đã tinh chỉnh.

Ở giai đoạn online, mỗi truy vấn lần lượt đi qua bốn bước. Trước hết, bộ phân loại ý định xác định câu hỏi thuộc loại tra cứu (tra_cuu) hay tính toán (tinh_toan) để chọn cách sinh câu trả lời phù hợp. Tiếp theo, hệ thống truy hồi lại kết hợp BM25 và truy hồi dày dựa trên embedding tinh chỉnh, hợp nhất kết quả bằng RRF. Tập ứng viên sau đó được reranker nén xếp hạng lại để chọn ngữ cảnh tốt nhất. Cuối cùng, mô hình ngôn ngữ sinh câu trả lời bám sát ngữ cảnh, với prompt được chọn theo ý định đã phân loại.

Như vậy, ba thành phần đề xuất không hoạt động độc lập mà phối hợp trong một hệ thống đầu cuối: embedding tinh chỉnh cải thiện chất lượng truy hồi, reranker nén giảm chi phí ở bước xếp hạng lại, và bộ phân loại ý định cục bộ thay thế bộ

định tuyến dựa trên mô hình ngôn ngữ lớn. Các mục tiếp theo của chương trình bày chi tiết từng thành phần, bắt đầu từ việc phân tích hệ thống v1 để làm rõ động lực thiết kế.

3.2 Vai trò của hệ thống v1 trong đồ án

Hệ thống RAG ban đầu, gọi là v1 trong đồ án, được xây dựng như một prototype để kiểm chứng khả năng áp dụng RAG cho bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt. Mục tiêu của v1 không phải tạo ra hệ thống cuối cùng, mà là một bước phát triển trung gian nhằm trả lời ba câu hỏi thực tế: pipeline RAG cơ bản có hoạt động ổn định trên dữ liệu kỹ thuật tiếng Việt hay không; thành phần nào tạo ra bottleneck khi triển khai; và cần tối ưu ở đâu để hình thành đóng góp kỹ thuật rõ ràng.

Trong giai đoạn này, hệ thống được chạy thử trên 991 câu hỏi trắc nghiệm và 200 tài liệu PDF từ tập phát triển. Chính 991 câu hỏi này sau đó được dùng để xây dựng dữ liệu huấn luyện cho v2, gồm nhãn ý định, cặp query–positive đoạn và negative samples cho reranker và embedding. Vì vậy, kết quả trên 991 câu chỉ được xem là tín hiệu phát triển nội bộ, không phải kết quả đánh giá cuối cùng.

Tập kiểm thử độc lập gồm 343 câu hỏi từ 50 tài liệu PDF được giữ riêng cho giai đoạn đánh giá v2 và không được sử dụng để huấn luyện.

Table 3.1: Vai trò của các tập dữ liệu trong quy trình phát triển

Nhóm dữ liệu	Quy mô	Vai trò
Tập phát triển v1	991 câu hỏi, 200 tài liệu PDF	Chạy thử pipeline, quan sát lỗi, đo bottleneck và tạo dữ liệu huấn luyện v2
Dữ liệu huấn luyện v2	Xây dựng từ 991 câu hỏi	Huấn luyện reranker, embedding và bộ phân loại ý định
Tập kiểm thử độc lập	343 câu hỏi, 50 tài liệu PDF	Chỉ dùng để đánh giá v2, không dùng để huấn luyện hoặc lựa chọn cấu hình

3.3 Quy trình tiền xử lý tài liệu

3.3.1 Trích xuất văn bản từ tài liệu PDF

Tập dữ liệu gồm các tài liệu PDF kỹ thuật thuộc nhiều lĩnh vực như phần mềm kiểm thử, thiết bị y tế, ô tô, điện, hệ thống HVAC và hướng dẫn vận hành thiết bị. Đặc điểm chung của các tài liệu này là chứa nhiều bảng thông số, ký hiệu kỹ thuật và cấu trúc phân cấp theo đề mục. Do đó, bước trích xuất văn bản cần bảo toàn được cả nội dung lẫn cấu trúc, thay vì chỉ rút ra văn bản thuần. Quy trình tiền xử lý gồm ba giai đoạn: loại bỏ phần header lặp lại, chuyển đổi PDF sang Markdown có cấu trúc, và chuẩn hóa định dạng đầu ra.

Loại bỏ header lặp lại đầu trang.

Nhiều tài liệu kỹ thuật có một bảng thông tin (tiêu đề, mã hiệu, phiên bản) lặp lại ở đầu mỗi trang. Nếu giữ nguyên, phần lặp này tạo nhiều cho cả bước chia đoạn lẫn truy hồi. Hệ thống xác định vùng cần cắt bằng cách phát hiện bảng đầu tiên trên trang một: tỉ lệ cắt được tính theo vị trí cạnh dưới của bảng so với chiều cao trang, cộng thêm một biên nhỏ. Tỉ lệ này sau đó được áp dụng để cắt phần trên của mọi trang. Với tài liệu không phát hiện được bảng ở trang đầu, hệ thống dùng một tỉ lệ cắt mặc định.

Chuyển đổi PDF sang Markdown.

Các tài liệu sau khi cắt header được chuyển đổi sang định dạng Markdown bằng công cụ OCR Marker [23]. Quá trình này giữ lại cấu trúc đề mục (heading) và biểu diễn bảng dưới dạng HTML, cho phép bước chia đoạn ở sau phân biệt được bảng với văn bản thường. Một vấn đề thường gặp là một bảng dài bị tách qua nhiều trang sẽ được OCR nhận diện thành nhiều bảng rời rạc. Để khắc phục, hệ thống tự động ghép các bảng liền kề: khi hai bảng chỉ cách nhau bởi khoảng trắng hoặc dấu phân trang (không có văn bản xen giữa), chúng được gộp lại thành một bảng, trong đó hàng tiêu đề lặp lại của bảng phía dưới được chuyển thành hàng dữ liệu thường. Mỗi tài liệu được lưu thành một file main.md trong thư mục riêng theo định danh tài liệu, kèm một đề mục cấp cao nhất chứa định danh đó.

Chuẩn hóa định dạng Markdown.

Đầu ra của OCR còn một số điểm không nhất quán về định dạng đề mục. Bước chuẩn hóa cuối cùng xử lý các trường hợp này: các tiêu đề được đánh số dạng in đậm (ví dụ ****1.2Tiêu đề****) được chuyển thành đề mục Markdown với cấp tương ứng số thứ tự; các tiêu đề bị OCR đảo vị trí số (ví dụ Tiêu đề1.2) được nhận diện và sửa lại; và các ký hiệu danh sách được thống nhất về một dạng. Việc chuẩn hóa đề mục là cần thiết vì bước chia đoạn sau đó dựa vào cấu trúc đề mục để bảo toàn ngữ cảnh phân cấp.

Mỗi tài liệu được lưu theo định danh Public_XXX; định danh này được giữ trong metadata của từng đoạn để hỗ trợ truy xuất trực tiếp khi câu hỏi nêu rõ tài liệu nguồn, chẳng hạn Public_021. Sau toàn bộ quy trình, một bảng thông số trong tài liệu gốc vẫn giữ được cấu trúc hàng–cột. Việc bảo toàn cấu trúc bảng giúp bước chia đoạn ở mục sau xử lý đúng quan hệ giữa hàng tiêu đề và từng ô dữ liệu.

3.3.2 Chiến lược chia đoạn nhận biết bảng

Chia đoạn (chunking) quyết định đơn vị thông tin được đưa vào truy hồi. Với tài liệu kỹ thuật, đoạn quá lớn (chẳng hạn trên 1.000 từ) làm loãng thông tin và giảm độ chính xác truy hồi; ngược lại, đoạn quá nhỏ (dưới 100 từ) có thể làm mất ngữ

cảnh hoặc tách hàng tiêu đề khỏi dữ liệu bảng. Vì văn bản thường và bảng có cấu trúc khác nhau, đồ án sử dụng một chiến lược chia đoạn nhận biết bảng, xử lý riêng hai loại nội dung này.

Chia đoạn văn bản thường.

Văn bản thuần được chia theo đơn vị đoạn văn (paragraph) với ngân sách khoảng 512 từ mỗi đoạn và phần chồng lấn (overlap) khoảng 100 từ giữa hai đoạn liên tiếp. Phần chồng lấn hạn chế mất thông tin tại ranh giới giữa các đoạn. Mỗi đoạn lưu lại đường dẫn phân cấp đề mục (chuỗi heading dạng $H1 > H2 > H3$ dẫn tới đoạn văn) nhằm bảo toàn ngữ cảnh cấu trúc của tài liệu.

Chia đoạn bảng biểu.

Mỗi bảng được chia theo cửa sổ trượt 10 hàng với phần chồng lấn 2 hàng. Hàng tiêu đề được thêm lại vào đầu mỗi đoạn con để giữ ý nghĩa của từng cột. Nội dung HTML của bảng sau đó được chuyển thành văn bản có cấu trúc, ví dụ:

Cột: Thông số | Giá trị | Đơn vị

Dòng 1: Nhiệt độ vận hành tối đa | 85 | độ C

Dòng 2: Áp suất vận hành | 3.0 | bar

Dòng 3: Công suất tiêu thụ | 500 | W

Biểu diễn dạng văn bản này phù hợp với mô hình embedding hơn HTML thô, vì mô hình chủ yếu được huấn luyện trước trên ngôn ngữ tự nhiên.

Metadata của đoạn.

Mỗi đoạn được gán bốn trường metadata chính: định danh tài liệu nguồn, đường dẫn phân cấp đề mục chứa đoạn, loại đoạn (văn bản hoặc bảng), và chỉ số thứ tự của đoạn trong tài liệu. Các trường này hỗ trợ lọc theo tài liệu, xây dựng chỉ mục cấp tài liệu và truy vết nguồn gốc khi phân tích lỗi.

Tổng hợp lại, quy trình chia đoạn gồm bốn bước: tách file Markdown thành các đoạn văn bản và bảng dựa trên thẻ HTML; với đoạn văn bản, gộp các paragraph tới giới hạn kích thước đồng thời giữ phần chồng lấn cho đoạn kế tiếp; với bảng, chia theo cửa sổ hàng và thêm lại hàng tiêu đề; cuối cùng chuyển mỗi đoạn thành văn bản, gán metadata và chỉ số thứ tự.

3.3.3 Xây dựng chỉ mục

Sau chunking, các đoạn được đưa vào ba cấu trúc chỉ mục phục vụ những pipeline khác nhau.

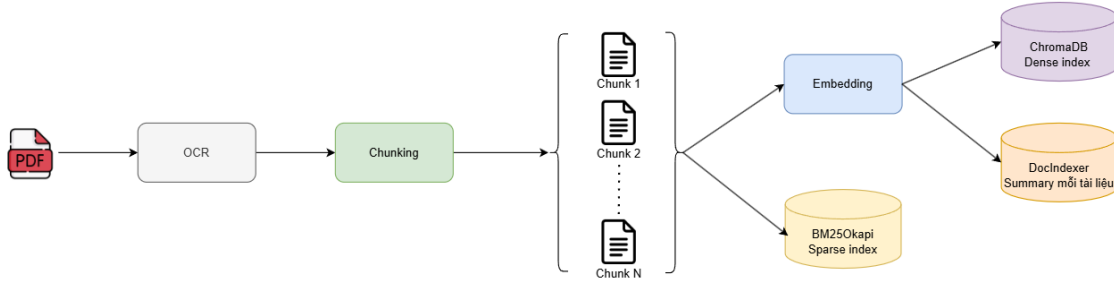


Figure 3.2: Luồng xử lý xây dựng chỉ mục

ChromaDB

ChromaDB lưu dense embeddings của các đoạn. Embedding được tạo bằng AITeamVN/Vietnamese_Embedding_v2 dưới dạng vector 1024 chiều chuẩn hóa L2.

BM25Okapi

BM25Okapi lập chỉ mục toàn bộ đoạn text cho sparse truy hồi. Thành phần này không cần GPU, có độ trễ dưới khoảng 10ms/query và phù hợp với mã số, ký hiệu hoặc tên thiết bị mà dense model có thể chưa biểu diễn tốt.

DocIndex

DocIndex là chỉ mục bổ sung dành cho Router-Summary, trong đó mỗi tài liệu được biểu diễn bằng một *summary vector* ở cấp tài liệu. Từ “summary” ở đây không phải là đoạn tóm tắt sinh bởi LLM, mà là biểu diễn vector cô đọng được tính offline từ các đoạn của tài liệu. Mục tiêu là chọn nhanh một vài tài liệu liên quan trước khi truy hồi chi tiết ở cấp đoạn.

Với tài liệu có ít nhất ba text đoạn, DocIndex dùng TextRank trên graph tương đồng giữa các đoạn. Mỗi text đoạn đã có embedding chuẩn hóa L2. Các đoạn được xem như các đỉnh của graph, còn trọng số cạnh là cosine similarity giữa hai embedding:

$$s_{ij} = \mathbf{e}_i^\top \mathbf{e}_j, \quad i \neq j. \quad (3.1)$$

Sau khi bỏ self-loop và chuẩn hóa ma trận similarity theo hàng, thuật toán chạy PageRank với hệ số damping $d = 0.85$:

$$\mathbf{w}^{(t+1)} = \frac{1-d}{n} \mathbf{1} + dS^\top \mathbf{w}^{(t)}. \quad (3.2)$$

Quá trình lặp dừng khi trọng số hội tụ hoặc đạt tối đa 100 vòng. Vector summary

phần text của tài liệu sau đó được tính bằng trung bình có trọng số:

$$\mathbf{v}_{text} = \sum_i w_i \mathbf{e}_i. \quad (3.3)$$

Nếu tài liệu có cả bảng, vector cuối cùng được blend giữa text summary và trung bình embedding của bảng theo tỉ lệ 85/15. Với tài liệu chỉ có một hoặc hai text đoạn, TextRank không ổn định vì graph quá nhỏ, nên hệ thống dùng mean text embedding và blend với table embedding theo tỉ lệ 70/30. Với tài liệu chỉ có bảng, hệ thống dùng trung bình có trọng số theo phân cấp: đoạn nằm sâu hơn trong cây heading và xuất hiện sớm hơn trong tài liệu được gán trọng số cao hơn.

Khi truy vấn, DocIndex thu hẹp không gian tìm kiếm từ toàn bộ kho tài liệu xuống top-3 tài liệu trước khi thực hiện truy vấn cấp đoạn.

3.4 Kiến trúc hai pipeline RAG

Trên nền tiền xử lý và chỉ mục chung, đồ án so sánh hai pipeline chính: Baseline Router và Router-Summary. Hai pipeline này dùng chung đoạn, embedding, cache, router GPT-4o-mini và reranker BGE-reranker-v2-m3, nhưng khác nhau ở phạm vi truy xuất và việc sử dụng tóm tắt cấp tài liệu.

3.4.1 Pipeline Baseline Router

Baseline thực hiện truy xuất lai (hybrid truy hồi) trực tiếp trên toàn corpus rồi đưa ngữ cảnh vào LLM để sinh đáp án.

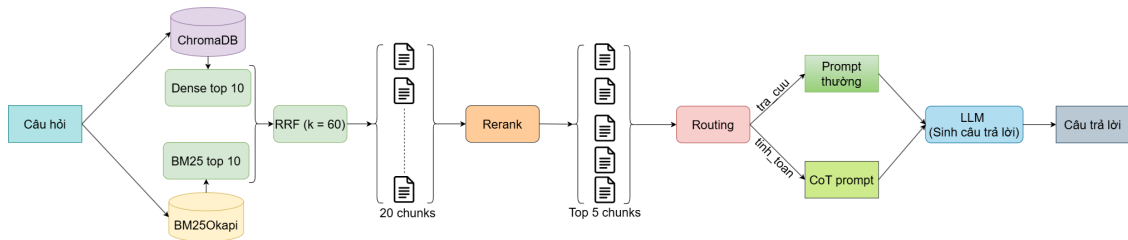


Figure 3.3: Luồng xử lý của pipeline Baseline

Bước 1 – Truy hồi lai

BM25 top-10 và ChromaDB dense top-10 chạy độc lập. Kết quả được hợp nhất bằng RRF với $k = 60$ để tạo ứng viên pool.

Bước 2 – Xếp hạng lại bằng BGE-reranker-v2-m3

Cross-encoder BGE reranker chấm điểm từng cặp query–ứng viên. Trong code, model tương ứng là BAAI/bge-reranker-v2-m3. Sau khi toàn bộ tập ứng viên được điểm, hệ thống giữ top-5 đoạn có relevance cao nhất.

Bước 3 – Phát hiện Public ID

Nếu câu hỏi chứa định danh tài liệu như Public_021, tập ứng viên được giới hạn trực tiếp vào các đoạn của tài liệu đó trước khi tái xếp hạng. Heuristic này hiệu quả trong nhiều trường hợp nhưng có thể sai nếu câu hỏi nhắc tới một Public ID trong khi thông tin cần trả lời nằm ở tài liệu khác.

Bước 4 – Định tuyến và sinh câu trả lời

GPT-4o-mini router phân loại câu hỏi thành tra_cuu hoặc tinh_toan. Top-5 đoạn được ghép cùng câu hỏi và các lựa chọn A/B/C/D. Với câu tra_cuu, GPT-4o-mini sinh đáp án trực tiếp với giới hạn 16 token; với câu tinh_toan, pipeline sinh reasoning trước rồi mới chọn đáp án cuối.

Baseline Router đã có router và prompt theo ý định, nhưng truy xuất vẫn thực hiện trên toàn corpus hoặc theo Public ID heuristic. Hạn chế chính là chưa có bước chọn tài liệu cấp cao như DocIndexer, và phân rerank bằng BGE-reranker-v2-m3 là thành phần nặng nhất trong pipeline.

3.4.2 Pipeline Router-Summary

Khác với Baseline, Router-Summary không truy xuất trực tiếp trên toàn bộ kho tài liệu cho mọi câu hỏi mà chia bài toán thành hai tầng: trước hết xác định phạm vi tài liệu liên quan, sau đó mới truy hồi ở cấp đoạn trong phạm vi hẹp hơn. Luồng xử lý gồm bốn bước chính.

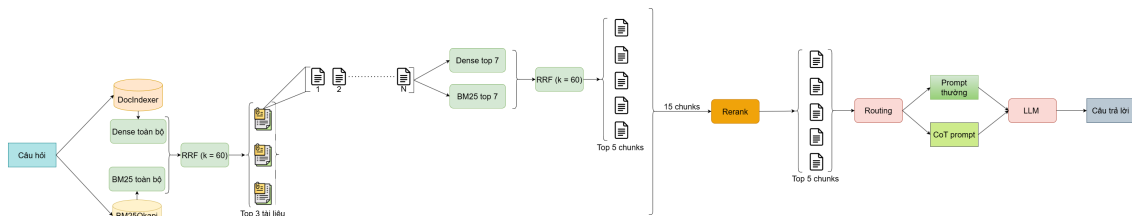


Figure 3.4: Luồng xử lý của pipeline Router-Summary

Bước 1 – Phân loại ý định và phát hiện Public ID

Router phân loại câu hỏi thành tra_cuu hoặc tinh_toan bằng GPT-4o-mini, đồng thời phát hiện Public ID nếu câu hỏi nêu trực tiếp tài liệu nguồn. Public ID quyết định cách chọn tài liệu ở bước sau: nếu có, hệ thống tìm trực tiếp trong tài liệu được nêu; nếu không, hệ thống chuyển sang chọn tài liệu bằng DocIndexer.

Bước 2 – Chọn tài liệu cấp cao

Khi không có Public ID, DocIndexer chọn ra một số tài liệu liên quan nhất từ toàn corpus. Mỗi tài liệu được biểu diễn sẵn bằng một summary vector (mục 3.3.3)

nên bước này không cần đọc lại toàn bộ tài liệu lúc truy vấn. DocIndexer kết hợp hai tín hiệu cấp tài liệu — BM25 trên văn bản ghép của tài liệu và độ tương đồng cosine với summary vector — qua Reciprocal Rank Fusion, rồi lấy top-3 tài liệu.

Bước 3 – Truy hồi và xếp hạng lại cấp đoạn

Trong phạm vi các tài liệu đã chọn, hệ thống thực hiện truy hồi lại cấp đoạn (BM25 và dense, hợp nhất bằng RRF, giữ tối đa 5 đoạn mỗi tài liệu), sau đó dùng cross-encoder BGE-reranker-v2-m3 xếp hạng lại và giữ top-5 đoạn làm ngữ cảnh. Việc thu hẹp phạm vi tài liệu trước khi rerank giúp giảm nhiễu, đặc biệt hiệu quả với nhóm `tinh_toan` — nơi ngữ cảnh sai hoặc thiếu một con số có thể làm sai đáp án cuối.

Bước 4 – Sinh câu trả lời theo ý định

Với câu `tra_cuu`, hệ thống dùng prompt trực tiếp để chọn đáp án A/B/C/D. Với câu `tinh_toan`, hệ thống sinh phân tích từng bước trước, rồi mới chọn đáp án cuối.

3.4.3 So sánh kiến trúc hai pipeline

Table 3.2: So sánh kiến trúc Baseline Router và Router-Summary

Đặc điểm	Baseline Router	Router-Summary
Phạm vi truy hồi	Hybrid truy hồi trực tiếp trên toàn corpus	DocIndexer chọn top-3 tài liệu trước khi retrieve cấp đoạn
Phạm vi rerank	Tập ứng viên toàn bộ tài liệu	Tập ứng viên sau khi thu hẹp theo tài liệu
Phân loại ý định	GPT-4o-mini router	GPT-4o-mini router
Số lần gọi LLM/câu	2 lần với <code>tra_cuu</code> ; 3 lần với <code>tinh_toan</code>	2 lần với <code>tra_cuu</code> ; 3 lần với <code>tinh_toan</code>
Prompt theo ý định	Prompt trực tiếp cho <code>tra_cuu</code> ; CoT cho <code>tinh_toan</code>	Prompt trực tiếp cho <code>tra_cuu</code> ; CoT cho <code>tinh_toan</code>
Tập test	200 câu hỏi	200 câu hỏi

3.5 Xác định các nút thắt

Quá trình vận hành và đo đạc hệ thống v1 cho thấy ba nút thắt chính về độ trễ, chi phí và chất lượng truy xuất. Mỗi vấn đề gắn với một thành phần cụ thể và dẫn đến một hướng tối ưu riêng trong v2. Các số liệu định lượng chi tiết của v1 (accuracy, độ trễ) được trình bày trong Chương 4; mục này tập trung phân tích bản chất của từng nút thắt.

3.5.1 Nút thắt độ trễ: BGE-reranker-v2-m3

BAAI/bge-reranker-v2-m3 gồm 568M tham số và là thành phần nặng nhất trong quy trình truy xuất. Đo đạc trên v1 cho thấy bước tái xếp hạng bằng Bge-reranker-v2-m3 chiếm phần lớn tổng độ trễ của pipeline; việc thu hẹp phạm vi tài liệu trước khi tái xếp hạng (như trong Router-Summary) giảm được một phần chi phí này,

nhưng reranker vẫn là thành phần cần tối ưu nếu muốn giảm thời gian suy luận. Số liệu độ trễ cụ thể được trình bày ở Chương 4.

Giảm kích thước model giúp giảm yêu cầu bộ nhớ và thời gian tải, nhưng một MiniLM chưa qua huấn luyện trên miền có chất lượng xếp hạng thấp. Vì vậy, bài toán đặt ra là chung cất tri thức từ BGE-reranker-v2-m3 sang các reranker nhỏ hơn (từ MiniLM 33M tới mmarco/pruned) mà vẫn duy trì chất lượng xếp hạng. So sánh độ trễ và chất lượng giữa các reranker được trình bày ở Chương 4.

3.5.2 Nút thắt chi phí: bộ định tuyến ý định GPT

Router-Summary dùng GPT-4o-mini cho bước phân loại ý định trước khi sinh đáp án. Lần gọi router bổ sung độ trễ và chi phí API cho mỗi truy vấn, dù tác vụ chỉ phân biệt hai lớp tra_cuu và tinh_toan — một bài toán không đòi hỏi model có năng lực sinh ngôn ngữ lớn. Khi triển khai ở quy mô lớn, chi phí gọi API cho bước định tuyến tích lũy đáng kể, đồng thời tạo phụ thuộc vào dịch vụ ngoài. Một router cục bộ dựa trên Sentence-BERT/SetFit có thể chạy nội bộ với chi phí suy luận biên gần bằng không và loại bỏ phụ thuộc mạng. Phân tích chi phí và độ trễ định lượng được trình bày ở Chương 4.

3.5.3 Nút thắt chất lượng: embedding chưa tối ưu theo miền

Vietnamese_Embedding_v2 là mô hình embedding tiếng Việt mạnh nhưng chưa được tối ưu riêng cho miền tài liệu kỹ thuật của đề án. Khi một phần truy vấn chưa có gold đoạn ở thứ hạng cao nhất, gánh nặng cải thiện thứ hạng dồn sang bộ tái xếp hạng. Nếu gold đoạn rơi khỏi nhóm top-k ứng viên, bộ tái xếp hạng cũng không thể khôi phục. Phân tích định tính cho thấy lỗi truy xuất là một nhóm lỗi đáng kể: các câu hỏi về thông số, bảng số liệu, ký hiệu và thuật ngữ chuyên ngành đòi hỏi embedding nắm được quan hệ đặc thù của miền, thay vì chỉ dựa trên ngữ nghĩa tiếng Việt tổng quát.

Vì mô hình nền vốn đã khá mạnh, hướng tối ưu embedding ở v2 không chỉ nhằm tới độ chính xác mà còn tới hiệu quả biểu diễn: tinh chỉnh với GIST và học theo chương trình để thích nghi miền, kết hợp Matryoshka Representation Learning để tạo biểu diễn đa kích thước, cho phép sử dụng số chiều nhỏ hơn mà không làm giảm đáng kể chất lượng. Mức cải thiện cụ thể được đánh giá ở Chương 4.

Ba nút thắt trên xác định ba hướng tối ưu của v2: xây dựng bộ tái xếp hạng nhẹ bằng chung cất tri thức để giảm độ trễ; huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit để thay thế GPT router; và tinh chỉnh embedding bằng MRL, học theo chương trình và GIST để thích nghi miền và giảm chi phí biểu diễn.

3.6 Tổng quan định hướng v2

Từ ba nút thắt được xác định ở các mục trên, hệ thống v2 được thiết kế theo nguyên tắc mỗi thay đổi kỹ thuật phải giải quyết một vấn đề đã được đo lường, thay vì tối ưu dựa trên giả định. Ba hướng tối ưu tương ứng được trình bày trong Bảng 3.3.

Table 3.3: Ánh xạ nút thắt của v1 sang giải pháp v2

Nút thắt v1	Giải pháp v2	Mục tiêu thiết kế
Mô hình embedding nền chưa tối ưu theo miền	Tinh chỉnh bằng MRL, học theo chương trình và GIST	Cải thiện truy xuất, hỗ trợ triển khai 512 chiều và giảm dung lượng lưu trữ
BGE-reranker-v2-m3: 568M tham số, độ trễ lớn khi tái xếp hạng top-20	Chung cất sang mmarco 117M, pruned 35.6M và MiniLM 33M	Giữ MRR@10 gần với mô hình teacher, giảm độ trễ pipeline và giảm dung lượng khi triển khai
GPT router: độ trễ và chi phí API cho mỗi truy vấn	Huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit	Đạt độ chính xác cao, độ trễ thấp và có thể chạy cục bộ

Ba hướng được trình bày theo trình tự của pipeline: trước hết là tinh chỉnh embedding ở tầng truy xuất, tiếp đến là xây dựng bộ tái xếp hạng nhẹ, rồi xây dựng bộ phân loại ý định. Phần chuẩn bị dữ liệu huấn luyện được xây dựng chung, phục vụ embedding trước và được tái sử dụng cho bộ tái xếp hạng. Kết quả đánh giá của cả ba hướng được tổng hợp trong Chương 4.

3.7 Tinh chỉnh embedding bằng MRL và học theo lộ trình

3.7.1 Pipeline xây dựng dữ liệu embedding

Dữ liệu huấn luyện được xây dựng từ 991 câu hỏi gốc qua một quy trình năm giai đoạn. Cùng một nguồn dữ liệu này được dùng cho cả tầng embedding và tầng reranker; phần khác biệt về tiêu chí lọc và ghép cặp riêng cho reranker được trình bày ở mục 3.8.2.

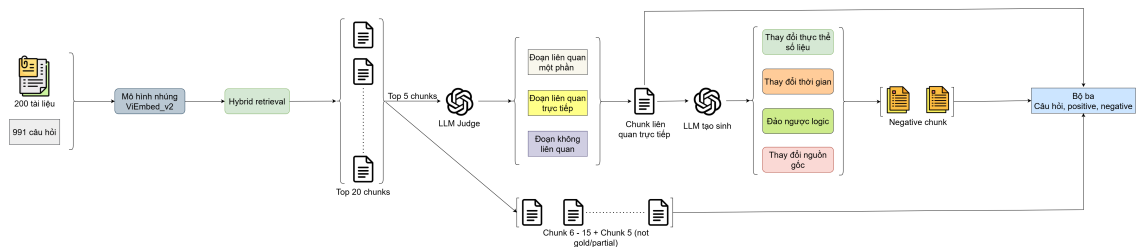


Figure 3.5: Quy trình xây dựng dữ liệu huấn luyện embedding

Giai đoạn 1 – Truy hồi và xếp hạng lại. Toàn bộ 991 câu hỏi được đưa qua truy hồi lại (BM25, truy hồi dense và hợp nhất bằng RRF), sau đó được xếp hạng lại bằng BGE-reranker-v2-m3. Mỗi câu hỏi giữ lại 20 ứng viên hàng đầu kèm điểm

số và thứ hạng của teacher, tạo thành tập ứng viên đầu vào cho các giai đoạn sau.

Giai đoạn 2 – Xác định positive bằng mô hình ngôn ngữ. Thay vì mặc định ứng viên xếp hạng cao nhất là positive, một mô hình ngôn ngữ được dùng làm giám khảo để phân loại từng ứng viên thành liên quan trực tiếp, liên quan một phần hoặc không liên quan. Cách này tránh nhận nhầm các đoạn chỉ liên quan một phần làm positive. Sau khi loại các câu hỏi không có positive đáng tin cậy, còn lại 786 câu hỏi hợp lệ để xây dựng dữ liệu embedding.

Giai đoạn 3 – Tổng hợp hard negative. Với mỗi câu hỏi có positive, mô hình ngôn ngữ tạo ra các phiên bản gần giống positive nhưng thay đổi một thông tin quyết định khả năng trả lời, chẳng hạn thực thể, số liệu, điều kiện hoặc phạm vi áp dụng. Các negative này rất gần positive về bề mặt nhưng được thiết kế để không trả lời đúng câu hỏi, tạo ra ranh giới ngữ nghĩa có kiểm soát.

Giai đoạn 4 – Khai thác hard negative từ kết quả xếp hạng. Bên cạnh negative tạo sinh, hệ thống khai thác các đoạn thật nằm gần positive trong kết quả truy hồi và xếp hạng. Với embedding, các negative này được lấy từ những ứng viên ở hạng 5 đến 15, sau khi loại bỏ các đoạn đã được đánh giá là liên quan trực tiếp hoặc một phần. Đây là các negative sát với phân phối lỗi thực tế của hệ thống truy hồi hơn so với negative tạo sinh.

Giai đoạn 5 – Gộp và chia tập. Với mỗi câu hỏi, positive đã xác minh và toàn bộ negative (cả tạo sinh lẫn khai thác) được gom thành một bản ghi. Tập dữ liệu sau đó được chia thành tập huấn luyện và tập kiểm thử theo tỉ lệ 80/20, phân tầng theo ý định. Để huấn luyện theo dạng tương phản, mỗi bản ghi trong tập huấn luyện được tách thành nhiều bộ ba dạng (câu hỏi, positive, negative) — mỗi negative ghép với positive tạo một bộ ba; tập kiểm thử được giữ riêng để đánh giá, không dùng khi huấn luyện. Bộ dữ liệu tổng hợp gồm 786 câu hỏi hợp lệ và hai nguồn negative như trong Bảng 3.4.

Table 3.4: Các nguồn negative trong bộ dữ liệu embedding (pool thô trước khi tạo bộ ba)

Nguồn	Số negative	Ghi chú
Negative tạo sinh	1.291	Có ranh giới ngữ nghĩa được kiểm soát
Negative khai thác	9.103	Đoạn thật nằm gần positive trong không gian vector
Tổng	10.394	—

Khi tạo bộ ba huấn luyện, mỗi negative được ghép với positive tương ứng, số negative khai thác được giới hạn tối đa 10 trên mỗi câu hỏi, và dữ liệu được chia

phân tầng theo ý định với tỉ lệ 80/20 (629 câu hỏi huấn luyện, 157 câu hỏi kiểm thử in-domain — chính là tập D1). Vì vậy số bộ ba thực tế nhỏ hơn tổng pool thô. Curriculum cuối cùng gồm hai giai đoạn: giai đoạn một dùng 1.040 bộ ba chỉ với negative tạo sinh, giai đoạn hai dùng 7.330 bộ ba kết hợp cả negative tạo sinh và negative khai thác.

3.7.2 Thiết kế hàm mất mát cho tầng embedding

Mô hình nền là AITeamVN/Vietnamese_Embedding_v2, dựa trên BGE-M3 và tạo vector 1024 chiều. Hàm mất mát nền cho học tương phản (contrastive learning) là MultipleNegativesRankingLoss (MNRL): với kích thước batch N , mỗi câu hỏi phân biệt positive của nó với $N - 1$ negative trong cùng batch (in-batch negative):

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)} \quad (3.4)$$

Trong thực nghiệm, hệ số scale bằng 20, tương ứng nhiệt độ $\tau = 0.05$.

MNRL thuần xem toàn bộ các mẫu còn lại trong cùng batch là negative. Với tài liệu kỹ thuật, giả định này có thể tạo ra false negative (mẫu bị coi là negative nhưng thực chất vẫn liên quan), do nhiều đoạn khác nhau vẫn có khả năng cùng liên quan đến một câu hỏi. Khi đó, positive của một câu hỏi có thể thực chất cũng liên quan đến một câu hỏi khác trong batch, nhưng vẫn bị xem là negative trong quá trình huấn luyện. Để giảm ảnh hưởng của false negative, đề án áp dụng GISTEmbedLoss. Phương pháp này sử dụng một mô hình dẫn hướng (guide model) mạnh để đánh giá mức độ tương đồng giữa câu hỏi và các ứng viên trong batch. Trong phạm vi đề án, BGE-M3 được dùng làm mô hình dẫn hướng. Với mỗi câu hỏi, mô hình dẫn hướng chấm điểm tương đồng cho các cặp câu hỏi–ứng viên và loại khỏi tập negative những ứng viên có độ tương đồng với câu hỏi lớn hơn hoặc bằng positive tương ứng. Cấu hình sử dụng là chiến lược biên tuyệt đối với $\delta = 0$ và nhiệt độ bằng 0.01. Nhờ cơ chế lọc negative này, mô hình hạn chế bị phạt khi xếp hạng cao các mẫu thực chất vẫn liên quan đến câu hỏi, từ đó giảm nhiễu trong quá trình học biểu diễn.

Điểm thiết kế quan trọng của tầng embedding là dùng hàm mất mát theo lộ trình (curriculum): GIST ở giai đoạn 1 và MNRL ở giai đoạn 2, mỗi hàm mất mát đều được bọc trong MatryoshkaLoss để tối ưu đồng thời ba kích thước $\mathcal{M} = \{256, 512, 1024\}$:

$$\mathcal{L}^{(1)} = \text{MatryoshkaLoss}(\mathcal{L}_{\text{GIST}}), \quad \mathcal{L}^{(2)} = \text{MatryoshkaLoss}(\mathcal{L}_{\text{MNRL}}) \quad (3.5)$$

trong đó mỗi hàm mất mát thành phần được tính trên phần đầu (prefix) $f_m(\mathbf{x}) = \mathbf{x}[:m]$ với ba kích thước có trọng số bằng nhau. Lý do tách hàm mất mát theo giai đoạn: giai đoạn 1 học từ negative tạo sinh — dữ liệu dễ lẫn false negative nên cần GIST lọc bớt; giai đoạn 2 tinh chỉnh với negative khai thác đã sạch (được chọn theo chênh lệch điểm BGE-reranker-v2-m3, thuộc tài liệu khác). Nếu dùng GIST ở giai đoạn 2, mô hình dẫn hướng có thể lọc nhầm chính các negative khai thác này vì chúng “trông giống” positive, làm mất tín hiệu phân biệt tinh vi; trong khi MNRL giữ lại toàn bộ hard negative nên kỳ vọng học phân biệt tốt hơn. Hiệu quả của thiết kế lộ trình GIST→MNRL so với các phương án khác được đánh giá bằng thực nghiệm loại bỏ (ablation) ở Chương 4.

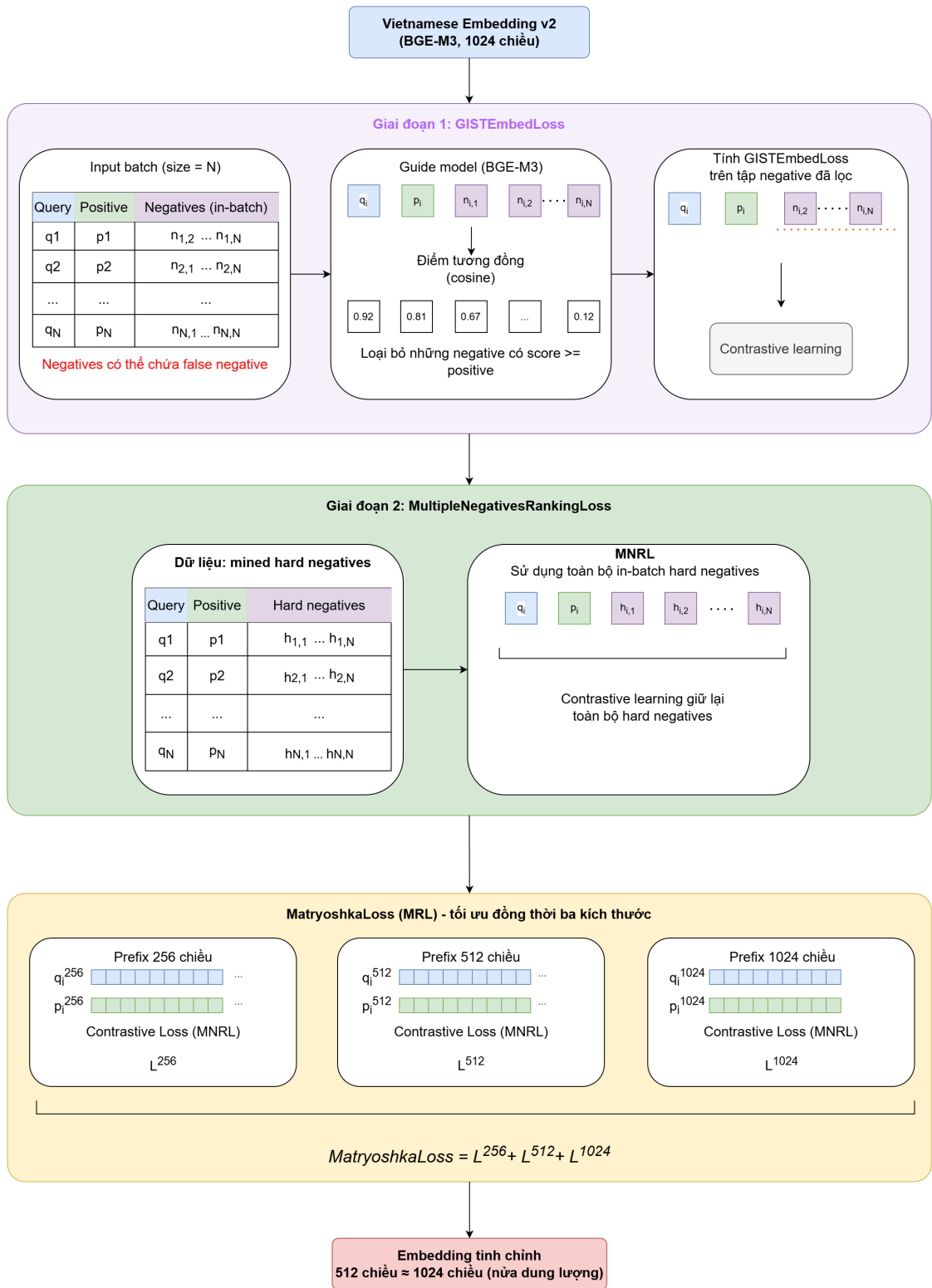


Figure 3.6: Hai giai đoạn huấn luyện cho tầng embedding

3.7.3 Huấn luyện theo lộ trình hai giai đoạn

Hai giai đoạn được huấn luyện nối tiếp. Checkpoint của giai đoạn trước là điểm khởi đầu của giai đoạn sau, và tốc độ học (learning rate) được giảm khi chuyển từ negative tạo sinh có ranh giới được kiểm soát sang negative khai thác — các đoạn

thật được hệ thống truy hồi xếp gần positive, nên sát với phân phối lỗi thực tế hơn.

Table 3.5: Cấu hình huấn luyện theo lộ trình cho embedding

Giai đoạn	Hàm mất mát	Nguồn negative	Số bộ ba	Epoch	Batch	Tốc độ học
Giai đoạn 1	GIST	Negative tạo sinh	1.040	2	64	2×10^{-6}
Giai đoạn 2	MNRL	Negative khai thác	7.330	1	32	5×10^{-7}

Các tham số chung gồm bộ tối ưu AdamW, hệ số suy giảm trọng số bằng 0.01, ngưỡng cắt gradient bằng 1.0, huấn luyện ở độ chính xác hỗn hợp (mixed precision) và độ dài chuỗi tối đa 512 token.

Giai đoạn 1 dùng negative tạo sinh có ranh giới rõ ràng cùng GIST để lọc false negative trong khi mô hình học ranh giới ngữ nghĩa. Giai đoạn 2 chuyển sang MNRL trên negative khai thác — các đoạn thật nằm gần positive trong không gian vector — giữ lại toàn bộ hard negative để tinh chỉnh khả năng phân biệt. Số bước khởi động (warmup) được giữ ở mức thấp để tránh tình trạng giai đoạn ngắn nhưng mô hình chưa kịp đạt tốc độ học chính.

3.7.4 Chiến lược triển khai với MRL

Một checkpoint hỗ trợ ba cấu hình:

Table 3.6: Các chế độ triển khai từ một checkpoint MRL

Số chiều	Cách sử dụng	Mục tiêu
256	Cắt vector còn 256 chiều	Tìm kiếm nhanh, chỉ mục nhỏ hơn 4 lần
512	Cắt vector còn 512 chiều	Cân bằng chất lượng và tài nguyên
1024	Kích thước mặc định	MRR/NDCG cao nhất khi không bị giới hạn dung lượng

Một checkpoint MRL hỗ trợ ba cấu hình triển khai: 256 chiều khi ưu tiên tốc độ và bộ nhớ (chỉ mục nhỏ hơn 4 lần), 512 chiều cân bằng giữa chất lượng và tài nguyên (dung lượng nhỏ hơn 2 lần so với 1024 chiều), và 1024 chiều khi cần tối đa hóa chất lượng. So sánh chất lượng giữa ba cấu hình và lựa chọn cấu hình triển khai được trình bày ở Chương 4.

3.8 Reranker nhẹ bằng chứng cất tri thức

3.8.1 Động lực và thiết kế tổng quan

Mục tiêu của hướng reranker là thay BGE-reranker-v2-m3 (568M tham số) bằng các reranker nhỏ hơn nhưng vẫn giữ được chất lượng xếp hạng. Đề án thử ba mức dung lượng mô hình: MiniLM-L12 33M cho cấu hình siêu nhẹ, mmarco-mMiniLMv2 117M cho cấu hình cân bằng, và mmarco sau cắt tia còn 35.6M để giảm chi phí triển khai (dung lượng mô hình, tài nguyên bộ nhớ và chi phí suy luận) nhưng vẫn tận dụng được encoder đa ngôn ngữ mạnh.

Một quyết định thiết kế quan trọng là có cần bước thích nghi miền (giai đoạn A) trước khi chưng cất hay không. Giả thuyết là khi chưa thích nghi với miền dữ liệu đích, student chưa hình thành biểu diễn phù hợp cho đầu vào tiếng Việt — do khoảng cách miền giữa MS MARCO tiếng Anh [24] và tài liệu kỹ thuật tiếng Việt — nên việc tiếp nhận tín hiệu xếp hạng từ teacher kém hiệu quả. Vì vậy, đề án thiết kế quy trình hai giai đoạn (giai đoạn A học tương phản trước, giai đoạn B chưng cất sau); so sánh với phương án chưng cất trực tiếp được trình bày ở Chương 4.

- **Giai đoạn A – Học tương phản:** thích nghi miền bằng dữ liệu kỹ thuật tiếng Việt và hàm mất mát BCE (entropy chéo nhị phân).
- **Giai đoạn B – Chưng cất tri thức:** từ checkpoint giai đoạn A, chưng cất tín hiệu xếp hạng của BGE-reranker-v2-m3 để học mức độ liên quan tinh tế hơn.

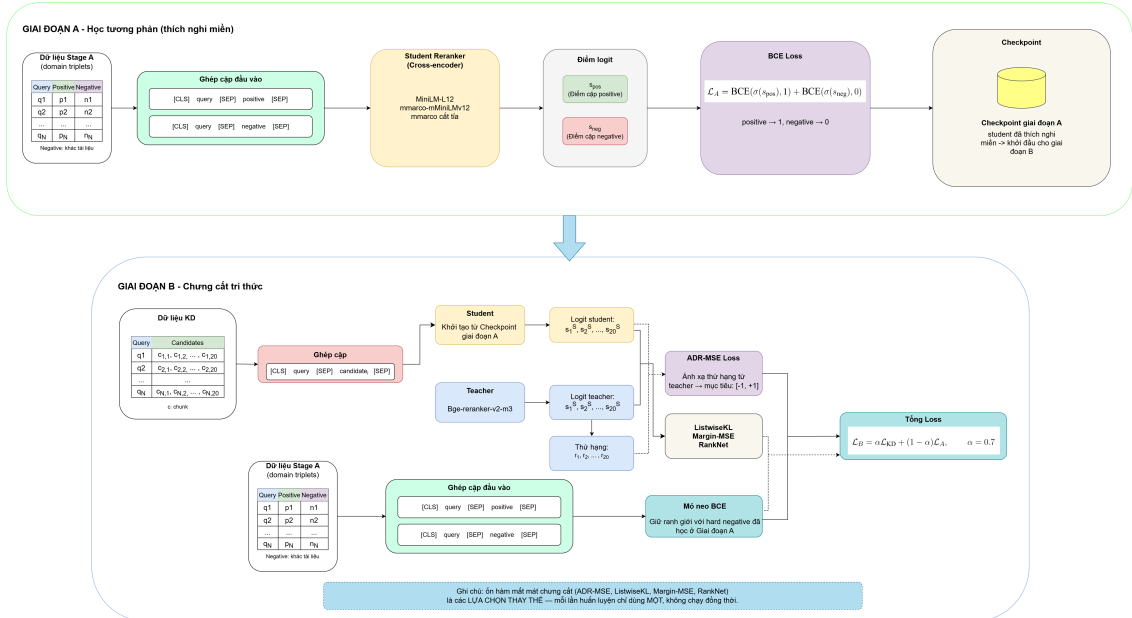


Figure 3.7: Kiến trúc huấn luyện reranker hai giai đoạn

Tín hiệu teacher từ BGE-reranker-v2-m3.

BGE-reranker-v2-m3 là mô hình cross-encoder: với mỗi cặp (q, c_i) gồm câu hỏi

và đoạn ứng viên, mô hình trả về một điểm liên quan thực s_i^T (logit), không phải xác suất đã chuẩn hóa trong khoảng $[0, 1]$. Sau khi chấm 20 ứng viên của một câu hỏi, hệ thống sắp xếp các đoạn theo s_i^T giảm dần để tạo thứ hạng teacher. Vì vậy, dữ liệu chung cất lưu hai loại tín hiệu: điểm logit thô của từng ứng viên và vị trí thứ hạng sau khi xếp hạng lại.

Điểm logit thô mang thông tin về độ tự tin tương đối nhưng không được hiệu chỉnh xác suất; khoảng cách logit có thể rất khác nhau giữa các câu hỏi. Do đó các hàm mất mát được thiết kế theo hai hướng. ListwiseKL chuyển logit của teacher và student thành phân phối mềm bằng softmax theo từng danh sách ứng viên; Margin-MSE và RankNet dùng chênh lệch logit giữa các cặp ứng viên; còn ADR-MSE bỏ qua thang điểm tuyệt đối của teacher và chỉ chung cất thứ hạng bằng cách ánh xạ thứ hạng về mục tiêu trong khoảng $[-1, +1]$. Như vậy, BGE-reranker-v2-m3 đóng vai trò cung cấp tín hiệu liên quan và thứ hạng, còn từng hàm mất mát quyết định cách chuẩn hóa hoặc khai thác tín hiệu đó trong quá trình chung cất.

3.8.2 Điều chỉnh dữ liệu cho reranker

Reranker không xây dựng lại dữ liệu từ đầu mà kế thừa kết quả trung gian của quy trình dữ liệu embedding ở mục 3.7.1: tập câu hỏi kèm ứng viên đã xếp hạng, nhãn positive do mô hình ngôn ngữ xác định, và tập negative tạo sinh. Điểm khác là reranker cần dữ liệu dạng bộ ba hoặc danh sách để học xếp hạng, nên các negative phải được lọc chặt hơn và được gộp, chia theo câu hỏi để tránh rò rỉ dữ liệu (leakage).

Cụ thể, reranker khác embedding ở ba điểm chính:

- Negative tạo sinh được kiểm tra thêm bằng một giám khảo khả năng trả lời độc lập (dùng mô hình ngôn ngữ), chỉ giữ những phiên bản không trả lời được câu hỏi.
- Negative khai thác dùng tiêu chí chặt hơn: ngoài các đoạn khác tài liệu, đoạn cùng tài liệu chỉ được giữ nếu giám khảo xác nhận là không liên quan; đồng thời loại các mẫu có chênh lệch điểm quá nhỏ.
- Việc gộp và chia cho reranker tạo các bộ ba huấn luyện và kiểm định theo câu hỏi với tỉ lệ 90/10, khác với tỉ lệ 80/20 của embedding.

Lọc negative tạo sinh.

Nguồn tạo sinh ban đầu gồm 1.291 ứng viên từ các phép sửa như thay đổi thực thể, thay đổi thời gian, đảo ngược logic và thay đổi nguồn gốc. Với reranker, mỗi negative đi qua ba tầng lọc: tự kiểm tra trong prompt sinh dữ liệu, giám khảo khả năng trả lời độc lập, và bộ lọc theo chênh lệch điểm BGE-reranker-v2-m3. Sau lọc,

405 ứng viên được giữ lại làm hard negative thật sự cho reranker.

Table 3.7: Thống kê negative tạo sinh sau lọc

Loại chỉnh sửa	Tổng sinh	Giữ lại	Tỉ lệ
Thay đổi thực thể	599	195	32.6%
Thay đổi thời gian	606	183	30.2%
Đảo ngược logic	55	18	32.7%
Thay đổi nguồn gốc	31	9	29.0%
Tổng	1.291	405	31.4%

Tỉ lệ loại 68.6% phản ánh tiêu chuẩn nghiêm ngặt: chỉ các negative khó nhưng không trả lời được câu hỏi mới được dùng cho reranker.

Khai thác negative cho reranker.

Reranker cũng dùng các ứng viên đã xếp hạng từ BGE-reranker-v2-m3, nhưng tiêu chí khai thác khác embedding. Với các đoạn ngoài top-5 đã được giám khảo đánh giá, hệ thống xét các ứng viên ở hạng 6 đến 20; riêng trong top-5, những đoạn được xác nhận là không liên quan vẫn có thể được giữ. Các đoạn liên quan trực tiếp hoặc một phần luôn bị loại. Đoạn cùng tài liệu với positive chỉ được giữ nếu được xác nhận không liên quan, còn đoạn khác tài liệu được giữ nếu vượt ngưỡng chênh lệch điểm. Chênh lệch điểm được dùng để phân loại độ khó: khó trong khoảng 0.1–0.4, trung bình trong khoảng 0.4–0.7, và dễ khi lớn hơn 0.7.

Quy trình thu được 9.917 negative khai thác, trong đó 913 negative khó và 1.676 negative trung bình được sử dụng cho tập reranker.

Table 3.8: Thống kê negative khai thác theo nguồn

Nguồn	Số negatives	Tỉ lệ
Khác tài liệu	9.475	95.5%
Cùng tài liệu và được xác nhận không liên quan	442	4.5%
Tổng	9.917	100%

Gộp và chia tập cho reranker.

Negative tạo sinh sau lọc (405) và negative khai thác khó cùng trung bình (2.589) được gộp thành 2.994 bộ ba. Dữ liệu được chia theo câu hỏi thay vì theo bộ ba, để cùng một câu hỏi không xuất hiện ở cả tập huấn luyện và tập kiểm định. Tỉ lệ 90/10 tạo 2.689 bộ ba huấn luyện từ 439 câu hỏi và 305 bộ ba kiểm định từ 50 câu hỏi.

Table 3.9: Tổng quan tập dữ liệu huấn luyện reranker

Nguồn negative	Số bộ ba	Độ khó	Tỉ lệ
Negative tạo sinh	405	khó	13.5%
Negative khai thác khó	913	khó	30.5%
Negative khai thác trung bình	1.676	trung bình	56.0%
Tổng	2.994	—	100%

3.8.3 Giai đoạn A: Học tương phản

Giai đoạn A tinh chỉnh MiniLM-L12 bằng hàm mất mát entropy chéo nhị phân (BCE) trên các bộ ba trong miền:

$$\mathcal{L}_A = \text{BCE}(\sigma(s_{\text{pos}}), 1) + \text{BCE}(\sigma(s_{\text{neg}}), 0) \quad (3.6)$$

$$\mathcal{L}_A = -\log \sigma(s_{\text{pos}}) - \log (1 - \sigma(s_{\text{neg}})) \quad (3.7)$$

trong đó s_{pos} và s_{neg} là logit của MiniLM cho đoạn positive và đoạn negative.

Một câu hỏi thiết kế ở giai đoạn A là có nên bổ sung dữ liệu mMARCO-VI [25] hay không. Động cơ là MiniLM chỉ huấn luyện trên tiếng Anh nên chưa có biểu diễn tốt cho tiếng Việt; một lượng dữ liệu truy hồi tiếng Việt tổng quát có thể giúp student làm quen với ngôn ngữ trước khi học phân biệt hard negative kỹ thuật. Tuy nhiên, mMARCO-VI là dữ liệu tổng quát, khác phân bố với miền kỹ thuật của đề án, nên lượng bổ sung cần cân nhắc: quá ít thì không đủ tín hiệu ngôn ngữ, quá nhiều thì có thể lấn át dữ liệu miền đích. Đề án khảo sát ảnh hưởng của liều lượng mMARCO-VI (0, 50k, 100k mẫu) ở giai đoạn A; kết quả và phân tích phân bố độ khó của negative được trình bày ở Chương 4. Với các student mmarco-mMiniLMv2 và mmarco sau cắt tia, do đã có encoder đa ngôn ngữ mạnh, đề án không bổ sung mMARCO-VI ở giai đoạn A.

Table 3.10: Siêu tham số của giai đoạn A

Tham số	Giá trị
Tốc độ học	2×10^{-5}
Kích thước batch	32
Số epoch	5
Dừng sớm (patience)	2
Hạt giống ngẫu nhiên	42
Bộ tối ưu	AdamW

Một vấn đề thiết kế tiếp theo là cách tổ chức dữ liệu trong giai đoạn thích nghi:

trộn mMARCO-VI với dữ liệu miền ngay từ đầu (2-stage), hay tách thành một lộ trình tuần tự học tiếng Việt trước rồi mới thích nghi miền (3-stage). Với phương án tuần tự, tốc độ học ở giai đoạn thích nghi miền cần được giảm để bảo toàn biểu diễn tiếng Việt đã học, tránh việc tốc độ học quá lớn ghi đè tri thức từ giai đoạn trước. So sánh giữa hai cách tổ chức và vai trò của tốc độ học được trình bày ở Chương 4.

3.8.4 Giai đoạn B: Chung cất tri thức

Giai đoạn B chung cất BGE-reranker-v2-m3 vào checkpoint giai đoạn A. Với mỗi câu hỏi, teacher và student cùng chấm điểm 20 ứng viên, tạo hai vector logit $\mathbf{s}^T$ và $\mathbf{s}^S$ trên cùng một danh sách ứng viên. Hàm mất mát kết hợp tín hiệu chung cất và mỏ neo tương phản:

$$\mathcal{L}_B = \alpha \mathcal{L}_{KD} + (1 - \alpha) \mathcal{L}_A, \quad \alpha = 0.7 \quad (3.8)$$

Với cấu hình này, thành phần chung cất có trọng số 0.7, còn thành phần mỏ neo tương phản có trọng số $1 - \alpha = 0.3$. Thành phần mỏ neo giúp student duy trì khả năng phân biệt các negative trong miền kỹ thuật đã học ở giai đoạn A, thay vì chỉ tối ưu theo tín hiệu xếp hạng của teacher. Việc lựa chọn α và so sánh với cấu hình chỉ dùng chung cất ($\alpha = 1.0$) được phân tích ở Chương 4.

Listwise KL Divergence

$$\mathcal{L}_{KL} = \sum_i P_i^T \log \frac{P_i^T}{P_i^S}, \quad P_i^T = \frac{\exp(s_i^T/\tau)}{\sum_j \exp(s_j^T/\tau)}, \quad P_i^S = \frac{\exp(s_i^S/\tau)}{\sum_j \exp(s_j^S/\tau)} \quad (3.9)$$

Với $\tau = 2.0$, softmax làm mượt logit thô của teacher và tạo phân phối tương đối trong phạm vi 20 ứng viên của từng câu hỏi. Phân phối này không phải xác suất liên quan tuyệt đối, mà chỉ thể hiện mức ưu tiên tương đối giữa các ứng viên trong cùng một danh sách. Hàm mất mát tối ưu để phân phối của student tiến gần phân phối của teacher. Trong thực nghiệm, Listwise KL nhạy với các câu hỏi nhiễu; việc lọc còn 792 câu hỏi có positive rõ ràng giúp quá trình huấn luyện ổn định hơn.

Margin-MSE

$$\mathcal{L}_{MMSE} = \frac{1}{|P|} \sum_{(i,j) \in P} [(s_i^S - s_j^S) - (s_i^T - s_j^T)]^2 \quad (3.10)$$

Margin-MSE học khoảng cách tương đối giữa các ứng viên. Tuy nhiên, do logit

của teacher có biên độ rộng và một tỉ lệ đáng kể các cặp là cặp dễ với margin lớn, ở giai đoạn đầu khi margin của student còn nhỏ, các cặp dễ có margin teacher lớn có thể tạo gradient mạnh hơn nhiều so với các hard negative có margin nhỏ. Điều này khiến hàm mất mát dễ bị chi phối bởi các cặp vốn đã dễ phân biệt. Phân bố margin teacher cụ thể được trình bày ở Chương 4.

RankNet với nhãn mềm của teacher.

$$P_{ij}^T = \sigma(s_i^T - s_j^T), \quad P_{ij}^S = \sigma(s_i^S - s_j^S) \quad (3.11)$$

$$\mathcal{L}_{\text{RN}} = -\frac{1}{n^2 - n} \sum_{i \neq j} [P_{ij}^T \log P_{ij}^S + (1 - P_{ij}^T) \log(1 - P_{ij}^S)] \quad (3.12)$$

Với 20 ứng viên, RankNet tối ưu 380 cặp có thứ tự cho mỗi câu hỏi. Tín hiệu theo từng cặp này phong phú hơn so với chỉ học một positive, nhưng vẫn chịu ảnh hưởng bởi thang điểm của logit teacher: khi chênh lệch logit quá lớn, sigmoid dễ bão hòa và làm giảm độ nhạy giữa các mức liên quan gần nhau.

ADR-MSE (Asymmetric Distillation Ranking MSE)

ADR-MSE là một hàm mất mát theo hướng chưng cất dựa trên thứ hạng, được kế thừa từ Rank-DistILLM [10]. Thay vì dùng điểm teacher, hàm mất mát sử dụng vị trí thứ hạng $r_i \in \{0, \dots, n-1\}$ và ánh xạ tuyến tính về $[-1, +1]$:

$$\tilde{r}_i = 1 - \frac{2r_i}{n-1} \quad (3.13)$$

Logit của student được chuẩn hóa theo danh sách ứng viên:

$$\tilde{s}_i = \frac{s_i^S - \mu_s}{\sigma_s + \epsilon} \quad (3.14)$$

$$\mathcal{L}_{\text{ADR}} = \frac{1}{n} \sum_{i=1}^n (\tilde{s}_i - \tilde{r}_i)^2 \quad (3.15)$$

Vị trí thứ hạng là tín hiệu thứ tự, không phụ thuộc vào thang điểm tuyệt đối của teacher. Phép ánh xạ tạo mục tiêu bị chặn và cách đều; ADR-MSE gán trọng số bằng nhau cho các vị trí trong phiên bản cơ sở.

Table 3.11: Ví dụ minh họa cách tính ADR-MSE với năm ứng viên

Ứng viên	Hạng r_i	Mục tiêu $\tilde{r}_i$	Logit s_i	Điểm $\tilde{s}_i$	Thành phần loss
Doc A (positive)	0	+1.00	3.2	+1.455	0.207
Doc B	1	+0.50	1.8	+0.712	0.045
Doc C	2	0.00	0.3	-0.085	0.007
Doc D	3	-0.50	-0.9	-0.722	0.049
Doc E	4	-1.00	-2.1	-1.359	0.129

Với giá trị trung bình logit $\mu_s = 0.460$ và độ lệch chuẩn $\sigma_s = 1.883$, ví dụ trên cho:

$$\mathcal{L}_{\text{ADR}} \approx \frac{0.207 + 0.045 + 0.007 + 0.049 + 0.129}{5} \approx 0.088 \quad (3.16)$$

Table 3.12: Siêu tham số của giai đoạn B

Tham số	Giá trị
Trọng số chung cắt α	0.7
Tốc độ học	1×10^{-5}
Kích thước batch	8 câu hỏi, 20 ứng viên/câu
Nhiệt độ Listwise KL τ	2.0
Số epoch	5
Dừng sớm	Patience 2 theo MRR@10 trên tập kiểm định

3.8.5 Cắt tỉa từ vựng cho mmarco

Bên cạnh MiniLM 33M và mmarco 117M, đồ án thử nén mmarco bằng từ vựng cắt tỉa. Động cơ xuất phát từ cấu trúc tham số của mmarco-mMiniLMv2: model dùng từ vựng XLM-R gồm 250,002 tokens cho 100 ngôn ngữ, nên riêng embedding matrix đã chiếm $250,002 \times 384 \approx 96\text{M}$ tham số, tương đương hơn 80% tổng số tham số (117.6M). Trong khi đó, corpus của đồ án chỉ gồm tiếng Việt và tiếng Anh kỹ thuật, sử dụng một phần nhỏ từ vựng này; phần lớn token của các ngôn ngữ khác không bao giờ xuất hiện.

Ý tưởng là cắt embedding matrix theo các bước sau, giữ nguyên transformer encoder:

1. Thu thập toàn bộ tokens thực sự xuất hiện trong corpus (2,317 đoạn) và queries (991 câu).
2. Giữ lại: các special tokens ([CLS], [SEP], [PAD], [UNK]), toàn bộ tokens xuất hiện trong dữ liệu, cùng 30,000 tokens phổ biến nhất của từ vựng gốc làm dự phòng cho tài liệu mới.

3. Tổng cộng giữ 36,324 trong 250,002 tokens (loại 85.5%).
4. Slice embedding matrix theo các token được giữ, remap token IDs, và rebuild tokenizer với từ vựng Unigram mới.

Cách này giảm tổng số tham số từ 117.6M xuống 35.6M, tức giảm 69.7% dung lượng. Trên corpus mới, UNK rate chỉ 0.225%, cho thấy việc loại 85.5% từ vựng gần như không làm mất token tiếng Việt quan trọng. Vì transformer encoder, phần đảm nhiệm gần như toàn bộ chi phí tính toán được giữ nguyên, độ trễ không giảm tỷ lệ thuận với số tham số: thao tác tra cứu embedding là $O(1)$, nên cắt embedding matrix chỉ giảm bộ nhớ chứ không giảm thời gian suy luận. Lợi ích chính của cắt tia vì vậy là giảm dung lượng model và yêu cầu bộ nhớ (phù hợp triển khai trên thiết bị hạn chế tài nguyên), đồng thời vẫn giữ nguyên năng lực đa ngôn ngữ của encoder mmarco. Đánh giá chất lượng chi tiết được trình bày ở Chương 4 (mục 4.2.3).

3.9 Tinh chỉnh bộ phân loại ý định Sentence-BERT

3.9.1 Kiến trúc và dữ liệu

Bộ phân loại ý định v2 được xây dựng để phân biệt hai lớp: `tra_cuu`, tương ứng với các câu hỏi yêu cầu tra cứu thông tin có sẵn trong tài liệu, và `ting_toan`, tương ứng với các câu hỏi yêu cầu suy luận hoặc tính toán để tạo ra kết quả mới. Backbone của bộ phân loại là mô hình Sentence-BERT tiếng Việt keepitreal/vietnamese-sbert [26], dựa trên PhoBERT-base, với embedding đầu ra 768 chiều.

Thay vì gắn một head tuyến tính lên token [CLS] và tinh chỉnh toàn bộ mô hình bằng cross-entropy, đề án sử dụng phương pháp SetFit. Cụ thể, encoder được tinh chỉnh bằng contrastive learning trên các cặp câu hỏi, sau đó một head Logistic Regression nhẹ được huấn luyện trên embedding câu. Trước khi đưa vào mô hình, câu hỏi được tách từ bằng Pyvi do các mô hình dựa trên PhoBERT yêu cầu đầu vào đã được tách từ. Sau đó, câu hỏi được encode thành embedding bằng mean pooling và chuẩn hóa L2:

$$\mathbf{e}_q = \text{MeanPool}(\text{SBERT}(\text{segment}(q))) \in \mathbb{R}^{768}, \quad \|\mathbf{e}_q\|_2 = 1 \quad (3.17)$$

$$\hat{y} = \text{LogisticRegression}(\mathbf{e}_q) \in \{\text{tra_cuu}, \text{ting_toan}\} \quad (3.18)$$

Trong giai đoạn contrastive fine-tuning, encoder được huấn luyện bằng CosineSimilarityLoss trên các cặp câu. Các cặp cùng ý định được gán target tương đồng bằng 1, trong khi các cặp khác ý định được gán target bằng 0. Cơ chế này

giúp kéo các câu hỏi cùng ý định lại gần nhau và đẩy các câu hỏi khác ý định ra xa trong không gian embedding.

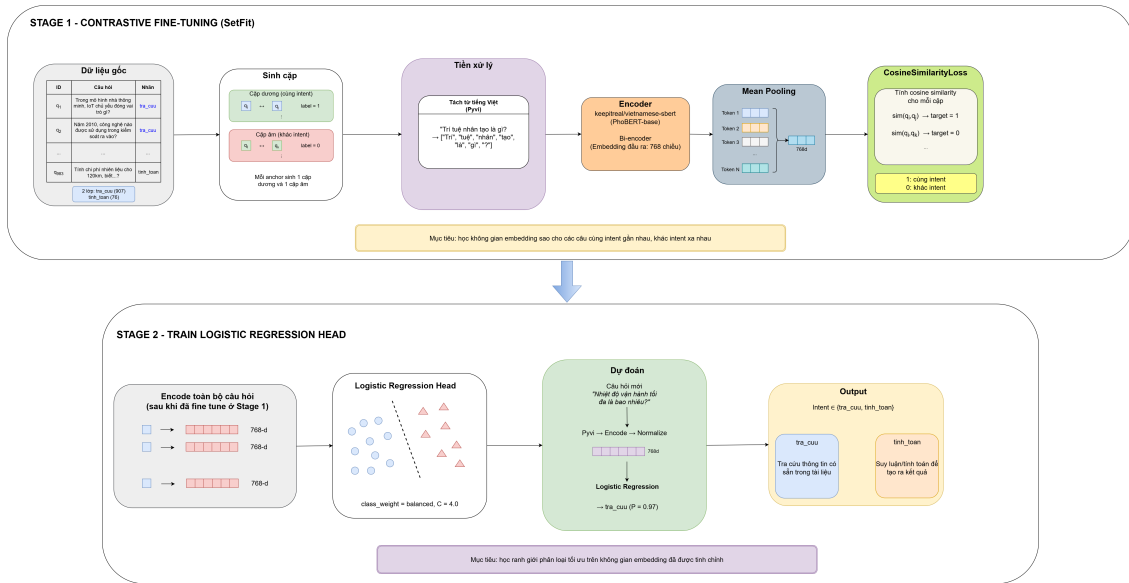


Figure 3.8: Kiến trúc huấn luyện bộ phân loại ý định theo SetFit

Việc lựa chọn SetFit thay cho kiến trúc [CLS] + Linear xuất phát từ quan sát rằng embedding chưa tinh chỉnh không phân biệt tốt hai lớp: mô hình dễ nhầm các câu `tra_cuu` có chứa số liệu thành `inh_toan`, ví dụ câu hỏi “Hằng số điện môi ε_0 có giá trị bao nhiêu?” — thực chất là tra cứu một giá trị có sẵn, không phải yêu cầu tính toán. Ranh giới giữa “thông tin có sẵn” và “kết quả cần tính toán” không được biểu diễn đủ tốt nếu chỉ huấn luyện head phân loại trên embedding đông cứng. Do đó, đề án tinh chỉnh encoder bằng contrastive learning để học biểu diễn phù hợp hơn cho bài toán. So sánh định lượng giữa hai phương án được trình bày ở Chương 4.

Để kiểm tra chất lượng nhãn, đề án phân tích các cặp câu gần giống nhưng khác nhãn, với ngưỡng cosine similarity lớn hơn 0.8. Kết quả chỉ phát hiện một trường hợp mâu thuẫn nhãn thực sự trên toàn bộ tập dữ liệu, cho thấy dữ liệu có độ nhất quán cao và lỗi phân loại chủ yếu đến từ hạn chế biểu diễn của mô hình, không phải do nhiễu nhãn.

Tập dữ liệu ban đầu gồm 991 câu hỏi được gán nhãn thủ công; sau khi loại bỏ trùng lặp còn 983 câu. Do lớp `inh_toan` là lớp thiểu số, một tập kiểm thử cố định có thể chứa quá ít mẫu tính toán để phản ánh ổn định chất lượng mô hình. Vì vậy, đề án sử dụng stratified 5-fold cross-validation để đánh giá. Mô hình triển khai cuối cùng được huấn luyện lại trên toàn bộ tập dữ liệu sạch. Ví dụ, câu hỏi “Theo tài liệu Public_021, nhiệt độ vận hành tối đa là bao nhiêu?” thuộc lớp `tra_cuu`, trong khi câu hỏi yêu cầu tính chi phí nhiên liệu từ quãng đường, mức tiêu hao và

giá xăng thuộc lớp `tin_h_toan`.

3.9.2 Xử lý mất cân bằng lớp

`tra_cuu` chiếm 907/983 câu (khoảng 92.3%), còn `tin_h_toan` chỉ có 76/983 câu (khoảng 7.7%), tỉ lệ xấp xỉ 12:1. Một classifier luôn dự đoán lớp đa số đã đạt accuracy khoảng 92%, do đó accuracy là metric dễ gây hiểu nhầm; F1-macro được dùng để đánh giá bình đẳng hai lớp. Mất cân bằng được xử lý ở hai tầng.

Lấy mẫu tăng cường cặp tương phản.

Khi sinh cặp huấn luyện cho SetFit, các cặp được lấy mẫu cân bằng theo lớp (oversampling) thay vì giữ tỉ lệ tự nhiên 92/8, để lớp thiểu số xuất hiện đủ trong cả cặp dương lẫn cặp âm. Mỗi anchor sinh một cặp dương và một cặp âm, nên tổng số cặp xấp xỉ $2 \times T \times N$, với T là số vòng lặp sinh cặp.

Trọng số lớp ở bộ phân loại.

Bộ phân loại Logistic Regression dùng trọng số lớp cân bằng (balanced): trọng số mỗi lớp tỉ lệ nghịch với tần suất, $w_c = n/(K \cdot n_c)$ với $K = 2$ lớp:

$$w_{\text{tra_cuu}} = \frac{983}{2 \times 907} \approx 0.54, \quad w_{\text{tin_h_toan}} = \frac{983}{2 \times 76} \approx 6.47 \quad (3.19)$$

Lỗi trên lớp `tin_h_toan` vì vậy bị phạt mạnh hơn khoảng 12 lần.

Table 3.13: Siêu tham số tinh chỉnh Sentence-BERT (SetFit) cho bộ phân loại ý định

Tham số	Giá trị
Backbone	keepitreal/vietnamese-sbert (PhoBERT-base)
Contrastive loss	CosineSimilarityLoss
Số vòng lặp sinh cặp	20 (sinh $\approx 2 \cdot 20 \cdot N$ cặp)
Chiến lược lấy mẫu	oversampling (lấy mẫu tăng cường)
Epochs	1
Batch size	16
Learning rate	2×10^{-5}
Warmup ratio	0.1
Mixed precision	FP16
Optimizer	AdamW
Bộ phân loại	Logistic Regression, trọng số lớp cân bằng, $C=4.0$

Đánh giá theo F1-macro (qua stratified 5-fold cross-validation) đảm bảo không tối ưu nhằm vào accuracy tổng thể khi recall của lớp thiểu số còn thấp. Kết quả định lượng (F1-macro của SetFit so với baseline embedding đông cứng, và so sánh

các backbone PhoBERT-base) được trình bày ở Chương 4.

3.9.3 Logic định tuyến trong pipeline v2

Trong v1, query được gửi tới GPT-4o-mini qua API để nhận ý định label. Trong v2, query được tách từ và chạy qua Sentence-BERT local; logic sau classification không đổi: tra_cuu dùng direct prompt, còn tinh_toan dùng Chain-of-Thought prompt. Regex trích xuất Public ID chạy song song với classifier.

Sentence-BERT local loại bỏ phụ thuộc vào external API và giữ dữ liệu câu hỏi trong hạ tầng nội bộ, phù hợp với yêu cầu bảo mật của môi trường doanh nghiệp. Các số liệu định lượng về ý định accuracy và độ trễ định tuyến được trình bày ở Chương 4 trên cùng bộ 390 câu MCQ.

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM

4.1 Thiết lập thực nghiệm

4.1.1 Môi trường và phần cứng

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên hai môi trường. Huấn luyện reranker và embedding sử dụng GPU NVIDIA RTX 6000, cấu hình đủ để duy trì batch size lớn cho MNRL (128) và chạy teacher BGE-reranker-v2-m3 song song với student trong giai đoạn B.

Các phép đo xếp hạng và pipeline độ trễ mới trong chương này được thực hiện trên RTX 6000 để so sánh các cấu hình production trong cùng một môi trường. Riêng phép đo đầu cuối MCQ được thực hiện trong môi trường CPU/API trên máy cá nhân, dùng để so sánh ảnh hưởng tổng thể của pipeline v1 và v2 tới độ chính xác và độ trễ.

4.1.2 Tập dữ liệu đánh giá

Đồ án sử dụng hai tập đánh giá chính cho truy hồi và xếp hạng. D1 là tập in-domain dùng để kiểm tra chất lượng embedding trên miền huấn luyện. D2 là tập độc lập gồm 343 queries trên 50 PDF kỹ thuật tiếng Việt chưa dùng khi huấn luyện; cùng một bộ D2 này được dùng làm tập kiểm thử zero-shot cho embedding, reranker và các phân tích pipeline liên quan. Như vậy, kết quả embedding độc lập và kết quả reranker được đo trên cùng nguồn tài liệu/câu hỏi, chỉ khác phạm vi xếp hạng: embedding xếp hạng toàn corpus, còn reranker xếp hạng lại candidate pool top-20 do embedding tạo ra.

Table 4.1: Các tập dữ liệu dùng trong đánh giá

Tập	Quy mô	Mục đích
D1 in-domain embedding	157 queries, 2,317 đoạn	So sánh các cấu hình embedding truy hồi, tài liệu thuộc miền huấn luyện
D2 độc lập embedding/reranker	343 queries, 813 đoạn từ 50 PDF mới	Đánh giá zero-shot chung cho embedding và reranker trên tài liệu mới
Dev reranker	305 triplets, 50 queries	Theo dõi và lựa chọn checkpoint trong quá trình huấn luyện

Tập D2 (343 queries, 50 PDF) hoàn toàn tách biệt với tập phát triển v1 (991 câu, 200 PDF), không có tài liệu nào trùng lặp. Đây là điều kiện zero-shot: model chưa từng thấy bất kỳ đoạn nào từ 50 PDF này trong quá trình huấn luyện. Kết quả trên tập này do đó phản ánh khả năng tổng quát hóa của model, không phải học thuộc

dữ liệu.

4.1.3 Độ đo đánh giá

Đánh giá reranker sử dụng bốn metrics tiêu chuẩn trong information truy hồi: Hit@1 (tỉ lệ gold đoạn xuất hiện ở rank 1), Recall@5 (tỉ lệ gold đoạn xuất hiện trong top-5), MRR@10 (Mean Reciprocal Rank, đo vị trí trung bình của gold đoạn) và NDCG@10 (Normalized Discounted Cumulative Gain, đo chất lượng toàn bộ thứ tự xếp hạng). Metric chính để so sánh là MRR@10 và Recall@5 vì chúng phản ánh trực tiếp hiệu quả của reranker trong pipeline RAG thực tế, nơi top-5 đoạn được đưa vào LLM.

Đánh giá embedding.

Các metrics gồm Acc@1, Acc@5, MRR@10 và NDCG@10 cho embedding. D1 (in-domain) gồm 157 câu trên corpus 2,317 đoạn dùng làm benchmark phát triển chính. D2 (độc lập) gồm 343 câu trên corpus 813 đoạn của 50 tài liệu mới, đồng thời là tập test chung với reranker.

Đánh giá đầu cuối sử dụng accuracy trên 390 câu hỏi trắc nghiệm (exact match với đáp án A/B/C/D), kết hợp đo độ trễ (ms/câu) và so sánh trực tiếp pipeline v1 với v2.

4.2 Đánh giá reranker

4.2.1 Kết quả chính trên tập kiểm thử reranker

Tập test reranker chính là D2 gồm 343 queries zero-shot. Candidate pool là top-20 đoạn do đã tinh chỉnh embedding 512d (curriculum GIST→MNRL) truy xuất từ toàn bộ corpus 813 đoạn, đúng cấu hình pipeline thực tế ở v2. Khi chưa rerank, riêng embedding đạt $\text{MRR@10} = 0.8023$ trên candidate pool top-20, đây là điểm tham chiếu để đánh giá đóng góp của từng reranker.

Các model trong Bảng 4.2 bao phủ ba nhóm so sánh.

BGE reranker là teacher và là mốc chất lượng mạnh nhất của v1.

ViRanker [27] và PhoRanker [28] là các cross-encoder reranker tiếng Việt công khai. mmarco-mMiniLMv2 base là reranker đa ngôn ngữ đã học passage xếp hạng từ mMARCO, dùng để tách đóng góp của năng lực pretrained khỏi đóng góp của domain distillation. MiniLM-L12 base [29] là baseline 33M rất nhẹ, đại diện cho trường hợp ưu tiên độ trễ nhưng chưa thích nghi với tiếng Việt kỹ thuật. Bảng 4.2 so sánh teacher, các student sau distillation và các baseline này trên cùng candidate pool.

Table 4.2: Kết quả reranker trên 343 câu hỏi

Model	Params	Hit@1	Recall@5	MRR@10	NDCG@10
BGE-reranker-v2-m3 teacher	568M	0.8192	0.9563	0.8804	0.9007
mmarco-mMiniLMv2 + ADR-MSE	117M	0.8163	0.9621	0.8753	0.8979
Pruned mmarco + ADR-MSE	35.6M	0.8134	0.9504	0.8718	0.8961
ViRanker SOTA VI	568M	0.7638	0.9534	0.8434	0.8703
mmarco-mMiniLMv2 base	117M	0.7522	0.9359	0.8352	0.8670
MiniLM-L12 + ADR-MSE (with mMARCO)	33M	0.6939	0.9213	0.7869	0.8201
PhoRanker VI (segmented)	135M	0.6910	0.8921	0.7772	0.8083
MiniLM-L12 base	33M	0.5219	0.8338	0.6520	0.7060

Ghi chú: BGE-reranker-v2-m3 teacher được dùng làm mốc tham chiếu chất lượng. In đậm biểu thị student tốt nhất cho cấu hình triển khai.

Student mmarco-mMiniLMv2 117M sau distillation đạt $\text{MRR@10} = 0.8753$, tương đương 99.4% so với teacher BGE-reranker-v2-m3, trong khi số tham số nhỏ hơn khoảng 4.8 lần. Từ vệt cắt tỉa tạo ra phiên bản pruned 35.6M gần như không làm suy giảm chất lượng: pruned đạt $\text{MRR@10} = 0.8718$, bằng 99.6% so với mmarco 117M và 99.0% so với teacher.

So với ViRanker, pruned 35.6M cao hơn 0.0284 điểm MRR@10 dù có số tham số nhỏ hơn khoảng 16 lần. Kết quả này cho thấy dữ liệu đúng miền và quá trình distillation phù hợp có thể quan trọng hơn việc chỉ tăng kích thước mô hình hoặc sử dụng dữ liệu tổng quát quy mô lớn. Đối với PhoRanker, văn bản đầu vào được tách từ bằng pyvi trước khi đánh giá để phù hợp với cách tiền xử lý của mô hình này. Ngay cả trong thiết lập đó, các student sau distillation vẫn vượt PhoRanker trên toàn bộ các metric.

MiniLM-L12 33M vẫn là lựa chọn siêu nhẹ: MRR@10 tăng từ 0.6520 lên 0.7869 sau tinh chỉnh ở cấu hình có bổ sung mMARCO tại giai đoạn A, đồng thời vượt PhoRanker 135M trên toàn bộ các metric. Tuy nhiên, trên candidate pool sinh bởi bộ truy hồi mạnh, MiniLM 33M chỉ đạt $\text{MRR@10} = 0.7869$, thấp hơn baseline embedding-only chưa rerank là 0.8023. Điều này cho thấy khi bộ truy hồi đã đủ mạnh, mô hình 33M không còn đủ năng lực để cải thiện thứ hạng như các student 35.6M hoặc 117M.

4.2.2 Nghiên cứu loại bỏ thành phần

Table 4.3: So sánh cấu hình huấn luyện trên MiniLM, mmarco và pruned

Cấu hình	Hit@1	Recall@5	MRR@10	NDCG@10
MiniLM KD trực tiếp, không giai đoạn A	0.6297	0.8892	0.7436	0.7862
MiniLM giai đoạn A+B, no mMARCO, $\alpha = 0.7$	0.6501	0.9096	0.7572	0.7983
MiniLM giai đoạn A+B, no mMARCO, $\alpha = 1.0$	0.6501	0.8980	0.7508	0.7914
MiniLM giai đoạn A+B, with mMARCO, $\alpha = 0.7$	0.6939	0.9213	0.7869	0.8201
mmarco ADR-MSE, $\alpha = 0.7$	0.8163	0.9621	0.8753	0.8979
mmarco ADR-MSE, $\alpha = 1.0$	0.7959	0.9563	0.8634	0.8886
Pruned ADR-MSE, $\alpha = 0.7$	0.8134	0.9504	0.8718	0.8961
Pruned ADR-MSE, $\alpha = 1.0$	0.8017	0.9504	0.8650	0.8911

Ba kết luận chính rút ra từ Bảng 4.3. Thứ nhất, giai đoạn A cần thiết cho MiniLM: KD trực tiếp không giai đoạn A chỉ đạt $\text{MRR@10} = 0.7436$, thấp hơn cấu hình có giai đoạn A (0.7572), cho thấy domain adaptation giúp student tiếp nhận xếp hạng signal tốt hơn. Thứ hai, CL anchor với $\alpha = 0.7$ tốt hơn KD-only ($\alpha = 1.0$) trên cả ba kiến trúc: MiniLM +0.0064, mmarco +0.0119 và pruned +0.0068 MRR@10 ; giữ lại một phần tín hiệu contrastive giúp student không trôi khỏi vùng phân biệt domain đã học ở giai đoạn A. Thứ ba, thêm dữ liệu mMARCO-VI generic ở giai đoạn A có lợi cho MiniLM English-only: cấu hình with-mMARCO đạt $\text{MRR@10} = 0.7869$, cao hơn no-mMARCO 0.7572 (+0.0297), vì MiniLM cần thêm nền tảng tiếng Việt mà domain data nhỏ chưa cung cấp đủ. Hiệu ứng này chỉ áp dụng cho MiniLM; các student mmarco và pruned đã có encoder đa ngôn ngữ mạnh nên không dùng mMARCO.

Table 4.4: So sánh các hàm mất mát chung cất trên MiniLM 33M (giai đoạn A no-mMARCO, giai đoạn B $\alpha = 0.7$)

Loss	Teacher signal	Hit@1	Recall@5	MRR@10	NDCG@10
ADR-MSE	Rank positions	0.6501	0.9096	0.7572	0.7983
Discounted ADR-MSE	Rank + NDCG weights	0.6239	0.8921	0.7409	0.7829
ListwiseKL	Score distribution	0.6297	0.8863	0.7371	0.7780
Top-K ADR-MSE ($k = 10$)	Top-10 ranks	0.6239	0.8834	0.7356	0.7787
Margin-MSE	Score margins	0.6152	0.8892	0.7334	0.7778
RankNet	Pairwise probabilities	0.6122	0.8892	0.7311	0.7764

ADR-MSE tốt nhất, cao hơn loss xếp thứ hai (Discounted ADR-MSE) 0.0163 MRR@10 , vì dùng rank positions đã chuẩn hóa trong $[-1, +1]$, tránh scale mis-

match của teacher logits. Margin-MSE và RankNet bị ảnh hưởng bởi teacher margin range rộng, trong khi Discounted ADR và Top-K ADR làm mất một phần tín hiệu của candidate list nhỏ chỉ gồm 20 vị trí.

Liều lượng mMARCO-VI ở giai đoạn A.

Bảng 4.3 cho thấy thêm mMARCO-VI có lợi cho MiniLM, nhưng câu hỏi tiếp theo là dùng bao nhiêu là đủ. Bảng 4.5 khảo sát ba mức liều lượng. Thêm 50,000 mẫu nâng MRR@10 từ 0.7357 lên 0.7465, nhưng tăng tiếp lên 100,000 mẫu lại làm giảm xuống 0.7243, thấp hơn cả khi không dùng mMARCO-VI.

Table 4.5: Ảnh hưởng của tập MMARCO-VI ở giai đoạn A trên MiniLM

Cấu hình	Hit@1	Recall@5	MRR@10	NDCG@10
giai đoạn A, không mMARCO (0k)	0.6239	0.8863	0.7357	0.7792
Giai đoạn A + mMARCO 50k	0.6443	0.8863	0.7465	0.7843
giai đoạn A + mMARCO 100k	0.6152	0.8630	0.7243	0.7671

Phân tích phân bố độ khó của negative giải thích hiện tượng “sweet spot” này. Khi chấm bằng cross-encoder BGE-reranker-v2-m3 (logit thô), margin trung bình giữa positive và negative của mMARCO-VI là 8.94, với 78.5% mẫu thuộc nhóm easy negative (margin > 5); trong khi negative của dữ liệu kỹ thuật có margin trung bình chỉ 4.13 và chỉ 37.0% là easy. Điểm negative trung bình của mMARCO-VI (−5.68) cũng thấp hơn nhiều so với domain (−2.96), cho thấy negative của mMARCO-VI dễ phân biệt hơn hẳn. Như vậy mMARCO-VI chủ yếu cung cấp easy negative generic — chúng bổ sung nền tảng tiếng Việt cho MiniLM English-only nhưng không dạy phân biệt hard negative kỹ thuật. Ở liều 50k, lượng easy negative này đủ giúp student làm quen ngôn ngữ; ở liều 100k, lượng dữ liệu generic lớn lấn át 2,994 triplet domain, làm model lệch về phân bố tổng quát và giảm khả năng phân biệt hard negative đặc thù.

Cách tổ chức curriculum thích nghi miền.

Một câu hỏi thiết kế khác là nên trộn mMARCO-VI với dữ liệu miền ngay từ đầu (2-stage) hay tách thành curriculum tuần tự học tiếng Việt trước rồi mới thích nghi miền (3-stage). Bảng 4.6 so sánh hai cách trên MiniLM. Cấu hình 2-stage trộn đạt $\text{MRR@10} = 0.7869$. Curriculum 3-stage chỉ vượt rõ khi learning rate ở giai đoạn 2 (thích nghi miền) được giảm xuống 5×10^{-6} , đạt $\text{MRR@10} = 0.7953$; nếu giữ learning rate cao 2×10^{-5} , 3-stage không cải thiện đáng kể so với trộn (0.7896).

Nguyên nhân là learning rate quá lớn ở giai đoạn thích nghi miền ghi đè một phần biểu diễn tiếng Việt đã học từ mMARCO-VI ở giai đoạn trước.

Table 4.6: Ảnh hưởng của cách tổ chức curriculum và learning rate

Cấu hình	Hit@1	Recall@5	MRR@10	NDCG@10
2-stage, trộn mMARCO + domain	0.6939	0.9213	0.7869	0.8201
3-stage, lr giai đoạn 2 = 2×10^{-5}	0.6910	0.9125	0.7896	0.8243
3-stage, lr giai đoạn 2 = 5×10^{-6}	0.7055	0.9184	0.7953	0.8279

4.2.3 Cắt tỉa từ vựng cho mmarco

Table 4.7: Nén Mmarco bằng cắt tỉa từ vựng

Thành phần	Trước cắt tỉa	Sau cắt tỉa	Giảm
Từ vựng	250.002 token	36,324 tokens	85.5%
Total params	117.6M	35.6M	69.7%
Transformer encoder	Giữ nguyên	Giữ nguyên	—

Pruning chỉ cắt embedding matrix và giữ nguyên transformer encoder. Tokens được giữ gồm special tokens, toàn bộ tokens xuất hiện trong corpus và queries, cùng 30,000 tokens phổ biến nhất để an toàn cho tài liệu mới. Trên corpus mới, UNK rate chỉ 0.225%, cho thấy cắt tỉa không làm mất token tiếng Việt quan trọng.

Table 4.8: Chất lượng pruned reranker sau tinh chỉnh

Model	Params	Hit@1	Recall@5	MRR@10	NDCG@10
BGE-reranker-v2-m3 teacher	568M	0.8192	0.9563	0.8804	0.9007
mmarco 117M đã tinh chỉnh	117M	0.8163	0.9621	0.8753	0.8979
Pruned 35.6M đã tinh chỉnh	35.6M	0.8134	0.9504	0.8718	0.8961

Pruned 35.6M giữ 99.6% MRR@10 của mmarco 117M (0.8718/0.8753) nhưng giảm memory dung lượng 3.3 lần. Độ trễ không giảm tương ứng vì transformer layers vẫn giữ nguyên; lợi ích chính của cắt tỉa là giảm tham số, dung lượng lưu trữ và yêu cầu bộ nhớ, không phải tốc độ.

4.2.4 Phân tích sự phụ thuộc của reranker vào độ mạnh của bộ truy hồi

Table 4.9: So sánh các reranker trên bộ truy hồi mạnh

Reranker	Params	Hit@1	Recall@5	MRR@10	NDCG@10	Delta MRR	Total ms
Không reranker	—	0.7172	0.9271	0.8023	0.8398	baseline	6.8
H384 base	117M	0.7522	0.9359	0.8352	0.8670	+0.0330	23.8
Pruned base	35.6M	0.7580	0.9329	0.8389	0.8690	+0.0366	23.9
ViRanker SOTA VI	568M	0.7638	0.9534	0.8434	0.8703	+0.0411	138.9
Pruned 35.6M đã tinh chỉnh	35.6M	0.8134	0.9504	0.8718	0.8961	+0.0695	23.8
H384 117M đã tinh chỉnh	117M	0.8163	0.9621	0.8753	0.8979	+0.0731	23.7
BGE-reranker-v2-m3 teacher	568M	0.8192	0.9563	0.8804	0.9007	+0.0781	138.8

Với bộ truy hồi mạnh, MiniLM 33M làm giảm chất lượng, còn các reranker từ 35.6M trở lên vẫn cải thiện rõ rệt. H384 117M đã tinh chỉnh đạt 99.4% MRR của BGE-reranker-v2-m3 trên pipeline này (0.8753/0.8804) nhưng tổng độ trễ chỉ 23.7ms, nhanh hơn BGE-reranker-v2-m3 khoảng 5.9 lần. Pruned 35.6M vượt ViRanker 568M cả về MRR (0.8718 so 0.8434) lẫn độ trễ (23.8ms so 138.9ms, nhanh khoảng 5.8 lần), xác nhận lợi ích của distillation trên dữ liệu đúng miền. Đáng chú ý, độ trễ của pruned 35.6M xấp xỉ H384 117M dù giảm 69.7% tham số: vì cắt tia chỉ cắt embedding matrix mà giữ nguyên encoder, lợi ích là tiết kiệm bộ nhớ chứ không phải tốc độ.

4.2.5 Tương tác giữa reranker và bộ truy hồi

Để hiểu rõ khi nào reranker có ích, đồ án đánh giá ba reranker trên hai bộ truy hồi khác nhau về độ mạnh: bộ truy hồi yếu (vietnamese-bi-encoder, pre-rerank $\text{MRR@10} = 0.6190$) và bộ truy hồi mạnh (đã tinh chỉnh embedding GIST→MNRL 512d, pre-rerank $\text{MRR@10} = 0.8023$). Bảng 4.10 cho thấy tác động của reranker phụ thuộc mạnh vào chất lượng candidate pool đầu vào.

Table 4.10: Reranker trên bộ truy hồi yếu so với bộ truy hồi mạnh (MRR@10)

Reranker	Yếu (post)	Δ yếu	Mạnh (post)	Δ mạnh
MiniLM-L12 33M	0.7408	+0.1218	0.7953	−0.0070
Pruned 35.6M đã tinh chỉnh	0.7993	+0.1803	0.8718	+0.0695
H384 117M đã tinh chỉnh	0.8036	+0.1846	0.8753	+0.0730

Hai quan sát chính. Thứ nhất, MiniLM 33M cải thiện mạnh bộ truy hồi yếu, tăng 0.1218 MRR@10 , nhưng lại làm giảm nhẹ bộ truy hồi mạnh, giảm 0.0070 MRR@10 . Khi pool đầu vào đã tốt, một reranker 33M không đủ năng lực để sắp xếp lại tốt hơn embedding đã tinh chỉnh. Thứ hai, các reranker từ 35.6M trở lên cải thiện cả hai bộ truy hồi, với mức tăng trên bộ truy hồi yếu lớn hơn nhiều: khoảng 0.18 so với 0.07 MRR@10 . Đáng chú ý, reranker đủ mạnh có khả năng “san bằng”

chất lượng bộ truy hồi: pool yếu sau khi rerank bằng pruned đạt 0.7993, gần bằng điểm xuất phát của bộ truy hồi mạnh (0.8023). Điều này hàm ý rằng tăng top-K khi retrieve chỉ cải thiện recall (gold lọt vào pool) mà không thay đổi thứ hạng đầu, trong khi reranker mới là thành phần kéo gold lên các vị trí top — hai công cụ giải quyết hai vấn đề khác nhau.

4.2.6 So sánh các phương pháp truy hồi

Bảng 4.11 so sánh các phương pháp ở tầng truy hồi trước khi rerank, cùng trên 343 queries và corpus 813 đoạn. BM25 được cài đặt với word segmentation để công bằng cho tiếng Việt.

Table 4.11: So sánh các phương pháp truy hồi (trước rerank) và đóng góp của reranker

Phương pháp	Hit@1	Recall@5	MRR@10	NDCG@10
BM25 (segmented)	0.6939	0.9009	0.7836	0.8155
Dense (ft embedding 512d)	0.7172	0.9271	0.8023	0.8398
Hybrid (BM25 + Dense, RRF)	0.7347	0.9388	0.8218	0.8558
Hybrid + Pruned reranker	0.8134	0.9504	0.8718	0.8961

Thứ tự chất lượng ở tầng truy hồi là $\text{BM25} < \text{Dense} < \text{Hybrid}$: hybrid RRF cao hơn dense +0.0195 MRR@10 nhờ kết hợp tín hiệu lexical và semantic. Tuy nhiên, ngay cả bộ truy hồi tốt nhất (hybrid, 0.8218) vẫn thấp hơn nhiều so với khi thêm reranker (0.8718, +0.0500). Kết quả này khẳng định giá trị của kiến trúc hai tầng: fusion ở tầng truy hồi và xếp hạng lại ở tầng sau bổ sung cho nhau, không thay thế nhau.

4.2.7 Các hướng đã thử nhưng không cải thiện

Đồ án ghi lại bốn hướng đã thử nghiệm nhưng không vượt được cấu hình chính (pruned đã tinh chỉnh, MRR@10 = 0.8718), vì mỗi kết quả âm đều cung cấp một góc nhìn về thiết kế pipeline.

Hợp nhất điểm (RRF) ở tầng reranker. Kết hợp thứ hạng của embedding và reranker bằng RRF đạt MRR@10 = 0.8511, thấp hơn reranker đơn (0.8718). Vì reranker đã vượt trội embedding ở tầng này, fusion tĩnh kéo kết quả về phía nguồn yếu hơn. Điều này khác với fusion ở tầng truy hồi (BM25 + dense), nơi hai nguồn cân bằng và bổ sung nhau — fusion chỉ có lợi khi các nguồn có chất lượng tương đương.

Lấy mẫu phân tầng cho candidate chưng cất. Thay vì lấy top-20 candidate liên nhau theo điểm BGE-reranker-v2-m3, thử một chiến lược phân tầng: luôn giữ 6 candidate điểm cao nhất (để đảm bảo chứa positive), sau đó chia 14 candidate còn

lại của pool thành bốn tầng theo điểm BGE-reranker-v2-m3 và lấy đều mỗi tầng (lấy mẫu cách đều trong từng tầng) cho đủ 20. Mục tiêu là tăng đa dạng độ khó của negative đưa vào KD. Kết quả $MRR@10 = 0.8672$, thấp hơn top-20 (0.8718), cho thấy top-20 tự nhiên đã đủ đa dạng (gold, hard và easy negatives) mà việc phân tầng thủ công lại loại bỏ một số hard negative gần top có giá trị.

Tăng cường dữ liệu bằng diễn giải lại câu hỏi. Bổ sung 786 câu hỏi paraphrase (do GPT sinh) vào dữ liệu giai đoạn B đạt $MRR@10 = 0.8697$, thấp hơn dữ liệu gốc (0.8718). Phân tích phân bố gap cho thấy pool paraphrase có 90.7% easy negative (gap > 0.7) so với 66.2% của dữ liệu gốc: câu paraphrase rõ nghĩa hơn nên retrieve “sạch” hơn, thiếu hard negative có giá trị — tương tự hiện tượng quá liều mMARCO ở giai đoạn A.

Mở rộng từ vựng cho MiniLM tiếng Anh. Thử thêm token tiếng Việt vào tokenizer English của ms-marco-MiniLM nhằm cải thiện biểu diễn tiếng Việt. Phân tích token cho thấy vấn đề không nằm ở thiếu token (UNK chỉ 0.14%) mà ở việc mất dấu thanh: WordPiece English tách “Việt” thành [viet] (mất dấu), “Công nghệ” thành ba mảnh [cong, ng, ##he]. Do encoder pretrain trên tiếng Anh không có biểu diễn tiếng Việt có dấu, việc mở rộng từ vựng chỉ giảm phân mảnh chứ không sửa được encoder. Hướng cắt tia (giữ encoder đa ngôn ngữ) vì vậy hiệu quả hơn hẳn (0.8718 so với 0.7572 của MiniLM English).

Cả bốn hướng đều củng cố các lựa chọn thiết kế cuối: reranker đơn (không RRF tầng cuối), candidate top-20 (không stratified), dữ liệu domain sạch (không paraphrase augmentation), và cắt tia encoder đa ngôn ngữ (không từ vựng expansion English).

4.2.8 Đánh giá đầu cuối trên câu hỏi trắc nghiệm

Để đánh giá tác động đầu cuối của toàn bộ pipeline v2, đề án chạy lại bài toán MCQ trên 390 câu hỏi, so sánh trực tiếp pipeline v1 (baseline) với pipeline v2. Pipeline v1 dùng embedding 1024 chiều, GPT router và reranker BGE-reranker-v2-m3; pipeline v2 dùng embedding GIST→MNRL 512 chiều, router cục bộ dựa trên Sentence-BERT/SetFit (mục 4.4) và reranker pruned 35.6M. Phép đo thực hiện trong môi trường CPU/API trên máy cá nhân (mục 4.1.1), nên độ trễ phản ánh thiết lập triển khai thực tế chứ không so trực tiếp với độ trễ xếp hạng đo trên GPU ở các bảng trước.

Table 4.12: Đánh giá đầu cuối trên 390 câu trắc nghiệm: pipeline v1 so với v2

Pipeline	Accuracy	Độ trễ/câu	Intent acc.
v1 baseline (1024d, GPT router, BGE-reranker-v2-m3)	97.44% (380/390)	$\approx 25,898\text{ms}$	94.62%
v2 (512d, Sentence-BERT/SetFit router, pruned 35.6M)	97.18% (379/390)	$\approx 2,630\text{ms}$	97.69%

Về độ chính xác, hai pipeline gần như tương đương: v2 chỉ kém v1 đúng một câu (379 so với 380 trên 390), trong khi tổng thời gian mỗi câu giảm khoảng 9.85 lần và loại bỏ chi phí gọi API cho cả định tuyến lẫn rerank. Đáng chú ý, router cục bộ dựa trên Sentence-BERT/SetFit của v2 phân loại ý định tốt hơn GPT router của v1 (97.69% so với 94.62%, chi tiết ở mục 4.4).

Table 4.13: Độ trễ trung bình theo thành phần (ms/câu)

Module	v1 baseline	v2	Speedup
Routing	743.0	49.1	$15.1\times$
Truy hỏi	21.9	6.3	$3.5\times$
Rerank	23,166.6	1,151.5	$20.1\times$
Sinh câu trả lời (LLM)	1,966.2	1,423.5	$1.4\times$
Tổng	25,897.8	2,630.4	$9.85\times$

Rerank đóng góp phần tăng tốc lớn nhất, khoảng 20.1 lần khi chuyển từ BGE-reranker-v2-m3 sang pruned 35.6M; tiếp theo là định tuyến, khoảng 15.1 lần khi chuyển từ GPT API sang router cục bộ dựa trên Sentence-BERT/SetFit. Bước sinh câu trả lời bằng LLM gần như không đổi nên trở thành thành phần độ trễ chi phối còn lại của v2. Về truy hỏi, v2 có recall hơi thấp hơn v1 (gold-hit 96.15% so với 96.92%, nhiều hơn ba câu miss), một đánh đổi nhỏ khi dùng embedding 512 chiều nhưng không làm thay đổi đáng kể accuracy cuối cùng.

Table 4.14: Dung lượng lưu trữ của embedding cache và index trong pipeline đầu cuối

Thành phần	v1 baseline 1024d	v2 512d	Tỉ lệ v1/v2
Model embedding	2,187.00 MiB	2,181.91 MiB	$1.00\times$
Chunk embedding cache	3.18 MiB	1.59 MiB	$2.00\times$
Query embedding cache	1.58 MiB	0.82 MiB	$1.93\times$
Chroma DB	10.68 MiB	9.45 MiB	$1.13\times$
Cache + Chroma	15.44 MiB	11.85 MiB	$1.30\times$
Tổng model + runtime storage	2,202.44 MiB	2,193.77 MiB	$1.004\times$

Bảng 4.14 cho thấy lợi ích bộ nhớ chính của v2 nằm ở phần vector runtime: đoạn cache giảm từ ma trận (813, 1024) xuống (813, 512) float32, còn query cache giảm từ 390 vector 1024 chiều xuống 390 vector 512 chiều. Vì vậy phần raw embedding cache giảm gần đúng 2 lần. Chroma DB chỉ giảm 1.13 lần do ngoài vector còn chứa metadata và cấu trúc chỉ mục. Nếu tính cả thư mục model embedding, tổng dung lượng chỉ giảm nhẹ vì hai encoder đều có kích thước khoảng 2.18GB; tuy nhiên trong triển khai với corpus lớn hơn, phần cache/index sẽ tăng tuyến tính theo số đoạn và lợi thế 512d sẽ rõ hơn.

Nhìn chung, pipeline v2 giữ được độ chính xác của v1 nhưng nhanh hơn gần 10 lần, giảm lưu trữ runtime cho vector/index và chạy hoàn toàn cục bộ, đúng với mục tiêu hiệu quả của đồ án.

4.2.9 Phân rã nguồn lỗi đầu cuối

Bảng 4.12 cho thấy v2 kém v1 đúng một câu về độ chính xác (379 so với 380 trên 390). Để xác định khoản chênh này đến từ thành phần nào trong pipeline, đồ án phân rã từng câu trả lời sai theo hai trục độc lập: (1) bộ phân loại ý định định tuyến *đúng* hay *sai*, và (2) đáp án cuối *đúng* hay *sai*. Kết quả tạo thành ma trận 2×2 ở Bảng 4.15.

Table 4.15: Ma trận định tuyến $\times$ đáp án cuối trên 390 câu trắc nghiệm

Trường hợp	v1 (GPT router)		v2 (Sentence-BERT/SetFit router)	
	Đáp án đúng	Đáp án sai	Đáp án đúng	Đáp án sai
Định tuyến đúng	361	8	371	10
Định tuyến sai	19	2	8	1

Ồ đáng chú ý nhất là *định tuyến sai và đáp án sai* — tức lỗi định tuyến thực sự làm hỏng đáp án cuối: v2 chỉ có 1 câu, v1 có 2 câu. Như vậy, mặc dù v2 có chín câu định tuyến sai, gần như toàn bộ là vô hại với độ chính xác: sáu câu tra_cuu bị định tuyến nhầm sang tinh_toan chỉ phát sinh thêm một lượt suy luận (lỗi chi phí/độ trễ, không sai đáp án), còn trong ba câu tinh_toan bị nhầm sang tra_cuu, chỉ duy nhất một câu thực sự mất bước suy luận từng bước và bị lật đáp án (ví dụ câu hỏi “*kích thước large_string gấp bao nhiêu lần kích thước buffer*” — một phép chia bị nhận nhầm là tra cứu do cách phát biểu đơn giản, đúng dạng lỗi biên đã mô tả ở mục 4.4 liên quan đến câu tính toán phát biểu giống tra cứu). Hệ quả là việc thay GPT router bằng router cục bộ dựa trên Sentence-BERT/SetFit *không* làm giảm độ chính xác, thậm chí còn giảm một câu lỗi do định tuyến.

Như vậy, khoản chênh một câu của v2 không đến từ việc thay router mà phản ánh đánh đổi recall ở tầng truy hồi khi giảm chiều biểu diễn xuống 512 — mục 4.2.8

đã ghi nhận v2 có tỉ lệ gold-hit thấp hơn v1 một chút — một lựa chọn có chủ đích để đổi lấy dung lượng vector index và chi phí tìm kiếm tương đồng nhỏ hơn hai lần. Các hướng cải thiện tăng truy hồi được thảo luận ở mục 4.6.

4.3 Đánh giá embedding

4.3.1 Thiết lập đánh giá

Embedding được đánh giá trên hai bộ test. D1 (in-domain) gồm 157 queries trên corpus 2,317 đoạn của các tài liệu thuộc miền huấn luyện, với câu hỏi tách riêng và không dùng khi train. D2 (độc lập) gồm 343 queries trên 50 tài liệu độc lập (corpus 813 đoạn). D1 đánh giá truy hồi in-domain; D2 đánh giá zero-shot trên tài liệu chưa từng thấy và đồng thời là tập dùng chung cho đánh giá reranker.

Table 4.16: Các tập dữ liệu đánh giá embedding

Tập test	Queries	Corpus	Nguồn tài liệu	Cách sinh câu hỏi
D1 (in-domain)	157	2,317 đoạn	Miền huấn luyện (200 docs)	Tách từ tập câu hỏi, không dùng khi train
D2 (độc lập)	343	813 đoạn	Độc lập (50 docs)	Tập test chung với reranker

4.3.2 Kết quả trên hai tập kiểm thử

Table 4.17: Kết quả trên tập D1

Model	Acc@1	Acc@5	MRR@10	NDCG@10
BKAI bi-encoder	58.60%	80.89%	0.6877	0.6451
Vietnamese Embedding v1	71.34%	94.27%	0.8127	0.8037
Vietnamese Embedding v2 baseline	73.89%	93.63%	0.8259	0.8032
BGE-M3 raw	74.52%	94.90%	0.8384	0.8409
Tinh chỉnh 512d (GIST→MNRL)	76.43%	96.18%	0.8454	0.8392

Table 4.18: Kết quả trên tập D2

Model	Acc@1	Acc@5	MRR@10	NDCG@10
BKAI bi-encoder	50.73%	77.55%	0.6190	0.6627
Vietnamese Embedding v1	69.10%	90.09%	0.7830	0.8258
Vietnamese Embedding v2 baseline	70.55%	92.13%	0.7976	0.8334
BGE-M3 raw	68.80%	91.25%	0.7806	0.8172
Tinh chỉnh 512d (GIST→MNRL)	72.30%	92.42%	0.8042	0.8413

Tinh chỉnh 512d (GIST→MNRL) thắng Acc@1 rõ rệt trên cả D1 và D2. Trên D1 (in-domain), Acc@1 tăng từ 73.89% lên 76.43% và MRR@10 cao nhất (0.8454). Trên D2 (độc lập), Acc@1 tăng từ 70.55% lên 72.30%, đồng thời MRR@10 = 0.8042 và NDCG@10 = 0.8413 đều cao nhất — GIST ở giai đoạn 1 giúp khả năng

tổng quát hóa nhờ lọc false negatives khi học. Như vậy tình hình không chỉ cải thiện trên dữ liệu quen thuộc mà còn giữ lợi ích trên tài liệu chưa từng thấy.

Lưu ý về cách đo: $\text{MRR}@10$ của embedding trên D2 (0.8042) là kết quả xếp hạng toàn corpus bằng evaluator của embedding; trong phân tích pipeline reranker (mục 4.2.4), baseline khi chưa rerank được đo trên pool top-20 nên đạt 0.8023 — chênh nhỏ do khác phạm vi đánh giá, cùng một bộ truy hồi.

4.3.3 Phân tích số chiều biểu diễn MRL

Table 4.19: Phân tích số chiều biểu diễn MRL trên D1 và D2

Test	Dim	Acc@1	MRR@10	NDCG@10
D1	256	72.61%	0.8094	0.8067
D1	512	76.43%	0.8454	0.8392
D1	1024	75.80%	0.8519	0.8418
D2	256	69.68%	0.7784	0.8140
D2	512	72.30%	0.8042	0.8413
D2	1024	71.14%	0.8068	0.8428

Kích thước tăng theo đúng quy luật MRL ($256 < 512 \leq 1024$). 512-dim thắng Acc@1 ở cả D1 và D2 (76.43%/72.30% so với 75.80%/71.14% của 1024-dim); 1024-dim nhỉnh MRR@10 và NDCG@10 sát nút trên D2 (chênh dưới 0.003). Hai kích thước gần như tương đương về chất lượng tổng thể, nên 512-dim được chọn làm cấu hình chính nhờ giảm storage và chi phí similarity search 2 lần. 256-dim kém rõ rệt (giảm trên 3 điểm phần trăm Acc@1) nên chỉ dùng khi cần tối đa tốc độ hoặc bộ nhớ.

4.3.4 Nghiên cứu loại bỏ curriculum và GIST

Pipeline embedding cuối dùng curriculum hai stage với loss khác nhau theo stage. Giai đoạn 1 dùng GISTEmbedLoss học từ synthesized negatives — dữ liệu dễ lẫn false negative nên cần guide model BGE-M3 lọc bớt. Giai đoạn 2 chuyển sang MultipleNegativesRankingLoss (MNRL) với mined hard negatives — dữ liệu đã sạch (mine theo gap điểm BGE-reranker-v2-m3, khác tài liệu). Lý do không dùng GIST ở giai đoạn 2: GIST có xu hướng lọc nhầm chính các mined hard negative này vì chúng “trông giống” positive, làm mất tín hiệu phân biệt tinh vi; MNRL giữ lại toàn bộ hard negative nên học phân biệt tốt hơn.

Table 4.20: Cấu hình huấn luyện embedding cuối cùng

Giai đoạn	Loss	Nguồn negative	Epochs	Batch	LR
giai đoạn 1	GIST (guide BGE-M3, temp 0.01)	Synthesized, signal rõ	2	64	2×10^{-6}
giai đoạn 2	MNRL (scale 20)	Mined hard negatives	1	32	5×10^{-7}

GISTEmbedLoss ở giai đoạn 1 dùng BGE-M3 làm guide model để lọc false negatives trong in-batch negatives. Điều này quan trọng vì corpus kỹ thuật có nhiều đoạn cùng chủ đề; positive của query này có thể liên quan tới query khác nhưng MNRL thông thường vẫn xem nó là negative. GIST giảm nhiễu này và góp phần giúp đã tinh chỉnh embedding generalize tốt trên D2 độc lập, thể hiện qua việc GIST→MNRL đạt $MRR@10$ và $NDCG@10$ cao nhất trên D2 trong ablation curriculum loss.

Bảng 4.21 kiểm chứng lựa chọn loss bằng cách so sánh ba cấu hình (cùng base model, cùng dữ liệu, cùng hyperparameters, chỉ khác loss mỗi stage, đều ở 512d).

Table 4.21: Ablation curriculum loss: GIST vs MNRL theo stage (512d)

Cấu hình	D1 Acc@1	D1 MRR	D2 Acc@1	D2 MRR
GIST → GIST	73.89%	0.8308	71.14%	0.7999
MNRL → MNRL	75.80%	0.8441	71.14%	0.7975
GIST → MNRL	76.43%	0.8454	72.30%	0.8042

GIST→MNRL thắng $MRR@10$ trên cả D1 và D2. GIST→GIST kém hơn vì dùng GIST ở giai đoạn 2 có thể lọc nhầm các mined hard negative; MNRL→MNRL thiếu lọc false negative ở giai đoạn 1. GIST→MNRL kết hợp ưu điểm của cả hai: lọc nhiễu khi học rộng (giai đoạn 1) và giữ hard negative khi tinh chỉnh (giai đoạn 2). Chênh tuyệt đối nhỏ (khoảng 1–3 câu mỗi bộ), nhưng tính nhất quán trên cả in-domain và zero-shot D2 cùng cơ chế giải thích về vai trò hard negative cho thấy đây là xu hướng thật, không phải dao động ngẫu nhiên.

4.4 Đánh giá bộ phân loại ý định

Bộ phân loại ý định v2 (mô tả ở mục 3.9) tinh chỉnh một Sentence-BERT tiếng Việt (keepitreal/vietnamese-sbert, dựa trên PhoBERT-base) theo phương pháp SetFit — contrastive learning trên encoder rồi đặt một head Logistic Regression trên embedding. Dữ liệu gồm 983 câu sạch với tỉ lệ lớp xấp xỉ 12:1, nên metric chính là F1-macro và việc đánh giá dùng stratified 5-fold cross-validation; một tập test tách riêng 80/20 được dùng bổ sung để phân tích cơ cấu lỗi theo từng lớp.

Bảng 4.22 so sánh ba cấu hình theo F1-macro 5-fold. Tinh chỉnh encoder bằng contrastive learning nâng F1-macro từ 0.859 (embedding đông cứng, chỉ train head) lên 0.947, cải thiện khoảng 0.09 điểm. Hai backbone Sentence-BERT tiếng Việt dựa trên PhoBERT-base cho kết quả tương đương; BKAI Vietnamese bi-encoder có mean ngang nhưng độ lệch chuẩn nhỏ hơn (ổn định hơn).

Table 4.22: So sánh cấu hình ý định classifier theo F1-macro

Cấu hình	F1-macro (5-fold CV)
Embedding đông cứng + Logistic Regression	0.859
SetFit tinh chỉnh (keepitreal/vietnamese-sbert)	0.947 ± 0.028
SetFit tinh chỉnh (bkai-foundation-models/vietnamese-bi-encoder)	0.950 ± 0.017

Độ lệch chuẩn (± 0.02 – 0.03) chủ yếu đến từ việc lớp `trinh_toan` chỉ có 76 mẫu, bị chia mỏng qua các fold (khoảng 15 mẫu mỗi tập test); mỗi câu phân loại sai làm F1-macro của fold đó thay đổi vài phần trăm. Vì vậy con số 5-fold (0.947) đáng tin hơn kết quả single-split (lên tới 0.981), vốn là đầu lạc quan.

Cơ cấu cải thiện thể hiện rõ khi phân tích theo lớp trên tập test tách riêng. Ở cấu hình đông cứng, lớp `trinh_toan` có precision thấp (0.60–0.64): model nhầm nhiều câu `tra_cuu` có chứa số liệu thành `trinh_toan` (ví dụ “hàng số điện môi có giá trị bao nhiêu?” — thực chất là tra một giá trị có sẵn). Sau tinh chỉnh contrastive, precision lớp `trinh_toan` đạt 1.00 và không còn false positive nào trên tập test — ranh giới *có-sẵn so với phải-tính* đã được học đúng. Bảng 4.23 trình bày chất lượng nội tại của classifier v2; Bảng 4.24 mới so sánh router v1 và v2 trên cùng tập 390 câu MCQ đầu cuối, tránh trộn kết quả test tách riêng/CV với kết quả pipeline.

Table 4.23: Đánh giá nội tại tại bộ phân loại ý định v2 (SetFit)

Metric	Kết quả	Protocol
Accuracy	99.49%	Test tách riêng 80/20 trên tập ý định sạch
F1-macro	0.947 ± 0.028	Stratified 5-fold CV trên 983 câu
F1 lớp <code>tra_cuu</code>	0.997	Test tách riêng
F1 lớp <code>trinh_toan</code>	0.966	Test tách riêng
Độ trễ/câu	$\approx 11\text{ms}$ đơn, < 1ms batch	Đo suy luận local trên GPU T4
Chi phí/1M queries	\$0	Chạy local, không gọi API

Table 4.24: So sánh bộ định tuyến v1 (GPT) và v2 (Sentence-BERT/SetFit)

Metric	GPT-4o-mini router (v1)	Sentence-BERT/SetFit router (v2)
Độ chính xác ý định	94.62% (369/390)	97.69% (381/390)
Số câu sai ý định	21	9
Gold tra_cuu → routed tinh_toan	21	6
Gold tinh_toan → routed tra_cuu	0	3
Phân bố routed ý định	322 tra_cuu, 68 tinh_toan	340 tra_cuu, 50 tinh_toan
Độ trễ định tuyến trung bình	743.0ms/câu (API)	49.1ms/câu (local)
Chi phí/1M queries	Có chi phí API, phụ thuộc biểu giá và token đầu vào	\$0

Đồ án cũng khảo sát việc bổ sung dữ liệu bằng paraphrase do LLM sinh (786 câu) nhưng loại bỏ hướng này. Kiểm tra cho thấy paraphrase bị trôi nhãn: LLM thường bỏ số liệu cụ thể và biến câu yêu cầu tính toán thành câu hỏi công thức hoặc khái niệm (mang sắc thái tra_cuu), khiến 17/49 câu tinh_toan bị đổi ý nghĩa trong khi vẫn giữ nhãn cũ. Đưa dữ liệu này vào huấn luyện sẽ tiêm nhãn sai vào đúng lớp thiếu số đang yếu. Đây là biểu hiện của distribution gap khi dùng synthetic data, và là lý do model cuối chỉ dùng dữ liệu gán nhãn tay.

So với mục tiêu thiết kế (accuracy $\geq 97\%$, độ trễ thấp), F1-macro 0.947 và accuracy trên test tách riêng 99.49% đáp ứng ngưỡng chất lượng nội tại. Khi đưa vào pipeline MCQ 390 câu, router cục bộ dựa trên Sentence-BERT/SetFit đạt ý định accuracy 97.69%, cao hơn GPT router v1 94.62%. Về độ trễ, classifier local đạt khoảng 11ms/câu khi xử lý đơn lẻ trên GPU T4 và dưới 1ms/câu khi batch; trong phép đo đầu cuối trên máy cá nhân, định tuyến v2 trung bình 49.1ms/câu, nhanh hơn GPT router v1 743.0ms/câu khoảng 15 lần, đồng thời loại bỏ chi phí API và cho phép chạy offline.

4.5 Khuyến nghị cấu hình triển khai

Từ kết quả embedding và reranker, cấu hình được khuyến nghị cho pipeline v2 là đã tinh chỉnh embedding 512d để truy xuất top-20, sau đó dùng mmrco-mmMiniLMv2 117M đã tinh chỉnh để rerank. Cấu hình này đạt gần chất lượng teacher BGE-reranker-v2-m3 nhưng có độ trễ thấp hơn nhiều trong phép đo xếp hạng pipeline.

Table 4.25: Các cấu hình pipeline khuyến nghị

Ưu tiên	Cấu hình	MRR@10	Độ trễ	Nhận xét
Tốc độ tối đa	Embedding tinh chỉnh 512d, không reranker	0.8023	6.8ms	Phù hợp khi tài nguyên rất hạn chế
Cân bằng khuyến nghị	Embedding tinh chỉnh 512d + H384/mmario 117M đã tinh chỉnh	0.8753	23.7ms	Đạt 99.4% MRR của BGE-reranker-v2-m3 với độ trễ thấp hơn khoảng 5.9 lần
Bộ nhớ thấp	Embedding tinh chỉnh 512d + pruned mmario 35.6M	0.8718	23.8ms	Giảm tham số 69.7%, chất lượng gần mmario 117M
Chất lượng tối đa	Embedding tinh chỉnh 512d + BGE-reranker-v2-m3 teacher	0.8804	138.8ms	Chất lượng cao nhất nhưng độ trễ lớn

Kết quả này cho thấy lựa chọn tối ưu không phải luôn là model nhỏ nhất. Khi bộ truy hồi đã mạnh, reranker 33M không đủ capacity để khai thác candidate list tốt hơn, trong khi mmario 117M và pruned 35.6M vẫn cải thiện đáng kể. Vì vậy, pipeline production nên dùng 512d embedding làm bộ truy hồi chính và chọn giữa mmario 117M hoặc pruned 35.6M tùy ràng buộc bộ nhớ.

4.6 Phân tích lỗi tổng thể

Tổng hợp phân tích từ các thành phần cho thấy ba nguồn gốc lỗi chính theo thứ tự tác động giảm dần.

Trên reranker production pruned 35.6M, phân tích 64 query miss (Hit@1 = 81.3%, tức 279/343) cho thấy cơ cấu lỗi như sau: 67.2% là near miss (gold bị xếp ở rank 1–4 thay vì rank 1), 25.0% là far miss và chỉ 7.8% là truy hồi miss (gold không có trong pool top-20). Phần lớn nhầm lẫn đến từ cùng tài liệu (same-doc 72.9%) thay vì khác tài liệu (cross-doc 27.1%). Theo độ khó, Hit@1 đạt 83.5% ở nhóm easy, 81.0% ở medium và 75.0% ở hard. Như vậy, đa số lỗi là sai lệch thứ hạng sát ngưỡng và nhầm giữa các đoạn gần nhau trong cùng tài liệu, không phải mất gold khỏi pool.

Nguồn 1 – Giới hạn trần của truy hồi.

Một phần lỗi xuất hiện trước khi reranker nhìn thấy dữ liệu: gold đoạn không nằm trong top-20 candidates hoặc chỉ xuất hiện ở vị trí thấp. Embedding tinh chỉnh 512d cải thiện Acc@1 trên D1 (in-domain) từ 73.89% lên 76.43% và trên D2 độc lập từ 70.55% lên 72.30%, giúp giảm nhóm lỗi truy hồi nhưng không loại bỏ hoàn toàn. Các trường hợp khó nhất gồm acronym match, bảng số liệu liền kề và đoạn có nội dung gần giống nhau.

Nguồn 2 – Sai lệch nhỏ trong xếp hạng.

Nhiều lỗi không phải sai hoàn toàn mà là gold nằm gần ngưỡng chọn ngữ cảnh,

ví dụ rank 6–10. Tăng top-K từ 5 lên 6 hoặc 7 có thể khôi phục thêm một phần các query này, nhưng phải đánh đổi với ngữ cảnh window lớn hơn và chi phí generation cao hơn. Đây là trade-off cần được quyết định theo yêu cầu triển khai thực tế.

Nguồn 3 – Nhầm lẫn đặc thù miền.

Cross-domain noise và same-domain confusion phản ánh giới hạn của embedding và BM25 với tài liệu kỹ thuật đặc thù. Các hướng cải thiện tiếp theo gồm bổ sung dữ liệu domain-specific cho embedding tinh chỉnh, áp dụng query expansion để tăng recall trước khi rerank, hoặc cải thiện chunking strategy cho bảng biểu và đoạn định nghĩa ngắn.

Nhìn tổng thể, pipeline v2 đạt mục tiêu chính: đã tinh chỉnh embedding 512d (curriculum GIST→MNRL) đạt $\text{MRR}@10 = 0.8042$ trên D2 độc lập, mmarco 117M đã tinh chỉnh nâng pipeline lên $\text{MRR}@10 = 0.8753$ với độ trễ 23.7ms, còn pruned 35.6M đạt $\text{MRR}@10 = 0.8718$ với dung lượng nhỏ hơn. Các giới hạn còn lại tập trung ở truy hồi ceiling, dữ liệu domain-specific và chunking strategy.

CHƯƠNG 5. KẾT LUẬN

5.1 Tổng kết

Đề án giải quyết bài toán hỏi đáp trên kho tài liệu kỹ thuật tiếng Việt bằng kiến trúc Retrieval-Augmented Generation, với trọng tâm là tối ưu hai thành phần quyết định chất lượng truy hồi: mô hình embedding và reranker. Xuất phát từ một hệ thống nền tảng (v1), đề án đo lường và xác định ba điểm nghẽn cụ thể về độ trễ, chi phí và chất lượng, sau đó đề xuất hệ thống v2 với nguyên tắc mỗi thay đổi kỹ thuật phải giải quyết một vấn đề đã được đo lường thay vì tối ưu dựa trên giả định.

Về embedding, đề án tinh chỉnh mô hình Vietnamese_Embedding_v2 (dựa trên BGE-M3) theo curriculum hai giai đoạn GIST→MNRL kết hợp Matryoshka Representation Learning. Mô hình sau tinh chỉnh đạt $\text{Acc}@1 = 76.43\%$ và $\text{MRR}@10 = 0.8454$ trên tập in-domain, đồng thời vượt cả BGE-M3 gốc trên tập độc lập (held-out) ($\text{MRR}@10 = 0.8042$), cho thấy việc thích nghi miền có hiệu quả nhất quán chứ không chỉ trên dữ liệu quen thuộc. Nhờ MRL, biểu diễn 512 chiều đạt chất lượng gần tương đương 1024 chiều, giúp giảm một nửa dung lượng vector index mà không hy sinh đáng kể độ chính xác.

Về reranker, đề án chưng cất teacher BGE-reranker-v2-m3 (568M tham số) thành các student nhẹ thông qua quy trình hai giai đoạn (contrastive learning rồi chưng cất tri thức) với hàm mất mát ADR-MSE theo hướng rank-based. Student mmarco 117M fine-tuned nâng pipeline lên $\text{MRR}@10 = 0.8753$, còn phiên bản pruned 35.6M tham số đạt $\text{MRR}@10 = 0.8718$ với dung lượng nhỏ hơn nhiều. Đáng chú ý, reranker pruned 35.6M vượt ViRanker 568M (lớn hơn khoảng 16 lần) trên cùng tập đánh giá, khẳng định chưng cất và cắt tia từ vừng đúng miền có thể giữ phần lớn năng lực của teacher với chi phí thấp.

Về routing, đề án thay GPT router bằng một bộ phân loại ý định cục bộ dựa trên Sentence-BERT, loại bỏ phụ thuộc vào API ngoài, giảm độ trễ và chi phí trên mỗi truy vấn đồng thời giữ dữ liệu trong hạ tầng nội bộ.

5.2 Đóng góp chính

Các đóng góp chính của đề án gồm:

- **Quy trình xây dựng dữ liệu huấn luyện dùng chung cho embedding và reranker.** Quy trình năm giai đoạn dùng mô hình ngôn ngữ làm giám khảo xác định positive, tổng hợp hard negative từ hai nguồn (tạo sinh có kiểm soát và khai thác từ kết quả truy hồi), lọc khả năng trả lời, rồi gộp và chia tập. Cùng một nguồn dữ liệu được tái sử dụng cho cả hai tầng với tiêu chí lọc riêng.

- **Fine-tune embedding theo curriculum kết hợp MRL.** Curriculum GIST→MNRL cho phép học ranh giới ngữ nghĩa từ negative tạo sinh trước, rồi cải thiện khả năng phân biệt bằng hard negative khai thác; MRL tạo biểu diễn đa kích thước, giúp embedding 512 chiều đạt chất lượng gần tương đương 1024 chiều với nửa dung lượng.
- **Nén reranker bằng chưng cất tri thức kết hợp cắt tỉa từ vựng.** Chưng cất teacher BGE-reranker-v2-m3 qua hai giai đoạn rồi cắt tỉa từ vựng, tạo ra reranker 35.6M tham số đạt chất lượng cao hơn các baseline tiếng Việt lớn hơn nhiều lần, phù hợp triển khai trên môi trường tài nguyên hạn chế.
- **Hệ thống RAG tối ưu đầu cuối.** Tích hợp embedding tinh chỉnh 512 chiều, reranker chưng cất và bộ phân loại ý định cục bộ thay GPT router, giảm độ trễ, dung lượng và chi phí suy luận so với hệ thống v1 trong khi vẫn giữ chất lượng truy hồi và xếp hạng.

5.3 Hạn chế

Bên cạnh các kết quả đạt được, đồ án còn một số hạn chế.

Thứ nhất, mức cải thiện của tinh chỉnh embedding là vừa phải (khoảng hai điểm phần trăm Acc@1) do base model BGE-M3 đã rất mạnh và lượng dữ liệu miền còn nhỏ. Bi-encoder cũng có trần kiến trúc khi phải nén toàn bộ ngữ nghĩa vào một vector đơn, khiến việc phân biệt các hard negative khác nhau ở mức chi tiết factual (số liệu, thực thể) trở nên khó khăn; phần này được bù đắp bởi reranker nhưng chưa được giải quyết triệt để ở tầng embedding.

Thứ hai, một phần dữ liệu đánh giá được sinh tự động qua mô hình ngôn ngữ. Mặc dù dữ liệu đã qua nhiều bước lọc, việc kiểm chứng thủ công toàn bộ câu hỏi vẫn cần thiết để loại bỏ hoàn toàn nguy cơ thiên lệch trong quá trình sinh dữ liệu.

Thứ ba, cắt tỉa từ vựng giảm dung lượng mô hình nhưng không giảm đáng kể độ trễ của Transformer encoder do số tầng và kích thước ẩn được giữ nguyên; lợi ích chủ yếu nằm ở dung lượng lưu trữ chứ không phải tốc độ suy luận.

Thứ tư, hệ thống được đánh giá trên một miền tài liệu kỹ thuật cụ thể. Khả năng tổng quát hóa sang các miền khác hoặc các loại tài liệu khác cần được kiểm chứng thêm.

5.4 Hướng phát triển

Từ các hạn chế trên, đồ án đề xuất ba hướng phát triển tiếp theo.

Thứ nhất, về dữ liệu và chất lượng truy hồi, có thể xây dựng một bộ kiểm thử được xác minh thủ công hoàn toàn để củng cố độ tin cậy của đánh giá, đồng thời

mở rộng dữ liệu tinh chỉnh embedding với nhiều hard negative đặc thù hơn nhằm nâng trền chất lượng ở tầng truy hồi.

Thứ hai, về mô hình, có thể khảo sát các kỹ thuật nén giảm được cả độ trễ (như cắt tia tầng hoặc lượng tử hóa) thay vì chỉ giảm dung lượng, cũng như thử nghiệm các kiến trúc reranker khác để tìm điểm cân bằng tốt hơn giữa chất lượng và tốc độ.

Thứ ba, về khả năng tổng quát hóa và triển khai thực tế, việc đánh giá hệ thống trên nhiều miền tài liệu khác nhau và thử nghiệm trong môi trường vận hành thực tế sẽ giúp kiểm chứng tính ứng dụng của các giải pháp đề xuất ngoài phạm vi miền tài liệu kỹ thuật hiện tại.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2005.11401, 2020.
- [2] Beijing Academy of Artificial Intelligence, *BAAI/bge-reranker-v2-m3*, <https://huggingface.co/BAAI/bge-reranker-v2-m3>, 2024.
- [3] OpenAI, *GPT-4o mini: Advancing cost-efficient intelligence*, <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024.
- [4] AITeamVN, *Vietnamese_embedding_v2*, https://huggingface.co/AITeamVN/Vietnamese_Embedding_v2, 2025.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/15000000019
- [6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759. DOI: 10.1145/1571941.1572114
- [7] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *NeurIPS Deep Learning Workshop*, arXiv:1503.02531, 2015.
- [8] S. Hofstätter, S. Althammer, M. Schröder, M. Sertkan, and A. Hanbury, “Improving efficient neural ranking models with cross-architecture knowledge distillation,” *arXiv preprint arXiv:2010.02666*, 2020.
- [9] C. Burges et al., “Learning to rank using gradient descent,” in *Proceedings of the 22nd International Conference on Machine Learning*, 2005, pp. 89–96. DOI: 10.1145/1102351.1102363
- [10] F. Schlatt et al., “Rank-distillm: Closing the effectiveness gap between cross-encoders and LLMs for passage re-ranking,” in *Advances in Information Retrieval (ECIR 2025)*, arXiv:2405.07920, 2025.
- [11] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:1706.03762, 2017.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, arXiv:1810.04805, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423

- [13] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, arXiv:2003.00744, 2020, pp. 1037–1042. DOI: 10.18653/v1/2020.findings-emnlp.92
- [14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, arXiv:1908.10084, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410
- [15] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [16] M. Henderson et al., “Efficient natural language response suggestion for smart reply,” *arXiv preprint arXiv:1705.00652*, 2017.
- [17] A. V. Solatorio, “GISTEmbed: Guided in-sample selection of training negatives for text embedding fine-tuning,” *arXiv preprint arXiv:2402.16829*, 2024.
- [18] A. Kusupati et al., “Matryoshka representation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2205.13147, 2022.
- [19] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th International Conference on Machine Learning*, 2009, pp. 41–48. DOI: 10.1145/1553374.1553380
- [20] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2201.11903, 2022.
- [21] L. Tunstall et al., “Efficient few-shot learning without prompts,” *arXiv preprint arXiv:2209.11055*, 2022.
- [22] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, pp. 422–446, 2002. DOI: 10.1145/582415.582418
- [23] Datalab, *Marker: Convert documents to markdown, json, chunks, and html*, <https://github.com/datalab-to/marker>, 2024.
- [24] T. Nguyen et al., “MS MARCO: A human generated MACHine reading COMprehension dataset,” in *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches 2016 Co-located with the 30th Annual Conference on Neural Information Processing Systems*, arXiv:1611.09268, 2016.

- [25] L. Bonifacio et al., “mMARCO: A multilingual version of the MS MARCO passage ranking dataset,” *arXiv preprint arXiv:2108.13897*, 2021.
- [26] keepitreal, *Vietnamese-sbert*, <https://huggingface.co/keepitreal/vietnamese-sbert>.
- [27] P.-N. Dang, K.-L. Nguyen, and T.-H. Pham, *ViRanker: A BGE-M3 and block-wise parallel transformer cross-encoder for vietnamese reranking*, <https://huggingface.co/namdp-ptit/ViRanker>, 2025.
- [28] itdainb, *PhoRanker: A cross-encoder reranker for vietnamese*, <https://huggingface.co/itdainb/PhoRanker>, 2024.
- [29] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2002.10957, 2020.

PHỤ LỤC A. MÔI TRƯỜNG VÀ CẤU HÌNH HUẤN LUYỆN

A.1 Môi trường phần cứng và phần mềm

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên nền tảng Kaggle Notebook. Việc huấn luyện embedding và reranker dùng GPU NVIDIA RTX 6000; việc đánh giá độ trễ bộ phân loại ý định dùng GPU NVIDIA T4; còn phép đo đầu cuối trên bài toán trắc nghiệm được thực hiện trong môi trường CPU và API trên máy cá nhân để phản ánh thiết lập triển khai thực tế.

Các thư viện chính được sử dụng gồm:

Table A.1: Các thư viện chính sử dụng trong đồ án

Thư viện	Vai trò
PyTorch	Khung học sâu nền tảng
sentence-transformers	Huấn luyện embedding (MNRL, GIST, MatryoshkaLoss) và SetFit
transformers (Hugging Face)	Tải và tinh chỉnh reranker (cross-encoder)
scikit-learn	Bộ phân loại Logistic Regression, đánh giá phân loại
Pyvi (ViTokenizer)	Tách từ tiếng Việt
NumPy	Tính toán số học

A.2 Siêu tham số huấn luyện embedding

Mô hình embedding được tinh chỉnh từ AITeamVN/Vietnamese_Embedding_v2 (dựa trên BGE-M3, 1024 chiều) theo lộ trình hai giai đoạn, mỗi giai đoạn bọc trong MatryoshkaLoss với ba kích thước $\{256, 512, 1024\}$.

Table A.2: Siêu tham số huấn luyện embedding

Tham số	Giai đoạn 1 (GIST)	Giai đoạn 2 (MNRL)
Số epoch	2	1
Tốc độ học	2×10^{-6}	5×10^{-7}
Kích thước batch	64	32
Số bước khởi động	5	5
Kích thước MRL	$\{256, 512, 1024\}$	
Độ dài chuỗi tối đa	512 token	
Nhiệt độ GIST	0.01	
Scale MNRL	20 ($\tau = 0.05$)	
Mô hình dẫn hướng GIST	BGE-M3	

A.3 Siêu tham số huấn luyện reranker

Reranker được huấn luyện theo hai giai đoạn: giai đoạn A (học tương phản, thích nghi miễn) và giai đoạn B (chung cất tri thức từ teacher BGE-reranker-v2-m3). Bộ tối ưu dùng AdamW, dừng sớm theo $MRR@10$ trên tập kiểm định.

Table A.3: Siêu tham số huấn luyện reranker

Tham số	Giai đoạn A	Giai đoạn B
Số epoch	5	5
Tốc độ học	2×10^{-5}	1×10^{-5}
Kích thước batch	32	16
Dừng sớm (patience)	2	2
Hạt giống ngẫu nhiên	42	42
Độ dài chuỗi tối đa	512 token	512 token
Trọng số chung cất α	–	0.7
Nhiệt độ Listwise KL τ	–	2.0
Số ứng viên mỗi câu hỏi	–	20
Bộ tối ưu	AdamW	

A.4 Siêu tham số huấn luyện bộ phân loại ý định

Bộ phân loại ý định dùng SetFit với encoder keepitreal/vietnamese-sbert (dựa trên PhoBERT-base, 768 chiều), tính chỉnh tương phản bằng CosineSimilarityLoss rồi huấn luyện bộ phân loại Logistic Regression trên embedding câu đã chuẩn hóa.

Table A.4: Siêu tham số huấn luyện bộ phân loại ý định

Tham số	Giá trị
Encoder nền	keepitreal/vietnamese-sbert
Số epoch (SetFit)	1
Kích thước batch	16
Tốc độ học	2×10^{-5}
Số vòng lặp sinh cặp	20
Tỉ lệ khởi động	0.1
Hàm mất mát tương phản	CosineSimilarityLoss
Bộ phân loại	Logistic Regression
Trọng số lớp	cân bằng (balanced)
Hệ số điều chuẩn C	4.0
Tách từ	Pyvi (ViTokenizer)
Hạt giống ngẫu nhiên	42

PHỤ LỤC B. VÍ DỤ DỮ LIỆU HUẤN LUYỆN

Phụ lục này minh họa cấu trúc dữ liệu huấn luyện đã mô tả ở Chương 3, gồm ví dụ bộ ba cho embedding, các loại negative tạo sinh, và ví dụ hai lớp ý định. Các đoạn văn dài được rút gọn để minh họa; dấu [...] biểu thị phần đã lược bớt.

B.1 Ví dụ bộ ba cho embedding

Mỗi câu hỏi được ghép với một positive (đoạn chứa câu trả lời) và các negative. Ví dụ dưới đây lấy từ miền tài liệu về vật liệu chống cháy.

Table B.1: Ví dụ một bộ ba huấn luyện embedding

Thành phần	Nội dung
Câu hỏi	Chiều dày màng khô của lớp phủ chống cháy được xác định bằng cách nào?
Positive	“Độ dày màng khô phải được xác định trực tiếp trên các mẫu thử nghiệm, sau khi lớp phủ được để khô hoàn toàn theo quy định của khách hàng. Phòng thử nghiệm phải đo chiều dày bằng dụng cụ sử dụng nguyên tắc cảm ứng điện từ hoặc nguyên tắc dòng điện xoáy với đường kính tiếp xúc đầu dò ít nhất là 2,5 mm. [...]”
Negative (tạo sinh)	“[...] đo chiều dày bằng dụng cụ [...] với đường kính tiếp xúc đầu dò ít nhất là 3 mm . [...]”
Negative (khai thác)	“Đối với vật liệu bọc phủ dạng phản ứng, độ dày trung bình của lớp sơn lót phải được đo trước và trừ đi từ tổng chiều dày [...]”

Trong ví dụ trên, negative tạo sinh gần như sao chép positive nhưng thay đổi một chi tiết kỹ thuật quan trọng, từ 2,5 mm thành 3 mm. Do đó, đoạn này vẫn rất gần positive về chủ đề và cấu trúc, nhưng tạo ra nhiều chi tiết so với đoạn đúng. Dạng negative này được dùng để giúp embedding học phân biệt các đoạn rất giống nhau nhưng khác ở thông tin then chốt.

Negative khai thác là một đoạn thật trong corpus, gần positive trong không gian truy hồi nhưng không trực tiếp cung cấp thông tin cần thiết để trả lời câu hỏi.

B.2 Các loại negative tạo sinh

Bốn phép chỉnh sửa được dùng để tạo negative, mỗi loại thay đổi một yếu tố quyết định khả năng trả lời.

Table B.2: Ví dụ bốn loại negative tạo sinh

Loại chỉnh sửa	Cách thay đổi (so với positive)
Thay đổi thực thể	Đổi một thực thể quan trọng: “theo quy định của khách hàng ” → “theo quy định của nhà cung cấp ”; hoặc “cảm biến nhiệt độ ” → “cảm biến độ ẩm ”.
Thay đổi thời gian	Đổi mốc thời gian hoặc năm: “(H. Lee và cộng sự, 2019)” → “(H. Lee và cộng sự, 2020)”.
Đảo ngược logic	Thêm hoặc đảo một điều kiện làm sai ý nghĩa gốc, ví dụ thêm một mệnh đề ràng buộc không có trong positive.
Thay đổi nguồn gốc	Đổi nguồn hoặc phạm vi áp dụng của thông tin, khiến đoạn dẫn chiếu sai tài liệu.

B.3 Ví dụ hai lớp ý định

Câu hỏi được phân thành hai lớp: tra_cuu (tra cứu thông tin có sẵn trong tài liệu) và tinh_toan (cần suy luận hoặc tính toán để tạo ra kết quả).

Table B.3: Ví dụ câu hỏi hai lớp ý định

Lớp	Ví dụ câu hỏi
tra_cuu	Chiều dày màng khô của lớp phủ chống cháy được xác định bằng cách nào?
tra_cuu	Trong mô hình nhà thông minh, IoT chủ yếu đóng vai trò gì?
tinh_toan	Trong tài liệu Public_002, đối với đoạn liệt kê các học phần từ số 47 đến 81, nếu các học phần này được trải đều trên 7 trang, trung bình mỗi trang mô tả bao nhiêu học phần?
tinh_toan	Tính chi phí nhiên liệu cho quãng đường 120 km, mức tiêu thụ 7 L/100 km, giá xăng 24.000 đồng/L?

Điểm khác biệt giữa hai lớp: câu tra_cuu có thể trả lời bằng cách trích thông tin trực tiếp từ tài liệu, trong khi câu tinh_toan yêu cầu thực hiện phép tính trên các số liệu lấy được (ví dụ đếm số học phần rồi chia cho số trang, hoặc tính tích quãng đường với mức tiêu thụ và đơn giá).

PHỤ LỤC C. CÁC PROMPT SỬ DỤNG VỚI MÔ HÌNH NGÔN NGỮ

Phụ lục này tập hợp các prompt dùng với mô hình ngôn ngữ trong hai vai trò: (1) vận hành hệ thống hỏi đáp (định tuyến ý định và sinh câu trả lời), và (2) xây dựng dữ liệu huấn luyện (xác định positive, sinh hard negative và kiểm tra hard negative). Các trường trong dấu ngoặc nhọn như `\{question\}` là chỗ điền nội dung động khi chạy.

C.1 Prompt định tuyến ý định

Prompt phân loại câu hỏi thành một trong hai lớp `tra_cuu` hoặc `inh_toan`.

Phân loại intent của câu hỏi sau. Trả lời chỉ một từ:

- `"tra_cuu"` nếu câu hỏi yêu cầu tra cứu, tìm kiếm thông tin,
→ định nghĩa, giải thích
- `"inh_toan"` nếu câu hỏi yêu cầu tính toán, suy luận số liệu,
→ so sánh theo phép tính

Câu hỏi: `{question}`

Intent:

C.2 Prompt sinh câu trả lời theo ý định

Với câu `tra_cuu`, hệ thống dùng một lượt gọi để chọn đáp án trực tiếp:

Dựa vào thông tin sau đây, trả lời câu hỏi trắc nghiệm.

CONTEXT:

`{context}`

QUESTION:

`{question}`

OPTIONS:

`{options_text}`

Chỉ trả lời đúng 1 dòng, chỉ ghi chữ cái đáp án (A/B/C/D).

Nếu có nhiều đáp án đúng thì ghi tất cả, ví dụ: AB hoặc ACD.

Không giải thích.

ANSWER:

Với câu `inh_toan`, hệ thống dùng hai lượt gọi. Lượt một sinh phân tích từng bước:

Dựa vào context sau, phân tích câu hỏi từng bước.

CONTEXT:

{context}

QUESTION:

{question}

OPTIONS:

{options_text}

Phân tích từng bước, giữ nguyên số liệu quan trọng.

Chưa cần kết luận đáp án cuối cùng:

Lượt hai chọn đáp án cuối từ phần phân tích:

Dựa vào phân tích sau, chọn đáp án đúng cho câu hỏi trắc nghiệm.

PHÂN TÍCH:

{reasoning}

QUESTION:

{question}

OPTIONS:

{options_text}

Chỉ trả lời đúng 1 dòng, chỉ ghi chữ cái đáp án (A/B/C/D).

Nếu có nhiều đáp án đúng thì ghi tất cả, ví dụ: AB hoặc ACD.

Không giải thích.

ANSWER:

C.3 Prompt giám khảo xác định positive

Prompt yêu cầu mô hình phân loại từng đoạn ứng viên thành gold, partial hoặc irrelevant, trả về kết quả dạng JSON.

Judge relevance cho RAG tiếng Việt.

Phân loại từng chunk:

- gold: đủ thông tin trả lời trực tiếp
- partial: liên quan nhưng không đủ
- irrelevant: không liên quan

JSON only:

```
{
  "gold_indices": [...],
  "partial_indices": [...],
  "irrelevant_indices": [...],
  "confidence": "high|medium|low"
}
```

Phần prompt người dùng kèm câu hỏi và danh sách đoạn:

QUERY: {question}

INTENT: {intent}

CÁC ĐOẠN VĂN:

[CHUNK_0] (bge_score={score})

{chunk_text}

...

Phân loại từng chunk.

C.4 Prompt sinh hard negative

Prompt yêu cầu mô hình tạo hai phiên bản sai lệch tinh vi từ đoạn gốc, mỗi phiên bản dùng một loại chỉnh sửa khác nhau.

Bạn tạo dữ liệu huấn luyện cho retrieval system.

Với đoạn văn GỐC và query, tạo 2 phiên bản sai lệch tinh vi.

Chọn 2 loại edit PHÙ HỢP NHẤT với nội dung,

mỗi version dùng 1 loại khác nhau:

- entity_swap: đổi thực thể lỗi (tên, điều khoản, đối tượng)
- temporal_edit: đổi số liệu, ngưỡng, mốc thời gian
- logic_flip: đảo ngược điều kiện, quan hệ nhân quả
- provenance_conflict: đổi phạm vi áp dụng, nguồn tài liệu

Yêu cầu BẮT BUỘC:

- SAO CHÉP TOÀN BỘ đoạn văn gốc, chỉ thay đổi 1-2 chi tiết nhỏ
- KHÔNG rút ngắn, KHÔNG tóm tắt, KHÔNG bỏ câu nào
- Độ dài version phải gần bằng đoạn gốc (+-10%)
- KHÔNG thêm câu giveaway như "tuy nhiên...", "thực ra..."
- Mỗi version dùng 1 loại edit khác nhau
- Tự kiểm tra: still_plausible, answers_query,
- contains_giveaway_style

C.5 Prompt kiểm tra hard negative

Prompt dùng một mô hình độc lập để xác nhận hard negative hợp lệ, tức đoạn không trả lời được câu hỏi.

Bạn là judge độc lập để kiểm tra khả năng trả lời của hard
→ negative
trong dữ liệu huấn luyện RAG tiếng Việt.

Nhiệm vụ: Với một QUERY và một CANDIDATE CHUNK,
quyết định xem đoạn này có trả lời được query hay không.

Trả về answers_query=true nếu đoạn văn chứa đủ thông tin
để trả lời trực tiếp query, hoặc nếu đáp án có thể được
suy ra rõ ràng từ đoạn văn.

Trả về answers_query=false nếu đoạn văn chỉ liên quan cùng chủ
→ đề,
có chi tiết sai/mâu thuẫn, thiếu thông tin then chốt,
hoặc không trả lời được query.

Hãy đánh giá nghiêm ngặt: hard negative hợp lệ là đoạn
KHÔNG trả lời được query.

JSON only:

{"answers_query": true|false, "confidence": "high|medium|low"}